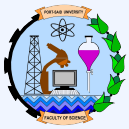


Port Said University



Faculty of Science



Department of Mathematics & Computer Science

Lecture Notes in

Advanced Programming

C412

Fourth Level

Information Technology and Computer Science

Prepared by

Prof. Dr. Mostafa Herajy

2026–2027

Last updated: Monday 17th August, 2026 at 10:58

Contents

Lecture 1: Introduction **11**
Core Concepts of OOP . . . 11
Classes and Objects . . . 11
Encapsulation . . . 11
Inheritance . . . 12
Polymorphism . . . 12
Abstraction . . . 13
Benefits of OOP . . . 13
Code Reusability . . . 13
Modularity . . . 13
Maintainability . . . 13
Scalability . . . 13
OOP in Different Programming Languages . . . 13
Java . . . 13
Python . . . 14
C++ . . . 14
Real-World Applications of OOP . . . 14
Software Development . . . 14
Game Development . . . 14
Data Science and AI . . . 14
Web Development . . . 14
Summary . . . 15

Lecture 2: Software Design using Object Oriented Programming **17**
Understanding Object-Oriented Thinking . . . 17
Identifying Real-World Entities as Objects . . . 17
Defining Classes and Objects . . . 17
Encapsulation and Information Hiding . . . 17
Inheritance and Reusability . . . 17
Polymorphism and Flexibility . . . 18
Designing Object-Oriented Software . . . 18
Step 1: Requirement Analysis . . . 18
Step 2: Identifying Key Objects and Classes . . . 18
Step 3: Defining Relationships Between Classes . . . 18
Step 4: Implementing Design Patterns . . . 18
Step 5: Writing Modular and Maintainable Code . . . 19
Unified Modeling Language (UML) . . . 19
Structural Diagrams . . . 19

Behavioral Diagrams . . . 21
Benefits of UML . . . 25
Summary . . . 26
Practice Lab: Object-Oriented Programming (OOP) Design . . . 26
Objective . . . 26
Step 1: Define the Classes and UML Design . . . 26
Step 2: UML Diagram (Class Diagram) . . . 27
Step 3: Reflection and Questions . . . 27
Practice Questions . . . 27

Lecture 3: C++ Classes **29**
Object Oriented Programming . . . 29
What are classes . . . 30
Relationship between Objects and classes . . . 33
Constructor: revisited . . . 34
Default Constructor: . . . 34
Parameterized Constructor: . . . 35
Copy Constructor: . . . 35
Importance of Initializer Lists in Constructors . . . 36
Initialization vs. Assignment . . . 36
Const and Reference Members . . . 36
Performance Considerations . . . 37
Initialization Order . . . 37
Base Class Initialization . . . 37
Summary . . . 37
Move Constructor . . . 38
Destructor: revisited . . . 39
Classes and Pointers . . . 41
Pointer to Objects: . . . 41
Passing Objects by Pointer: . . . 42
Dynamically Allocated Objects: . . . 42
Pointer to Class Members: . . . 42
Move Semantics in C++ . . . 43
Motivation . . . 43
Rvalue References . . . 43
Move Constructor and Move Assignment Operator . . . 43
When Are Moves Used? . . . 44
Advantages of Move Semantics . . . 45
Rule of Five . . . 45
Deleted and Defaulted Special Member Functions . . . 45
Special Member Functions . . . 45
Defaulted Functions . . . 46
Deleted Functions . . . 46
Common Use Cases . . . 46
Deleted Functions in Practice . . . 47
Combining Default and Delete . . . 47

Structures and Classes . . . 48
Structures: . . . 48
Classes: . . . 48
Static Members . . . 49
Static Member Variables . . . 49
Static Member Functions . . . 51
Constant Member Functions . . . 52
Lab 3: Introduction to C++ Classes . . . 54
Practice Questions . . . 55

Lecture 4: Inheritance and Polymorphism **57**
Base Class (Parent Class) . . . 57
Derived Class (Child Class) . . . 57
Access Specifiers . . . 58
Constructor and Destructor Inheritance . . . 58
Method Overriding: . . . 58
The Shape Example . . . 59
Multiple Inheritance . . . 60
Polymorphism . . . 63
Compile-Time Polymorphism (Static Polymorphism) . . . 63
Runtime Polymorphism (Dynamic Polymorphism) . . . 64
Shape drawing . . . 65
Abstract Classes . . . 66
Interface Classes . . . 68
Best practices and guidelines . . . 69
Summary . . . 70
Lab 4: Inheritance in C++ . . . 71
Questions . . . 72

Lecture 5: Operator Overloading **75**
Introduction to Operator Overloading . . . 75
Overloading Unary Operators . . . 76
Overloading Binary Operators . . . 77
Overloading Comparison Operators . . . 79
Overloading Subscript Operator (`[]`) . . . 80
Overloading Function Call Operator (()) . . . 82
Overloading Type Conversion Operators . . . 83
Common Pitfalls and Best Practices . . . 84
Summary . . . 86
Practice Lab . . . 87
Questions . . . 88

Lecture 6: C++ Templates: Generic and Reusable Programming **91**
Introduction . . . 91
Why Templates? Motivation and Benefits . . . 91
Function Templates . . . 91

Template Parameters . . . 92
Class Templates . . . 92
Template Specialization . . . 92
Non-Type Template Parameters . . . 93
Templates and Inheritance . . . 93
Templates and Friends . . . 94
Templates and Static Members . . . 94
Common Compilation Errors . . . 94
Modern Extensions in C++ Template Programming . . . 95
Templates in C++11: The Foundation of Modern Generic Programming . . . 95
C++14: Simplicity and Refinement . . . 96
C++17: Expressive Compile-Time Programming . . . 96
C++20: Concepts and Constraints . . . 97
C++23: Refinement and Polishing . . . 98
Summary of Modern Template Features . . . 99
Lab: Template Programming in C++ . . . 99
Objectives . . . 99
Pre-Lab Reading . . . 99
Lab Setup . . . 100
Part 1: Function Templates . . . 100
Part 2: Class Templates . . . 100
Part 3: Non-Type Template Parameters . . . 101
Part 4: Template Specialization . . . 102
Part 5: Modern Template Features (C++17 and Beyond) . . . 103
Lab Report and Discussion . . . 103
Practice Questions on C++ Templates . . . 104

Lecture 7: Design Patterns **107**
Creational Patterns . . . 107
Singleton Pattern . . . 107
Factory Method Pattern . . . 109
Abstract Factory Pattern . . . 111
Builder Pattern . . . 113
Structural Patterns . . . 116
Adapter Pattern . . . 116
Decorator Pattern . . . 118
Composite Pattern . . . 120
Proxy Pattern . . . 122
Behavioral Patterns . . . 124
Observer Pattern . . . 124
Strategy Pattern . . . 127
Command Pattern . . . 129
State Pattern . . . 131
Practice Lab . . . 133

Lecture 8: Standard Template Library (STL) **135**
What is the Standard Template Library (STL)? . . . 135
Key Components of the STL . . . 135
Containers . . . 135
Algorithms . . . 135
Iterators . . . 135
Function Objects (Functors) . . . 136
Allocators . . . 136
Benefits of Using the STL . . . 136
Code Reusability . . . 136
Standardization . . . 136
Performance . . . 136
Safety . . . 136
Using the STL in Your Code . . . 136
Example of STL Containers . . . 137
Vectors . . . 137
Arrays . . . 139
List . . . 141
Set . . . 142
Maps . . . 144
Pairs . . . 146
Tuples . . . 147
Queues . . . 149
Optional . . . 150
Example of STL Algorithms . . . 151
Find . . . 151
Find_if . . . 153
for_each . . . 154
Count . . . 156
`std::all_of` . . . 157
`std::any_of` . . . 157
`std::none_of` . . . 158
`std::copy` . . . 158
`std::copy_if` . . . 159
`std::move` . . . 159
`std::partition` . . . 160
Summary of STL Containers and Algorithms . . . 160
STL Containers . . . 160
STL Algorithms . . . 161
Iterators . . . 163
Advantages of STL . . . 163
Best Practices . . . 164
Lab 9: Exploring C++ Standard Library Containers . . . 164
Practice Questions . . . 165

Lecture 9: Error and Exception Handling in C++ **169**
Introduction . . . 169
Understanding Errors in C++ . . . 169
Traditional Error Handling in C . . . 169
Limitations of Traditional Error Handling . . . 170
The Exception Handling Model in C++ . . . 170
Mechanics of Exception Handling . . . 171
The `throw` Statement . . . 171
Using `try` and `catch` . . . 171
Exception Class Hierarchy . . . 172
User-Defined Exception Classes . . . 172
Stack Unwinding and RAII . . . 172
Catching by Value, Reference, or Pointer . . . 173
Exception Safety . . . 173
The `noexcept` Specifier . . . 173
Rethrowing Exceptions . . . 173
Exceptions in Constructors and Destructors . . . 174
Assertions in C++ . . . 174
When to Use Assertions . . . 174
Standard Library and Exceptions . . . 175
Performance Considerations . . . 175
Alternatives to Exceptions . . . 175
Best Practices . . . 176
Summary . . . 176
Practice Lab: Exception Handling in C++ . . . 176
Objectives . . . 176
Prerequisites . . . 177
Required Tools . . . 177
Background Theory . . . 177
Exercises . . . 177
Exercise 1: Basic Exception Handling . . . 177
Exercise 2: Multiple Catch Blocks . . . 178
Exercise 3: Custom Exception Class . . . 178
Exercise 4: Stack Unwinding . . . 179
Exercise 5: RAII and Resource Safety . . . 180
Exercise 6: Assertions . . . 180
Discussion Questions . . . 181
Challenge Task . . . 181
Summary . . . 181
Questions . . . 181
Practice Questions on Exception Handling in C++ . . . 181
Part A: Conceptual Questions . . . 182
Part B: Code Analysis Questions . . . 182
Part C: Programming Tasks . . . 183
Part D: Discussion and Reflection . . . 184
Part E: Challenge Problems . . . 184

Lecture 10: Files and Streams in C++ **185**
Understanding Streams . . . 185
What Are Streams? . . . 185
Stream Classes . . . 185
Stream States and Flags . . . 186
File Input and Output . . . 186
Opening and Closing Files . . . 186
Reading from Files . . . 187
Writing to Files . . . 187
Error Handling . . . 188
Checking for Errors . . . 188
Exception Handling . . . 188
Working with Binary files . . . 188
Writing Binary Data to a File . . . 188
Reading Binary Data from a File . . . 189
Binary File Considerations . . . 190
Lab 10: C++ Streams: Text and Binary File Operations . . . 191
Practice Questions . . . 192

Lecture 11: Writing Big Programs **195**
Header and Implementation Files . . . 195
Header Files (.h or .hpp) . . . 195
Implementation Files (.cpp) . . . 195
Example . . . 196
Forward Declarations in C++ . . . 197
Concept of Forward Declaration . . . 197
Why Use Forward Declarations? . . . 198
Example: Using Forward Declarations Between Classes . . . 198
Forward Declaration of Functions . . . 199
Forward Declarations of Structs and Enums . . . 199
Limitations of Forward Declarations . . . 200
Best Practices . . . 200
Benefits in Large Projects . . . 200
Summary . . . 200
Precompiled Headers (PCH) . . . 201
Concept of Precompiled Headers . . . 201
Advantages of Using Precompiled Headers . . . 201
How Precompiled Headers Work . . . 202
Creating and Using a Precompiled Header . . . 202
Best Practices for Precompiled Headers . . . 203
Limitations and Potential Issues . . . 203
Practical Recommendations . . . 204
Example Build Times . . . 204
Summary . . . 204
Static and Dynamic Libraries in C++ . . . 204
What Is a Library? . . . 205

Static Libraries . . . 205
Dynamic (Shared) Libraries . . . 207
Static vs Dynamic Linking: Comparison . . . 208
When to Use Each Type . . . 208
Best Practices . . . 208
Make and CMake . . . 208
Make . . . 208
Example Makefile . . . 209
Using the Makefile . . . 210
CMake . . . 210

Further Reading in C++ **213**
Recommended Topics for Continued Study . . . 213
Influential Books and References . . . 214
Online Resources . . . 214
Looking Ahead . . . 214

Suggested Projects **215**
Project 1: Analog Clock in C++ . . . 215
Overview . . . 215
Objectives . . . 215
System Components . . . 216
Rotating a Point to Draw Clock Hands . . . 217
Computing Hand Angles . . . 217
Expected Learning Outcomes . . . 217
Project 2: Slide Show Application in C++ . . . 218
Overview . . . 218
Objectives . . . 218
System Components . . . 218
Program Workflow . . . 219
Possible Extensions . . . 219
Project 3: Binary Parser in C++ . . . 220
Overview . . . 220
Objectives . . . 220
Binary File Format Specification . . . 220
Core Functionalities . . . 221
Optional Extensions . . . 221
Project 4: Puzzle Game: Mind Maze . . . 221
Project Overview . . . 221
Objectives . . . 221
Key Features . . . 222
OOP Design and Implementation . . . 222
Project 5: Chat Program: TCP Talk . . . 223
Project Overview . . . 223
Objectives . . . 223
Key Features . . . 223

OOP Design and Implementation 223
Sample TCP Socket Communication 224

Lecture 1: Introduction

Object-Oriented Programming (OOP) is a programming paradigm that uses objects and classes to design and develop software applications. It provides a way to model real-world entities using attributes and behaviors, making it a powerful approach for building scalable and maintainable software systems. OOP enhances code reusability, modularity, and abstraction, which are crucial for modern software development.

Core Concepts of OOP

Classes and Objects

A class is a blueprint for creating objects, defining the structure and behavior of an entity. An object is an instance of a class that contains data (attributes) and methods (functions) that operate on the data.

For example, a class Car can have attributes like color, model, and speed, and methods like accelerate() and brake().

Encapsulation

Encapsulation refers to the bundling of data and methods that operate on the data into a single unit (class). It restricts direct access to some of an object's components, ensuring data integrity and security.

In most OOP languages, encapsulation is implemented using access specifiers such as public, private, and protected. For example, in Java:

```
class Car {
    private String color;

    public void setColor(String c) {
        color = c;
    }

    public String getColor() {
        return color;
    }
}
```

Here, color is a private variable that can only be accessed through public methods.

Inheritance

Inheritance allows a class (subclass) to inherit properties and behaviors from another class (superclass). It promotes code reusability and establishes a hierarchical relationship between classes.

For instance, if a class Vehicle has properties like speed and fuel, a subclass Car can inherit these properties while adding its specific features.

```
class Vehicle {
    int speed;
    void move() {
        System.out.println("Vehicle is moving");
    }
}
class Car extends Vehicle {
    void horn() {
        System.out.println("Car horn");
    }
}
```

Polymorphism

Polymorphism enables a single interface to be used for different types. It can be achieved through method overloading (compile-time polymorphism) and method overriding (runtime polymorphism).

Method Overloading: Multiple methods with the same name but different parameters.

```
class MathOperation {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
}
```

Method Overriding: A subclass redefines a method from the superclass.

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    void makeSound() {
        System.out.println("Bark");
    }
}
```

Abstraction

Abstraction is the process of hiding implementation details and exposing only essential features. It is achieved using abstract classes and interfaces.

```
abstract class Animal {
    abstract void makeSound();
}
class Cat extends Animal {
    void makeSound() {
        System.out.println("Meow");
    }
}
```

Benefits of OOP

Code Reusability

By using inheritance and polymorphism, developers can reuse existing code, reducing redundancy and improving maintainability.

Modularity

Classes and objects promote modular development, allowing different components of an application to be developed and tested independently.

Maintainability

Encapsulation ensures that changes in one part of the code do not affect other parts, making software easier to maintain and extend.

Scalability

OOP supports the development of complex and large-scale applications by breaking them into manageable components.

OOP in Different Programming Languages

Java

Java is one of the most widely used OOP languages, known for its robustness and platform independence.

```
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Python

Python provides a simple and flexible implementation of OOP principles.

```
class Animal:
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        print("Bark")
```

C++

C++ supports OOP with additional features like multiple inheritance.

```
class Vehicle {
public:
    void move() {
        cout << "Vehicle is moving";
    }
};
class Car : public Vehicle {
};
```

Real-World Applications of OOP

Software Development

Most modern software, including web applications, enterprise software, and mobile applications, use OOP principles.

Game Development

OOP is widely used in game development, where objects like Player, Enemy, and Weapon interact with each other.

Data Science and AI

OOP helps organize complex data structures in machine learning and artificial intelligence models.

Web Development

Frameworks like Django (Python) and Spring (Java) use OOP to manage web applications efficiently.

Summary

Object-Oriented Programming (OOP) is a programming paradigm that structures software design around objects and classes. It enhances modularity, reusability, and maintainability through key principles: encapsulation, inheritance, polymorphism, and abstraction. Encapsulation secures data within classes, inheritance allows code reuse by enabling classes to derive properties from others, polymorphism enables flexibility through method overriding and overloading, and abstraction simplifies complex systems by exposing only essential details. OOP is widely used in languages like Java, Python, and C++ and is fundamental in software development, game design, AI, and web development. Its structured approach ensures scalable and efficient software solutions.

Lecture 2: Software Design using Object Oriented Programming

Thinking in an object-oriented manner requires a shift in perspective from procedural programming to modeling real-world interactions. Designing object-oriented software involves careful analysis of entities, relationships, and behaviors, followed by the application of proven methodologies and patterns. By following best practices, developers can create robust, scalable, and maintainable software solutions that stand the test of time.

Object-oriented software design is a systematic approach to structuring applications using objects, classes, and their relationships. It enables modularity, reusability, scalability, and maintainability in software development. To design effective object-oriented software, developers must understand core principles, apply design methodologies, and use best practices to create flexible and efficient systems.

Understanding Object-Oriented Thinking

Identifying Real-World Entities as Objects

The first step in designing object-oriented software is to model real-world entities as objects. Each object represents a unique entity with specific attributes (data) and behaviors (methods). For example, in a banking application, entities such as Customer, Account, and Transaction can be represented as objects.

Defining Classes and Objects

A class is a blueprint for creating objects. It defines the properties and behaviors that its objects will have. An object is an instance of a class. When defining classes, it is crucial to focus on encapsulating data and providing controlled access through methods.

Encapsulation and Information Hiding

Encapsulation refers to bundling an object's data and behavior into a single unit while restricting direct access to some components. This ensures data integrity and security. By exposing only necessary details through public methods, developers can hide implementation details and reduce dependencies between components.

Inheritance and Reusability

Inheritance allows a new class (subclass) to derive attributes and methods from an existing class (superclass). This promotes code reuse and helps maintain a hierarchical structure. For instance, a Vehicle class can serve as a parent class for Car, Bike, and Truck classes, reducing redundancy in the code.

Polymorphism and Flexibility

Polymorphism allows objects to be treated as instances of their parent class, enabling code flexibility. It supports method overriding (redefining a method in a subclass) and method overloading (having multiple methods with the same name but different parameters). This facilitates scalable software design by allowing the same interface to handle different behaviors.

Designing Object-Oriented Software

Step 1: Requirement Analysis

Before writing code, it is essential to gather requirements and identify the core functionalities of the system. This involves understanding user needs, defining use cases, and establishing the system's scope.

Step 2: Identifying Key Objects and Classes

Once the requirements are clear, identify the main objects involved in the system. These objects should correspond to real-world entities or abstract components required for functionality. Define their attributes and behaviors to model the application accurately.

Step 3: Defining Relationships Between Classes

After identifying classes, establish relationships between them. Common relationships include:

1. Association: A class is connected to another, e.g., a Customer has an Account.
2. Aggregation: A class contains another class but both can exist independently, e.g., a Library has Books.
3. Composition: A class is dependent on another for existence, e.g., a Car has an Engine that cannot exist separately.
4. Inheritance: A subclass derives properties from a superclass, e.g., a Dog is an Animal.

Step 4: Implementing Design Patterns

Design patterns provide reusable solutions to common design problems. Some popular patterns include:

- Singleton: Ensures a class has only one instance.
- Factory Method: Creates objects without specifying the exact class.
- Observer: Allows multiple objects to listen for state changes in another object.
- Strategy: Defines a family of algorithms and encapsulates them to allow interchangeability.

Step 5: Writing Modular and Maintainable Code

Break the system into smaller, self-contained modules that interact with each other through well-defined interfaces. Follow principles like:

- SOLID Principles (Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion).
- DRY (Don't Repeat Yourself) to avoid redundant code.
- KISS (Keep It Simple, Stupid) for ease of understanding and modification.

Unified Modeling Language (UML)

Unified Modeling Language (UML) is a standardized visual language used in software engineering to model and design complex systems. It provides a set of graphical notations that help developers, architects, and stakeholders understand, document, and communicate system structures and behaviors effectively. UML is widely used in object-oriented software development to represent various aspects of a system, from high-level architecture to detailed class interactions.

UML consists of several diagrams categorized into structural and behavioral diagrams. Each type serves a specific purpose in modeling different aspects of a system.

Structural Diagrams

Structural diagrams represent the static aspects of a system, including its components, relationships, and organization.

Class Diagram

A class diagram is a fundamental component of Unified Modeling Language (UML) that visually represents the structure of a system by illustrating its classes, attributes, methods, and the relationships among them. It provides a blueprint for developers to design and implement object-oriented software efficiently. Each class in a diagram consists of three compartments: the class name at the top, attributes in the middle, and methods at the bottom. The relationships between classes are depicted using associations, generalizations, and dependencies. Associations define how objects of different classes interact, while generalization represents inheritance where a subclass acquires properties from a parent class. Dependencies indicate how one class relies on another for its functionality. Visibility markers such as public (+), private (-), and protected (#) specify access control for attributes and methods, ensuring encapsulation and security. Class diagrams also include multiplicities to describe how many instances of one class can be associated with another. They are widely used in system design and serve as a crucial tool in both software development and documentation. By modeling real-world entities and their interactions, class diagrams facilitate clear communication between team members and help create scalable, maintainable, and well-structured software architectures.

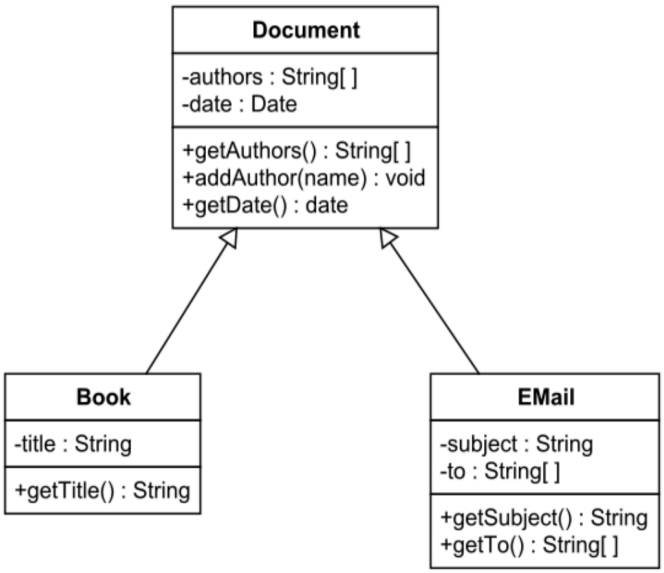


Figure 1: A class diagram that represents an inheritance hierarchy where the Document class serves as the parent (superclass), and Book and EMail are subclasses that extend its functionality.

Object Diagram

An object diagram is a specific instance of a class diagram that represents the state of a system at a particular moment. While class diagrams provide a static structure of a system, object diagrams illustrate how objects interact in runtime by showing instances of classes and their relationships. Each object is depicted with an underlined name indicating the instance and its corresponding class. Attributes within objects reflect real-time values, helping developers understand how data flows within the system. Object diagrams are particularly useful in validating class diagrams by demonstrating how objects of various classes are linked and instantiated. They help in identifying potential design flaws and ensuring logical consistency. These diagrams are beneficial in debugging, testing, and explaining system behaviors, especially in dynamic and complex applications. By capturing snapshots of system states, object diagrams assist in comprehending object interactions, making them an essential tool in object-oriented analysis and design.

Component Diagram

A component diagram is a type of UML diagram that visually represents the organization and dependencies of various software components in a system. It focuses on how components interact with each other and how they are connected through interfaces and dependencies. Each component is depicted as a self-contained module with clearly defined interfaces, facilitating a modular approach to software design. The diagram helps illustrate relationships such as dependencies, aggregations, and associations between components, ensuring that different parts of a system can function independently while still working together. Component diagrams are particularly useful in large-scale software systems where different teams develop separate modules. They enhance maintainability,

reusability, and scalability by clearly defining component interactions. By providing a high-level view of a system's architecture, component diagrams assist software architects and developers in designing robust, well-structured, and modular applications.

Deployment Diagram

A deployment diagram is a UML diagram that models the physical deployment of software components onto hardware nodes. It represents the system's runtime architecture by illustrating the relationships between hardware and software elements. Deployment diagrams consist of nodes, which represent physical devices or execution environments, and artifacts, which are the software components that get deployed. These artifacts can include executables, libraries, databases, or system configurations. Communication between nodes is represented by associations, showing how different hardware components interact over a network. Deployment diagrams are particularly useful for visualizing distributed systems, cloud-based applications, and enterprise architectures. They help in understanding system performance, scalability, and fault tolerance by showing how different parts of a system are deployed across infrastructure components. By providing a clear view of system topology, deployment diagrams assist developers, system administrators, and architects in designing efficient and maintainable deployment strategies.

Package Diagram

A package diagram is a UML diagram that represents the organization and dependencies of packages within a system. It helps in structuring large and complex software applications by grouping related elements into packages, reducing complexity and improving maintainability. Packages are depicted as folders containing various UML elements such as classes, components, and other packages. Dependencies between packages are represented by dashed arrows, showing how one package relies on another. Package diagrams are beneficial in modular system design, as they promote encapsulation and better organization of software components. They also aid in understanding system dependencies, allowing developers to manage dependencies efficiently and minimize coupling between modules. By providing a high-level view of the system's architecture, package diagrams facilitate team collaboration and enhance software scalability and reusability.

Behavioral Diagrams

Behavioral diagrams in UML represent the dynamic aspects of a system, showing how objects interact over time to produce the desired functionality. These diagrams help in understanding the flow of control, data, and interactions between different system elements. The primary types of behavioral diagrams include use case diagrams, sequence diagrams, activity diagrams, state diagrams, and communication diagrams. Use case diagrams illustrate the interactions between users (actors) and the system, providing a high-level view of system functionality. Sequence diagrams depict the chronological order of interactions between objects, showing the flow of messages over time. Activity

diagrams model workflows, illustrating the flow of actions, decisions, and concurrency in a process. State diagrams represent the various states an object can be in and the transitions between these states based on events. Communication diagrams, similar to sequence diagrams, emphasize object interactions and the exchange of messages in a structured manner. Behavioral diagrams are essential for analyzing system behavior, identifying potential issues, and designing effective interactions between system components, ensuring a well-structured and efficient implementation of software applications.

Use Case Diagram

A use case diagram is a type of UML diagram that visually represents the interactions between users (actors) and a system. It provides a high-level overview of system functionality by identifying different use cases and the roles of actors. Actors can be users, external systems, or devices that interact with the system. Each use case represents a specific functionality or process within the system, depicted as an oval shape with a descriptive name. The relationships between actors and use cases are illustrated using association lines. Additional relationships, such as include and extend, help in refining use cases by specifying dependencies or optional interactions. Use case diagrams are essential for gathering requirements, as they help stakeholders understand system behavior without requiring technical expertise. They also serve as a foundation for further design and development by clearly defining system functionalities and user interactions. By providing a clear and concise representation of user interactions, use case diagrams aid in designing user-friendly and well-structured software applications.

Sequence Diagram

A sequence diagram is a UML diagram that illustrates how objects interact in a particular scenario by showing the sequence of messages exchanged over time. It provides a detailed view of the interactions between different system components, making it an essential tool for modeling real-time behavior and communication. Sequence diagrams consist of objects, represented by rectangular boxes, with vertical lifelines extending downward to indicate the lifespan of each object during the interaction. Messages are depicted as horizontal arrows between lifelines, showing the flow of communication between objects. Solid arrows represent synchronous messages, where the sender waits for a response, while dashed arrows indicate asynchronous messages, where the sender does not wait for a reply. Activation bars on lifelines represent the duration an object is actively performing an operation. Sequence diagrams are widely used in system design to visualize process execution, analyze system behavior, and ensure correct object interactions. By clearly depicting message exchanges and control flows, they help developers understand, refine, and implement system functionalities efficiently.

Activity Diagram

An activity diagram is a UML behavioral diagram that represents the workflow of a system, illustrating the sequence of activities, decisions, and concurrent operations. It provides a high-level view of process execution, making it useful for modeling business

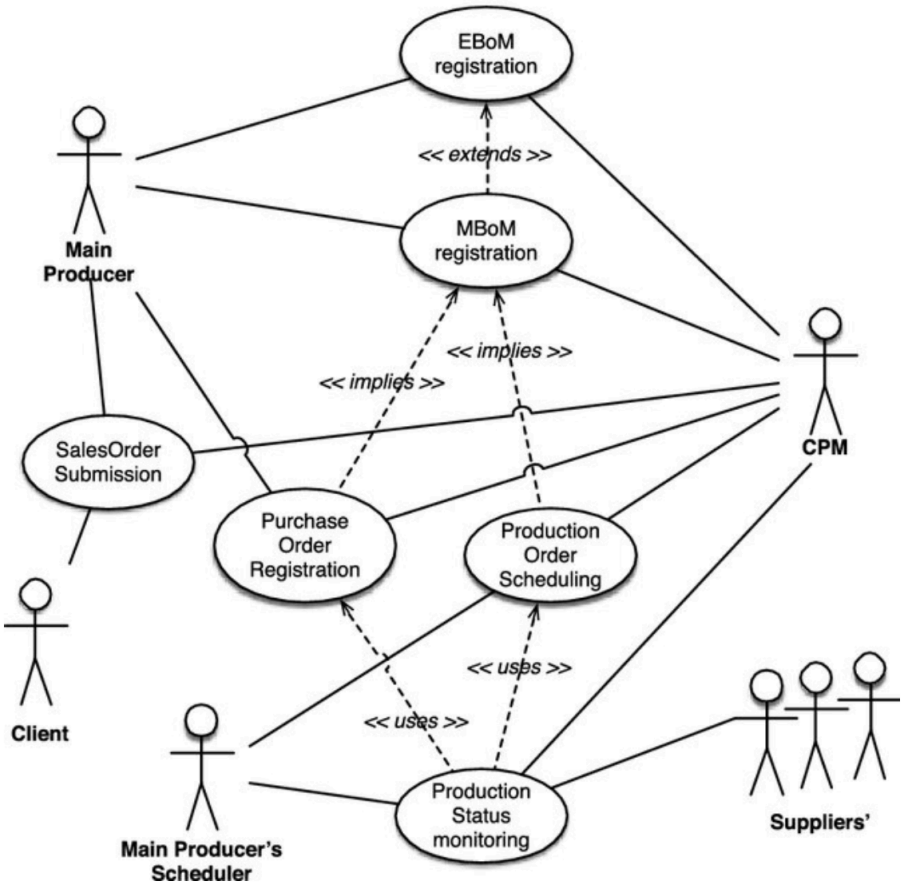


Figure 2: A UML Use Case Diagram that represents the interactions between different actors and use cases in a production management system.

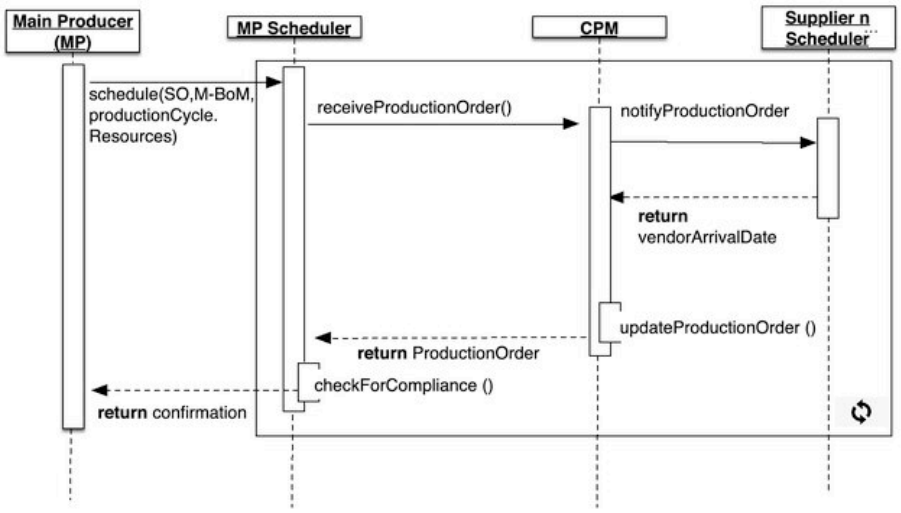


Figure 3: An example of a UML Sequence Diagram that models the interaction between different system components during a production scheduling process. The diagram consists of four main entities: (1) Main Producer (MP) – The primary entity responsible for initiating the scheduling process. (2) MP Scheduler – The component responsible for handling and processing production orders. (3) CPM – A coordinating system that ensures the proper flow of production requests. (4) Supplier n Scheduler – Represents an external supplier responsible for handling orders from the main producer.

processes, software workflows, and system operations. Activity diagrams consist of various elements, including actions (represented as rectangles with rounded edges), decision nodes (diamond shapes indicating branching points), and control flows (arrows showing the direction of execution). They also incorporate swimlanes to assign responsibilities to different actors or system components. Fork and join nodes are used to model parallel processes, showing concurrent execution paths within a workflow. Activity diagrams help developers analyze complex workflows, optimize process efficiency, and identify potential bottlenecks in system operations. By offering a clear and structured representation of process logic, activity diagrams facilitate better communication among stakeholders and contribute to the development of well-organized, efficient, and scalable software solutions.

State Diagram

A state diagram is a UML behavioral diagram that represents the different states an object can be in and the transitions between these states based on events. It provides a clear visualization of how an object responds to internal and external stimuli throughout its lifecycle. State diagrams consist of states (represented as rounded rectangles), transitions (arrows showing movement between states), and events (labels on transitions that trigger state changes). Some states may have entry and exit actions, defining what happens when an object enters or leaves a state. Initial and final states are depicted as filled circles and a bullseye, respectively. State diagrams are particularly useful in modeling real-time systems, embedded systems, and workflow-driven applications where understanding

state changes is crucial. By mapping out an object's behavior, state diagrams help developers analyze system logic, detect potential inconsistencies, and refine system interactions for efficient execution

Collaboration Diagram

A collaboration diagram, also known as a communication diagram, is a UML interaction diagram that illustrates how objects within a system collaborate to achieve a specific task. Unlike sequence diagrams, which emphasize the order of messages, collaboration diagrams focus on the relationships and interactions between objects. Objects are represented as rectangular nodes, connected by association lines that indicate communication pathways. Each interaction between objects is labeled with a numbered message, representing the sequence in which interactions occur. These diagrams provide a structural perspective of object interactions, making it easier to understand the roles and responsibilities of different system components. Collaboration diagrams are particularly useful for modeling complex interactions where multiple objects participate in achieving system functionality. They help developers design efficient object-oriented solutions by clarifying the relationships between objects and optimizing communication flows. By visualizing object collaboration, these diagrams enhance system comprehension, facilitate debugging, and contribute to the creation of well-organized software architectures.

Benefits of UML

- Standardization: Provides a universal way to represent software architectures.
- Improved Communication: Bridges the gap between technical and non-technical stakeholders.
- Better Documentation: Serves as a reference for future modifications and enhancements.
- Simplifies Complexity: Helps break down large systems into manageable components.
- Supports Various Development Processes: Can be used in Agile, Waterfall, and other methodologies.

UML is widely used in various phases of software development:

- Requirement Analysis: Use case diagrams help define functional requirements.
- System Design: Class and component diagrams assist in structuring the architecture.
- Implementation: Sequence and activity diagrams guide coding.
- Testing and Maintenance: State and collaboration diagrams help in debugging and optimizing the system.

Summary

OOP design is a software development approach that focuses on modeling real-world entities using objects, encapsulating data and behavior within them. It emphasizes four key principles: encapsulation, inheritance, polymorphism, and abstraction. Encapsulation ensures data security by restricting direct access to object attributes and requiring interactions through defined methods. Inheritance enables code reuse by allowing new classes to derive properties and behaviors from existing ones, reducing redundancy. Polymorphism allows objects to take multiple forms, enhancing flexibility and maintainability in code. Abstraction simplifies complexity by exposing only essential details while hiding implementation specifics.

Effective OOP design involves identifying objects, defining their attributes and methods, and establishing relationships between them. It encourages modularity, making software systems easier to manage, scale, and debug. Design patterns such as Singleton, Factory, and Observer further optimize OOP development by providing reusable solutions to common problems. Unified Modeling Language (UML) plays a crucial role in OOP design by offering standardized diagrams such as class, sequence, and use case diagrams to visualize system architecture and interactions.

By leveraging OOP principles and methodologies, developers create structured, efficient, and maintainable software applications that align with modern development practices.

Practice Lab: Object-Oriented Programming (OOP) Design

You have been hired to design an Online Banking System that allows customers to open accounts, deposit and withdraw money, and check their balance. You will implement this system using OOP principles and represent it using UML diagrams.

Objective

- Understand and apply OOP principles (Encapsulation, Inheritance, Polymorphism, and Abstraction).
- Design and implement a real-world system using OOP.
- Create UML diagrams to model the system.

Step 1: Define the Classes and UML Design

Before coding, create a class diagram that includes:

1. BankAccount (Base Class) Attributes: accountNumber, balance Methods: deposit(), withdraw(), getBalance()
2. SavingsAccount (Subclass of BankAccount) Additional attribute: interestRate Method: applyInterest()

3. CheckingAccount (Subclass of BankAccount) Additional attribute: overdraftLimit Overridden withdraw() method to consider overdraft.
4. Customer (Class) Attributes: name, customerID, accounts (list of BankAccount) Methods: openAccount(), closeAccount(), getAccounts()
5. Bank (Class) Attributes: customers (list of Customer) Methods: addCustomer(), removeCustomer(), getCustomerByID()

Step 2: UML Diagram (Class Diagram)

Draw a class diagram representing the system before implementing the code. Use tools like draw.io, Lucidchart, or UMLet.

- Classes: Represented as boxes with attributes and methods.
- Relationships: Use inheritance (solid line with empty triangle) between BankAccount and its subclasses.
- Associations: Use lines to connect Customer with BankAccount (one-to-many relationship).

Step 3: Reflection and Questions

- How does inheritance simplify the design in this banking system?
- What benefits does polymorphism provide in handling withdraw() differently for different accounts?
- How does encapsulation ensure data security?
- What role does a UML class diagram play before coding?
- How would you improve this system by adding more real-world features?

Practice Questions

1. Explain the four key principles of OOP: Encapsulation, Inheritance, Polymorphism, and Abstraction, with real-world examples.
2. How does encapsulation improve software security and maintainability? Provide an example in Java or Python.
3. What is the difference between class and object? Illustrate with an example.
4. Describe the concept of polymorphism and differentiate between method overloading and method overriding.
5. What are design patterns? Explain the Singleton and Factory design patterns with examples.

6. How does the principle of inheritance promote code reusability? Give an example.
7. What is the role of interfaces and abstract classes in OOP design? How do they differ?
8. Discuss the advantages of OOP over procedural programming.
9. What is the SOLID principle in OOP? Explain each of the five principles with an example.
10. How would you design a simple object-oriented system for an online bookstore? Identify key classes and their relationships.
11. What is UML, and why is it used in software development?
12. Describe the purpose of a class diagram and its key components.
13. Draw a class diagram for a library management system with classes such as Book, Member, and Librarian.
14. What is the difference between an object diagram and a class diagram? Provide an example.
15. How does a component diagram help in modeling large software systems?
16. Describe the use of a deployment diagram in system architecture. Provide an example.
17. What are behavioral diagrams in UML? Name at least three types and explain their purposes.
18. How does a use case diagram help in gathering software requirements? Provide an example.
19. Draw a sequence diagram for an online shopping checkout process.
20. Compare activity diagrams and state diagrams. When would you use each?

Lecture 3: C++ Classes

Object Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that is centered around the concept of "objects." It's a way of designing software by organizing code into objects that represent real-world entities and modeling their interactions. OOP is based on several key principles:

- Objects: Objects are instances of classes, and they encapsulate both data (attributes or properties) and behavior (methods or functions). These objects represent real-world entities and can interact with each other.
- Classes: Classes are blueprints or templates for creating objects. They define the structure and behavior that objects of the class will have. A class specifies the attributes and methods that its objects will possess.
- Encapsulation: Encapsulation is the concept of bundling data (attributes) and methods (functions) that operate on that data into a single unit, i.e., an object. It hides the internal details of how an object works, exposing only the necessary interfaces.
- Inheritance: Inheritance allows you to create a new class (a derived or child class) based on an existing class (a base or parent class). The derived class inherits the attributes and methods of the base class, enabling code reuse and creating a hierarchical structure of classes.
- Polymorphism: Polymorphism enables objects of different classes to be treated as objects of a common base class. It allows you to write code that can work with objects of various types, making the code more flexible and adaptable.
- Abstraction: Abstraction is the process of simplifying complex systems by breaking them down into smaller, more manageable parts. In OOP, it involves defining abstract classes or interfaces that define a set of methods without providing their implementation.
- OOP promotes code reusability, modularity, and a more intuitive way of thinking about and structuring software, making it a widely used and powerful programming paradigm. It is commonly employed in languages like Java, C++, Python, and many others.

Object-Oriented Programming (OOP) is guided by several key principles that help developers design and structure their software in a modular and maintainable way. The

five fundamental principles of OOP are often referred to as SOLID, an acronym formed from the first letter of each principle:

- Single Responsibility Principle (SRP): This principle states that a class should have only one reason to change, meaning it should have a single responsibility or purpose. A class should encapsulate a single piece of functionality, and if it has multiple responsibilities, it may become difficult to maintain and modify.
- Open/Closed Principle (OCP): The Open/Closed Principle emphasizes that software entities (e.g., classes, modules, functions) should be open for extension but closed for modification. Instead of altering existing code to add new features, you should extend or inherit from existing classes and build upon their functionality through new code.
- Liskov Substitution Principle (LSP): The Liskov Substitution Principle states that objects of a derived class should be able to replace objects of the base class without affecting the correctness of the program. this principle ensures that derived classes maintain the same contract (i.e., behavior) as their base classes, allowing for polymorphism and substitutability.
- Interface Segregation Principle (ISP): The Interface Segregation Principle suggests that clients should not be forced to depend on interfaces they do not use. t encourages the creation of smaller, more specific interfaces rather than large, monolithic ones, so that classes can implement only the methods they need.
- Dependency Inversion Principle (DIP): The Dependency Inversion Principle promotes the idea that high-level modules (e.g., classes, components) should not depend on low-level modules. Instead, both should depend on abstractions (interfaces or abstract classes). It encourages the use of dependency injection to achieve loose coupling and make software components more flexible and easier to test.

These SOLID principles guide the design and development of object-oriented systems, helping developers create software that is more modular, extensible, and maintainable. By adhering to these principles, developers can reduce the complexity of their code, improve its readability, and make it more resilient to changes and updates.

What are classes

C++ is an object-oriented programming (OOP) language, and classes are a fundamental concept in C++ that enable you to implement object-oriented design. Classes provide a blueprint for creating objects, which are instances of a class. Here's an introduction to C++ classes:

1. Class Definition: A class in C++ is a user-defined data type that defines a template for creating objects. It defines the properties (data members or attributes) and behaviors (member functions or methods) that objects of the class will have.

2. Encapsulation One of the key principles of OOP is encapsulation, and classes facilitate this. Encapsulation means bundling data and methods that operate on that data into a single unit (i.e., an object). In C++, you can specify the access level of data members (public, private, protected) to control who can access them.

3. Constructors Constructors are special member functions used to initialize objects of a class. They have the same name as the class and are automatically called when an object is created. Constructors allow you to set initial values for the object's data members.

4. Destructors Destructors are used to clean up resources or perform necessary actions when an object is destroyed (e.g., when it goes out of scope or is explicitly deleted). They have the same name as the class preceded by a tilde ( ).

5. Member Functions: Member functions are functions defined within a class that operate on the class's data members. These functions can be public, private, or protected. Public member functions can be called from outside the class and are used to interact with objects.

6. Data Members: Data members are variables declared within a class that represent the object's state. They define what data an object of the class can store.

7. Access Specifiers: C++ provides three access specifiers: public, private, and protected. These specifiers determine the visibility and accessibility of class members.

    - public: Members are accessible from any part of the program.
    - private: Members are accessible only within the class.
    - protected: Members are accessible within the class and its subclasses.

Here is an example about classes:

```
 // Class definition
class Car {
public:
    // Constructor
    Car(int year, string make) {
        this->year = year;
        this->make = make;
    }

    // Member function
    void startEngine() {
        cout << "Engine started!" << endl;
    }

private:
    // Data members
    int year;
    string make;
};
```

In this example, we define a Car class with a constructor, a member function startEngine, and private data members year and make. You can create Car objects, call member functions, and access data members to work with instances of the class.

Objects are instances of a class. To create an object, you declare a variable of the class type. For example:

```
Car myCar; // Creates an object of the class Car
```

You can access the data members and member functions of an object using the dot (.) operator. For example:

```
 Car myCar; // Creates an object of the class Car
 myCar.startEngine(); // Calls a member function
```

Here is a complete C++ class and how the class is called from inside main():

```
 #include <iostream>
#include <string>

using namespace std;

// Class definition
class Car {
public:
    // Constructor
    Car(int year, string make) {
        this->year = year;
        this->make = make;
        engineRunning = false;
    }

    // Member function to start the engine
    void startEngine() {
        engineRunning = true;
        cout << "Engine started!" << endl;
    }

    // Member function to stop the engine
    void stopEngine() {
        engineRunning = false;
        cout << "Engine stopped." << endl;
    }

    // Member function to display car info
    void displayInfo() {
        cout << "Year: " << year << endl;
        cout << "Make: " << make << endl;
        cout << "Engine status: " << (engineRunning ? "Running" :
            "Stopped") << endl;
    }
```

```
private:
    // Data members
    int year;
    string make;
    bool engineRunning;
};

int main() {
    // Create an object of the Car class
    Car myCar(2023, "Toyota");

    // Display car info
    cout << "Car Info:" << endl;
    myCar.displayInfo();

    // Start the engine
    myCar.startEngine();
    cout << "Car Info after starting the engine:" << endl;
    myCar.displayInfo();

    // Stop the engine
    myCar.stopEngine();
    cout << "Car Info after stopping the engine:" << endl;
    myCar.displayInfo();

    return 0;
}
```

In this example, we define a Car class with a constructor, member functions (startEngine, stopEngine, and displayInfo), and private data members (year, make, and engineRunning). In the main() function, we create an instance of the Car class, call its member functions, and display information about the car object.

When you run this program, it will create a Car object, manipulate its state (start and stop the engine), and display information about the car's year, make, and engine status.

Relationship between Objects and classes

In object-oriented programming (OOP), objects and classes are closely related and have a hierarchical relationship. Here's how they are connected:

- Classes Define Objects:
  - Classes act as blueprints or templates for creating objects. A class defines the structure (data members or attributes) and behavior (member functions or methods) that objects of that class will have.
  - Objects are instances of classes. When you create an object based on a class, you're essentially creating a specific instance that adheres to the blueprint defined by that class.

- Objects Are Instances of Classes:
  - Objects are concrete instances of a class. They represent real-world entities or concepts based on the design defined by the class.
  - Each object created from a class has its own set of data members and can perform actions defined by the class's methods.
- Multiple Objects from a Single Class:
  - Multiple Objects from a Single Class:
  - You can create multiple objects from the same class. Each object will have its own unique data values and can independently execute the methods defined by the class.
- Polymorphism
  - Polymorphism is a concept in OOP where objects of different classes can be treated as objects of a common base class. This allows you to write code that can work with objects of various types, making it more flexible and adaptable.
  - Polymorphism enables you to create functions or methods that can accept objects of different classes that share a common base class.

Constructor: revisited

In C++, constructors are special member functions used for initializing objects when they are created. Constructors have the same name as the class and are called automatically when an object is instantiated. There are several types of constructors in C++, each serving different purposes. Here are the main types of constructors with examples:

Default Constructor:

A default constructor is called when an object is created without any arguments. It initializes the object's data members to default values.

```
 class MyClass {
public:
    MyClass() {
        // Default constructor
        value = 0;
        message = "Hello, World!";
    }

    void display() {
        cout << message << " Value: " << value << endl;
    }

private:
    int value;
    string message;
};
```

Example usage:

```
 MyClass obj; // Calls the default constructor
obj.display(); // Output: Hello, World! Value: 0
```

Parameterized Constructor:

A parameterized constructor takes one or more arguments and initializes the object's data members with the provided values.

```
class Student {
public:
    Student(string n, int r) {
        // Parameterized constructor
        name = n;
        rollNumber = r;
    }

    void display() {
        cout << "Name: " << name << " Roll Number: " <<
           rollNumber << endl;
    }

private:
    string name;
    int rollNumber;
};
```

Example usage:

```
 Student s1("Alice", 101); // Calls the parameterized constructor
s1.display(); // Output: Name: Alice Roll Number: 101
```

Copy Constructor:

A copy constructor creates a new object as a copy of an existing object of the same class. It is used for deep copying data members.

```
 class Person {
public:
    Person(string n, int a) {
        name = n;
        age = a;
    }

    // Copy constructor
    Person(const Person& other) {
        name = other.name;
        age = other.age;
```

```
    }

    void display() {
        cout << "Name: " << name << " Age: " << age << endl;
    }

private:
    string name;
    int age;
};
```

Example usage:

```
 Person p1("Bob", 30);
Person p2 = p1; // Calls the copy constructor to create p2 as a
   copy of p1
p2.display();   // Output: Name: Bob Age: 30
```

Importance of Initializer Lists in Constructors

In C++, an initializer list is used in a constructor to directly initialize data members of a class. It appears after the parameter list and before the constructor body. Using initializer lists is not only a matter of style, but also of correctness and performance.

Initialization vs. Assignment

When data members are initialized using an initializer list, they are constructed with the given values. If members are instead assigned values inside the constructor body, they are first default-constructed and then reassigned. This can lead to unnecessary overhead.

```
class Example {
    int x;
public:
    Example(int val) : x(val) { }   // Direct initialization
    // Example(int val) { x = val; } // Assignment (less
       efficient)
};
```

Const and Reference Members

Members declared as `const` or references must be initialized using an initializer list, since they cannot be assigned after construction.

```
class Sample {
    const int id;
    int& ref;
public:
    Sample(int val, int& r) : id(val), ref(r) { } // Required
```

```
};
```

Performance Considerations

For complex data types such as `std::string` or `std::vector`, initializer lists prevent redundant default construction followed by reassignment, leading to more efficient code.

```
class Person {
    std::string name;
public:
    Person(std::string n) : name(std::move(n)) { } // Efficient
};
```

Initialization Order

Class members are initialized in the order they are declared, not in the order listed in the initializer list. Thus, consistent declaration and initialization order is crucial for predictable behavior.

Base Class Initialization

Initializer lists are also used to call constructors of base classes explicitly.

```
class Base {
public:
    Base(int a) { }
};

class Derived : public Base {
public:
    Derived(int a) : Base(a) { }  // Initializes the base class
};
```

Summary

Initializer lists in constructors are essential for:

- Efficient and direct member initialization
- Proper initialization of `const` and reference members
- Performance optimization for complex types
- Correct base class initialization
- Clear and predictable construction behavior

In modern C++ programming, it is best practice to use initializer lists whenever possible to ensure correctness and efficiency.

Move Constructor

A move constructor is a special constructor in C++ that allows for the efficient transfer of ownership of resources from one object to another. It is used to move the contents of one object (often a temporary or soon-to-be-destroyed object) into another object, typically without copying data. This is particularly useful for optimizing memory and performance when working with expensive-to-copy resources like dynamically allocated memory or large data structures.

The move constructor has the following signature:

```
class MyClass {
public:
    MyClass(MyClass&& other) noexcept {
        // Move constructor
        // Transfer ownership of resources from 'other' to 'this'
        // Usually involves shallow copying and resetting 'other'
    }
};
```

Here's an example illustrating the use of a move constructor with a simple class:

```
#include <iostream>
#include <string>

class MyString {
public:
    MyString(const char* s) : data(nullptr) {
        size = strlen(s);
        data = new char[size + 1];
        strcpy(data, s);
    }

    // Move constructor
    MyString(MyString&& other) noexcept : data(nullptr), size(0)
       {
        std::swap(data, other.data); // Efficiently transfer
           ownership
        std::swap(size, other.size);
    }

    ~MyString() {
        delete[] data;
    }

    void display() {
        std::cout << "String: " << data << " Length: " << size <<
            std::endl;
    }

private:
```

```
    char* data;
    size_t size;
};

int main() {
    MyString str1("Hello, World!");

    // Use the move constructor to efficiently transfer ownership
    MyString str2(std::move(str1));

    std::cout << "After moving:" << std::endl;
    str1.display(); // Output: String:  Length: 0
    str2.display(); // Output: String: Hello, World! Length: 13

    return 0;
}
```

In this example, we define a MyString class with a move constructor. When str2 is constructed using std::move(str1), the move constructor efficiently transfers ownership of the data and size resources from str1 to str2, leaving str1 in a valid but unspecified state.

Move constructors are particularly valuable when working with large data structures, managing memory, and optimizing performance in C++ programs. They play a crucial role in enabling the efficient use of move semantics and avoiding unnecessary data copying. In a later section, we will provide more details to the move semantics.

Destructor: revisited

In C++, a destructor is a special member function of a class that is responsible for cleaning up resources used by objects of that class when they go out of scope or are explicitly destroyed. Destructors have the same name as the class, preceded by a tilde (~).

Here's more information about C++ destructors:

1. Purpose of Destructors:
    - The primary purpose of a destructor is to release any resources held by an object, such as dynamic memory allocations, file handles, or other external resources.
    - Destructors are called automatically when an object goes out of scope or is explicitly destroyed using the delete operator (for dynamically allocated objects) or when it goes out of scope.
2. Destructor Syntax:
    - The syntax of a destructor is as follows:

```
class MyClass {
public:
    // Constructor(s) here
```

```
    MyClass() { /* Constructor code */ }
    // ...

    // Destructor
    ~MyClass() { /* Destructor code */ }
};
```

- Destructors do not take any parameters, and their name is the class name preceded by a tilde ( ).

3. Resource Cleanup:

   - Inside the destructor, you should release any resources that the object acquired during its lifetime.
   - Common tasks in a destructor include deleting dynamically allocated memory (e.g., with delete or delete[]), closing files, releasing network connections, or freeing any other acquired resources.

4. Automatic Invocation:

   - Destructors are called automatically when an object goes out of scope or is destroyed. For locally defined objects, this happens when they leave their scope. For dynamically allocated objects, you must use the delete operator to trigger the destructor.
   - Destructors are called in the reverse order of object creation.

5. Default Destructor:

   - If you don't provide a destructor in your class, C++ will generate a default destructor that does nothing. This is usually sufficient for classes that do not manage any resources.

6. Destructor in Inheritance:

   - When working with inheritance, derived classes can have their own destructors, which are called after the base class destructor. Make sure to call the base class destructor explicitly if necessary.

Here's an example of a class with a destructor:

```
#include <iostream>

class ResourceHolder {
public:
    ResourceHolder() {
        // Constructor acquires a resource
        resource = new int[100];
        std::cout << "Resource acquired." << std::endl;
    }
```

```
    ~ResourceHolder() {
        // Destructor releases the resource
        delete[] resource;
        std::cout << "Resource released." << std::endl;
    }

private:
    int* resource;
};

int main() {
    {
        ResourceHolder holder; // Object created within a scope
    } // Destructor called when 'holder' goes out of scope

    // ...
    return 0; // Program ends, and 'holder' destructor is called
       again
}
```

In this example, the ResourceHolder class acquires and releases a dynamically allocated resource in its constructor and destructor, respectively. When the holder object goes out of scope, its destructor is automatically called, releasing the resource. The destructor is called a second time when the program exits.

Classes and Pointers

Pointers and classes are closely related in C++. Pointers provide a way to work with objects of a class dynamically, and they are especially useful when dealing with objects that are created dynamically on the heap or when you want to pass objects as arguments to functions efficiently. Here's how pointers and classes interact in C++:

Pointer to Objects:

You can declare pointers to objects of a class just like you declare pointers to other data types. For example:

```
MyClass* myObjectPtr; // Pointer to an object of class MyClass
```

Once you have a pointer, you can use it to dynamically allocate objects on the heap:

```
myObjectPtr = new MyClass; // Dynamically allocate an object on
   the heap
```

You can then access the object's members (data members and member functions) through the pointer:

```
myObjectPtr->memberFunction(); // Call a member function using
   the pointer
```

```
myObjectPtr->dataMember = 42;  // Access a data member using the
    pointer
```

Passing Objects by Pointer:

Pointers are often used to pass objects as function arguments, especially when you want to avoid the overhead of copying the object. By passing a pointer to the object, you can modify the original object inside the function.

```
void modifyObject(MyClass* objPtr) {
    objPtr->dataMember = 99;
}
```

Example usage:

```
 MyClass myObj;
modifyObject(&myObj); // Pass a pointer to myObj
```

Dynamically Allocated Objects:

Pointers are essential for working with objects that are dynamically allocated on the heap using new. It's your responsibility to delete such objects to avoid memory leaks when they are no longer needed.

```
MyClass* dynamicObj = new MyClass;
// ...
delete dynamicObj; // Deallocate the object to release memory
```

Pointer to Class Members:

C++ allows you to create pointers to members of a class, including data members and member functions. These pointers can be useful for various advanced use cases, such as implementing callback mechanisms.

```
int MyClass::*dataMemberPtr = &MyClass::dataMember; // Pointer to
    a data member
void (MyClass::*funcPtr)() = &MyClass::memberFunction; // Pointer
    to a member function
```

Example usage:

```
 MyClass myObj;
myObj.*dataMemberPtr = 42; // Access data member through the
    pointer
(myObj.*funcPtr)();       // Call member function through the
    pointer
```

Move Semantics in C++

Move semantics is one of the most significant features introduced in **C++11**. It enhances performance by eliminating unnecessary copying of temporary or large objects. Traditional C++ code relied heavily on copying objects, which could be expensive for classes managing dynamic resources such as memory, files, or network handles. Move semantics provides a mechanism to transfer ownership of such resources rather than duplicating them, improving both speed and efficiency.

Motivation

Consider a class that manages a dynamic resource, such as a memory buffer. When an object of this class is returned from a function or passed by value, a copy of the resource must be made. This can be slow and wasteful if the original object is about to be destroyed anyway.

Move semantics solves this problem by "moving" the resource from one object to another, leaving the original in a valid but unspecified state, rather than performing a deep copy. This transfer avoids unnecessary allocation and deallocation operations.

Rvalue References

To enable move semantics, C++ introduces the concept of **rvalue references**. An rvalue reference is declared using a double ampersand (`&&`). It can bind to a temporary object (an rvalue), allowing the function to "steal" its resources.

```
int&& r = 5;         // r is an rvalue reference to a temporary
   integer
std::string s = "Hello";
std::string&& t = std::move(s); // t now refers to the moved
   string
```

The standard library function `std::move()` is used to explicitly convert an object into an rvalue reference, signaling that its resources can be moved.

Move Constructor and Move Assignment Operator

A class that manages resources can define two special member functions to support move semantics:

- **Move Constructor** — transfers resources from a temporary object to a new one.
- **Move Assignment Operator** — transfers resources from a temporary object to an existing one.

```
class Buffer {
private:
    int* data;
    size_t size;
```

```
public:
    // Constructor
    Buffer(size_t n) : size(n), data(new int[n]) {}

    // Destructor
    ~Buffer() { delete[] data; }

    // Move Constructor
    Buffer(Buffer&& other) noexcept
        : data(other.data), size(other.size) {
        other.data = nullptr;
        other.size = 0;
    }

    // Move Assignment Operator
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data;          // free current resource
            data = other.data;      // transfer ownership
            size = other.size;
            other.data = nullptr;   // leave source in valid
               state
            other.size = 0;
        }
        return *this;
    }
};
```

The keyword `noexcept` is often used with move operations to indicate that they will not throw exceptions. This allows the compiler and standard containers (like `std::vector`) to optimize performance further.

When Are Moves Used?

The compiler automatically uses move semantics in the following cases:

- When returning a local object by value from a function.
- When passing or returning temporary objects.
- When inserting or assigning objects into standard containers (e.g., `std::vector`, `std::map`) that support move operations.

```
Buffer createBuffer() {
    Buffer temp(100);
    return temp; // move semantics used (no deep copy)
}
```

Advantages of Move Semantics

- **Performance Improvement:** Avoids deep copies of large data structures.
- **Resource Safety:** Ensures resources are transferred rather than duplicated or leaked.
- **Efficiency in Containers:** Standard containers can resize and reallocate elements faster by moving instead of copying.

Rule of Five

With the introduction of move semantics, the traditional *Rule of Three* in C++ (constructor, copy constructor, and destructor) expanded into the **Rule of Five**, which includes:

- Copy constructor
- Copy assignment operator
- Destructor
- Move constructor
- Move assignment operator

If a class manages resources, it should define all five (or rely on compiler-generated defaults) to ensure proper ownership semantics.

Deleted and Defaulted Special Member Functions

Modern C++ (since C++11) provides two important keywords, `=delete` and `=default`, that give programmers fine-grained control over how special member functions are generated or restricted. These keywords improve code safety, clarity, and efficiency by making object behavior explicit, rather than relying on compiler-generated defaults.

Special Member Functions

C++ classes can define up to six special member functions that manage object creation, copying, moving, and destruction:

- Default constructor
- Copy constructor
- Copy assignment operator
- Move constructor
- Move assignment operator
- Destructor

By default, if not explicitly defined, the compiler generates these functions automatically. However, this automatic behavior is sometimes undesirable, especially when a class manages non-copyable resources or requires specific construction logic. The `=delete` and `=default` specifiers address these cases.

Defaulted Functions

A function declared with `=default` instructs the compiler to generate its standard implementation explicitly. This is particularly useful when you want to emphasize that the default behavior is intended and safe.

```
class Example {
public:
    Example() = default;                    // Explicitly use the
       default constructor
    Example(const Example&) = default;      // Default copy
       constructor
    Example(Example&&) = default;           // Default move
       constructor
    ~Example() = default;                   // Default destructor
};
```

Using `=default` makes code more readable and allows defaulted functions to appear in any access section (e.g., `private`), providing additional control over object creation.

Deleted Functions

A function declared with `=delete` tells the compiler that this function is intentionally not allowed to be called. Any attempt to use a deleted function will result in a compile-time error. This is a common technique for preventing unwanted copying, assignment, or conversion.

```
class NonCopyable {
public:
    NonCopyable() = default;                        // Allow
       construction
    NonCopyable(const NonCopyable&) = delete;       // Disallow
       copying
    NonCopyable& operator=(const NonCopyable&) = delete; //
       Disallow assignment
};
```

In this example, objects of `NonCopyable` can be constructed normally, but cannot be copied or assigned, preventing accidental duplication of resources.

Common Use Cases

- **Preventing Copying or Moving:** Classes managing exclusive resources (e.g., file handles, network sockets) often disable copying to avoid double-free errors.

- **Controlling Implicit Conversions:** Functions like constructors or type conversion operators can be deleted to prevent unintended type conversions.
- **Making Abstract Interfaces:** Abstract base classes can delete certain operations to ensure derived classes implement or avoid specific behaviors.

Deleted Functions in Practice

For example, the `std::unique_ptr` class in the C++ Standard Library deletes its copy constructor and copy assignment operator, ensuring that ownership of a resource is unique and cannot be copied:

```
template<typename T>
class unique_ptr {
public:
    unique_ptr(const unique_ptr&) = delete;             // No
       copy
    unique_ptr& operator=(const unique_ptr&) = delete;  // No
       copy assignment
    unique_ptr(unique_ptr&&) noexcept;                  // Move
       allowed
    unique_ptr& operator=(unique_ptr&&) noexcept;
};
```

This design makes resource ownership safe and predictable, leveraging move semantics instead of copying.

Combining Default and Delete

Defaulting and deleting can coexist in the same class to precisely control allowed operations:

```
class ResourceHandler {
public:
    ResourceHandler() = default;
    ResourceHandler(const ResourceHandler&) = delete;  // No
       copying
    ResourceHandler& operator=(const ResourceHandler&) = delete;
    ResourceHandler(ResourceHandler&&) noexcept = default; //
       Allow moving
    ResourceHandler& operator=(ResourceHandler&&) noexcept =
       default;
    ~ResourceHandler() = default;
};
```

This pattern is common in modern C++ to create move-only types that safely manage dynamic or external resources.

Specifier	Purpose
`=default`	Explicitly requests the compiler to generate a default implementation.
`=delete`	Explicitly forbids the compiler from generating or using the function.

Table 1: Comparison of `=default` and `=delete` Specifiers

Structures and Classes

In C++, both structures and classes are used to define user-defined data types that can group together data members and member functions into a single unit. However, there are some key differences between structures and classes. Here's an overview of both:

Structures:

- Default Member Accessibility: In a C++ structure, all data members have public accessibility by default. This means that you can access the data members directly without any restrictions.

```
struct Point {
    int x; // Public by default
    int y; // Public by default
};
```

- Usage for Data Containers: Structures are often used when you need a simple data container to group related data elements together. They don't typically have member functions but focus on storing and organizing data.

```
Point p;
p.x = 10;
p.y = 20;
```

- No Member Functions: While C++ allows you to add member functions to structures, it's not a common practice. By convention, structures are primarily used for holding data.

```
struct Rectangle {
    int width;
    int height;
    int area() {
        return width * height; // Member function to
           calculate area
    }
};
```

Classes:

- Default Member Accessibility: In a C++ class, all data members have private accessibility by default. This means that data members are not directly accessible from

outside the class. You need member functions (getters and setters) to access or modify them.

```
class Person {
private:
    std::string name; // Private by default
    int age;          // Private by default
public:
    // Member functions to access and modify private members
    void setName(const std::string& n) { name = n; }
    void setAge(int a) { age = a; }
};
```

- Usage for Object-Oriented Programming: Classes are the foundation of object-oriented programming (OOP). They encapsulate data members and member functions, allowing you to define behavior (methods) associated with the data (attributes).

```
Person p;
p.setName("Alice");
p.setAge(30);
```

- Encapsulation: Classes support encapsulation, a key OOP concept. This means you can control access to data members, hiding implementation details and providing a public interface for interacting with objects.

```
Person p;
p.setName("Alice"); // Access through a public member
   function
```

In summary, structures are primarily used for grouping data members together, and their members are typically public by default. Classes, on the other hand, are used for defining complex data types with private data members and member functions that provide an interface for interacting with the data. Classes are central to object-oriented programming and encapsulation, allowing you to model real-world entities with behavior and data.

Static Members

Static Member Variables

In C++, a static member variable is a variable that is associated with a class rather than with instances (objects) of the class. It is shared among all instances of the class and can be accessed without creating an object of the class. Here are some important characteristics and usage guidelines for static member variables:

1. Declaration and Initialization: Static member variables are declared inside a class like any other class member but with the static keyword. They are typically initialized outside the class definition in a source file.

```
class MyClass {
public:
    static int count; // Declaration
};

int MyClass::count = 0; // Initialization
```

2. Shared Among All Objects: All instances (objects) of the class share the same static member variable. Modifying the static member variable in one object will affect its value in all other objects of the class.

3. Accessing Static Member Variables: You can access a static member variable using the class name followed by the scope resolution operator (::) or by using an object of the class. However, it's a good practice to access static members through the class name to emphasize that they are shared among all instances.

```
 int main() {
    MyClass obj1, obj2;
    MyClass::count = 10; // Access through class name
    obj1.count = 20;     // Also valid but not recommended
       for static members
    cout << obj2.count << endl; // Access through object (not
        recommended)
    return 0;
}
```

4. Initialization: Static member variables should be explicitly initialized, either inside the class or in a source file outside the class definition.

5. Use Cases: Static member variables are often used to maintain data that is shared among all instances of a class. For example, you might use a static member variable to keep count of the number of objects created from a class.

```
class MyClass {
public:
    static int objectCount; // Static member variable
    MyClass() {
        objectCount++; // Increment count in the constructor
    }
};

int MyClass::objectCount = 0; // Initialize the static member
    variable
```

6. Constant Static Member Variables: You can also have static member variables that are constants, declared as static const. These values are fixed and cannot be modified once initialized.

```
class Constants {
public:
    static const int MAX_VALUE = 100; // Constant static
       member
};
```

7. Thread Safety: Be cautious when using static member variables in a multithreaded environment. Concurrent access without proper synchronization can lead to race conditions. If thread safety is a concern, consider using mutexes or thread-safe constructs.

Static member variables are a powerful feature in C++ and can be used for various purposes, including counting instances, sharing configuration settings, and managing shared resources among class instances. They provide a way to store class-level data that is independent of object-specific data.

Static Member Functions

In C++, a static member function is a member function of a class that is associated with the class itself rather than with instances (objects) of the class. They are called on the class itself, not on individual objects of the class. Static member functions have some distinctive characteristics:

1. A static member function is declared inside a class like any other member function, but it is also preceded by the static keyword. It is defined outside the class just like a regular function.

```
class MyClass {
public:
    static void staticFunction(); // Declaration
};

void MyClass::staticFunction() {
    // Definition
    // Static member functions do not have access to instance
       -specific data
}
```

2. No Access to Instance Data: Static member functions do not have access to instance-specific data (non-static data members) or this pointer because they are not called on instances. They can only access static members (variables or functions) of the class.

3. Accessing Static Member Functions: You can call a static member function using the class name followed by the scope resolution operator (::) or using an object of the class. However, it's more common to call them through the class name to emphasize that they are class-level functions.

```
int main() {
    MyClass::staticFunction(); // Access through class name
    MyClass obj;
    obj.staticFunction(); // Also valid but not common
    return 0;
}
```

4. Use Cases: Static member functions are often used for tasks that are not related to specific instances but are associated with the class as a whole. Common use cases include utility functions, factory methods, and functions that operate on static data members.

```
 class MathUtility {
public:
    static int add(int a, int b) {
        return a + b;
    }
};

int result = MathUtility::add(5, 3); // Calling a static
   member function
```

5. Benefits: Static member functions offer several benefits, such as encapsulating class-level functionality, improving code organization, and avoiding the need to create instances for certain operations.

6. Thread Safety: Static member functions, like static member variables, should be used with caution in multithreaded programs. Ensure proper synchronization mechanisms are in place if they access shared resources or data.

Static member functions are a valuable feature in C++ for organizing code and providing class-level functionality. They are a key part of the language's support for encapsulation and object-oriented design.

Constant Member Functions

In C++, a const member function is a member function of a class that promises not to modify the state of the object on which it is called. It is a way of declaring that the function does not modify the data members of the object. This promise is enforced by the compiler, which generates an error if you attempt to modify non-mutable data members within a const member function.

Here's how const member functions work and why they are useful:

1. Declaration and Syntax: To declare a member function as const, you add the const keyword after the function's parameter list but before its body in the class declaration.

```
class MyClass {
public:
    int getData() const; // Declaration of a const member
       function
};
```

When defining the function outside the class, you include const again to indicate that it's a const member function.

```
 int MyClass::getData() const {
    // Implementation of the const member function
    // Cannot modify non-mutable data members here
}
```

2. Promise of Non-Modification: By declaring a member function as const, you promise that it won't modify the state of the object on which it is called. This includes not modifying any non-mutable data members of the class.

3. Access to const and Non-const Objects: You can call a const member function on both const and non-const objects of the class. However, you cannot call a non-const member function on a const object, as it could potentially modify the object's state.

```
const MyClass obj1;
MyClass obj2;

int data1 = obj1.getData(); // Calling a const member
   function on a const object
int data2 = obj2.getData(); // Calling a const member
   function on a non-const object
```

4. Use Cases: const member functions are particularly useful when you want to provide read-only access to the object's data without allowing modifications. They are commonly used for getter methods that return the value of data members without altering them.

```
class TemperatureSensor {
public:
    double getTemperature() const {
        return temperature;
    }

private:
    double temperature;
};
```

5. Mutable Data Members: If you have data members that you want to modify even within const member functions, you can declare them as mutable. These mutable members are exempt from the promise of non-modification.

```
 class MyClass {
public:
    int getData() const {
        return data + mutableData;
    }

private:
    int data;
    mutable int mutableData; // Can be modified in const
       member functions
};
```

6. Error Prevention: Using const member functions helps prevent unintentional modifications to objects and promotes good coding practices by clearly indicating which functions can modify the object's state and which cannot.

const member functions play a crucial role in ensuring data integrity and providing safe access to object data within C++ classes. They are an important feature for designing classes with well-defined interfaces and controlling object behavior.

Lab 3: Introduction to C++ Classes

- Lab Objectives:

  1. Define a class with data members and member functions.
  2. Implement constructors and destructor.
  3. Create objects and use class member functions.
  4. Create objects and use class member functions.
  5. Understand and practice encapsulation.
  6. Apply object-oriented programming principles.

- Task 1: Create a Class

  1. Define a class named Student with the following private data members:
     - std::string name (Student's name)
     - int age (Student's age)
     - double gpa (Student's GPA)
  2. Add public member functions to set and get the values of these data members.

- Task 2: Constructors and Destructor

  1. Implement a default constructor that initializes the data members with default values (e.g., empty string, age of 0, and GPA of 0.0).
  2. Implement a parameterized constructor that accepts values for the data members and initializes them accordingly.

3. Implement a destructor that prints a message when an object is destroyed.

- Task 3: Create Objects
  1. Create two Student objects: one using the default constructor and another using the parameterized constructor.
  2. Set and display values for the data members of these objects using member functions.
- Task 4: Encapsulation
  1. Change the access level of the data members to be private.
  2. Modify the Student class to use public member functions to set and get the values of the data members.
- Task 5: Object-Oriented Principles
  1. Implement a member function void study() in the Student class that prints a message indicating that the student is studying.
  2. Implement a member function void displayInfo() in the Student class that displays the student's name, age, and GPA.
- Task 6: Main Program
  1. In the main() function, create a menu-driven program that allows the user to perform the following actions:
     - Create a new Student object using the parameterized constructor and set its values.
     - Display the information of a student using the displayInfo() function.
     - Make a student study using the study() function.
     - Make a student study using the study() function.
  2. Lab Completion:
     - Test your program thoroughly by creating and manipulating Student objects.
     - Ensure that constructors and destructors are called correctly.
     - Verify that encapsulation principles are followed.
     - Demonstrate the use of member functions to access and modify data members.

Practice Questions

1. Define a C++ class named Rectangle with private data members for length and width. Provide public member functions to set and get the values of these data members. Create objects of this class and demonstrate the use of these member functions.

2. Create a C++ class named Person with private data members for name and age. Implement a parameterized constructor to initialize these members and a member function to display the person's details. Create objects of this class and display the details of multiple persons.

3. Define a C++ class called BankAccount with private data members for accountNumber, accountHolderName, and balance. Implement member functions to:

   - Initialize the account with a constructor.
   - Deposit money into the account.
   - Withdraw money from the account.
   - Display the account details (account number, holder name, and balance).

4. Create a C++ class called Circle with a private data member for the radius. Implement member functions to calculate and return the area and circumference of the circle. Demonstrate the use of this class to calculate the area and circumference of multiple circles.

5. Define a C++ class named Book to represent a book with private data members for title, author, and publicationYear. Implement a parameterized constructor to initialize these members and a member function to display the book's details. Create an array of Book objects and display the details of all the books.

6. Create a C++ class called Date with private data members for day, month, and year. Implement a parameterized constructor to initialize these members. Write a member function to display the date in the format "dd/mm/yyyy." Create objects of this class to represent various dates and display them.

7. Implement a C++ class called Employee with private data members for name, employeeId, and salary. Provide a parameterized constructor to initialize these members. Implement a member function to give an annual salary raise to an employee. Create objects of this class and apply a salary raise to some employees.

8. Define a C++ class named Car with private data members for make, model, and year. Implement a parameterized constructor to initialize these members. Write a member function to display the car's details. Create objects of this class to represent different cars and display their details.

9. Create a C++ class called Student with private data members for name, rollNumber, and marks. Implement a parameterized constructor to initialize these members. Write a member function to calculate and return the grade based on the marks (A, B, C, D, or F). Create objects of this class to represent students and display their grades.

10. Implement a C++ class named Complex to represent complex numbers with private data members for the real and imaginary parts. Provide member functions to perform addition and multiplication of complex numbers. Create objects of this class and demonstrate these operations.

Lecture 4: Inheritance and Polymorphism

In C++, classes are the building blocks of object-oriented programming, and they play a pivotal role in inheritance. Inheritance allows you to create new classes (derived or child classes) by inheriting the properties and behaviors of existing classes (base or parent classes). Let's explore how classes are used in inheritance within C++:

Base Class (Parent Class)

The base class serves as the foundation for one or more derived classes. It defines common attributes and methods that can be shared among multiple derived classes. Inheritance allows the base class to pass on its characteristics to the derived classes.

```
class Animal {
public:
    void eat() {
        // Common behavior for all animals
    }

    void sleep() {
        // Common behavior for all animals
    }
};
```

Derived Class (Child Class)

Derived Class (Child Class): A derived class inherits the properties and behaviors of the base class. It can also add its own unique attributes and methods, making it more specialized. Derived classes extend the functionality of the base class.

```
class Dog : public Animal {
public:
    void bark() {
        // Unique behavior for dogs
    }
};

class Cat : public Animal {
public:
    void meow() {
        // Unique behavior for cats
    }
};
```

Access Specifiers

In C++, you can specify the access level for the inherited members from the base class in the derived class. The most common access specifiers are public, protected, and private. The choice of access specifier determines how the derived class and external code can access the inherited members.

- public: Inherited members are accessible to everyone.
- protected: Inherited members are accessible within the derived class and its subclasses but not directly from external code.
- private: Inherited members are not accessible in the derived class (unless a friend relationship is established), making them effectively hidden.

Constructor and Destructor Inheritance

Constructors and destructors are also inherited from the base class. The derived class can call the base class's constructor explicitly to initialize the inherited members.

```
class Dog : public Animal {
public:
    Dog() : Animal() {
        // Constructor for Dog
    }

    ~Dog() {
        // Destructor for Dog
    }
};
```

Method Overriding:

Method Overriding: Derived classes can override (replace) base class methods with their own implementations. This allows for polymorphism, where different objects of the same base class type can exhibit different behavior based on their actual derived class type.

```
class Bird : public Animal {
public:
    void eat() {
        // Override the eat method for birds
        // Custom behavior for birds
    }
};
```

The Shape Example

Let's illustrate inheritance in C++ using a simple example involving geometric shapes. We'll create a base class called Shape and two derived classes, Circle and Rectangle, to showcase how they inherit properties and behaviors from the base class.

```
#include <iostream>
#include <cmath>

// Base class: Shape
class Shape {
public:
    // Common method to calculate area (virtual, to be overridden
       )
    virtual double calculateArea() {
        return 0.0; // Default implementation for base class
    }
};

// Derived class: Circle (inherits from Shape)
class Circle : public Shape {
private:
    double radius;

public:
    // Constructor for Circle
    Circle(double r) : radius(r) {}

    // Override the calculateArea method for circles
    double calculateArea() override {
        return 3.14159265359 * radius * radius; // Area of a
           circle
    }
};

// Derived class: Rectangle (inherits from Shape)
class Rectangle : public Shape {
private:
    double length;
    double width;

public:
    // Constructor for Rectangle
    Rectangle(double l, double w) : length(l), width(w) {}

    // Override the calculateArea method for rectangles
    double calculateArea() override {
        return length * width; // Area of a rectangle
    }
```

```
};

int main() {
    // Create objects of Circle and Rectangle
    Circle circle(5.0);
    Rectangle rectangle(4.0, 6.0);

    // Calculate and display areas
    std::cout << "Area of the Circle: " << circle.calculateArea()
        << std::endl;
    std::cout << "Area of the Rectangle: " << rectangle.
       calculateArea() << std::endl;

    return 0;
}
```

In this example:

1. We define a base class called Shape, which includes a virtual method calculateArea(). This method will be overridden in the derived classes to calculate the area of specific shapes.

2. We create two derived classes, Circle and Rectangle, each inheriting from the Shape base class. These derived classes have their own specific attributes (radius for circles and length and width for rectangles) and override the calculateArea() method to provide their own implementation of area calculation.

3. In the main() function, we create instances of Circle and Rectangle. When we call the calculateArea() method on these objects, the appropriate overridden method is executed, and it calculates and returns the area specific to each shape.

Multiple Inheritance

Multiple inheritance is a feature in object-oriented programming languages, including C++, that allows a class to inherit properties and behaviors from more than one base class. This means a derived class can inherit from multiple parent classes. While multiple inheritance can be powerful, it also introduces complexities and challenges that need to be managed carefully. In C++, you can achieve multiple inheritance by specifying multiple base classes separated by commas in the declaration of a derived class.

Here's a basic example to illustrate multiple inheritance in C++:

```
#include <iostream>

// First base class
class A {
public:
    void methodA() {
        std::cout << "Method A from class A" << std::endl;
    }
```

```
};

// Second base class
class B {
public:
    void methodB() {
        std::cout << "Method B from class B" << std::endl;
    }
};

// Derived class inheriting from both A and B
class C : public A, public B {
public:
    void methodC() {
        std::cout << "Method C from class C" << std::endl;
    }
};

int main() {
    C cObj;

    // Accessing methods from both base classes
    cObj.methodA(); // Calls methodA from class A
    cObj.methodB(); // Calls methodB from class B

    // Accessing a method from the derived class
    cObj.methodC(); // Calls methodC from class C

    return 0;
}
```

In this example:

- We have two base classes, A and B, each with its own method.
- The C class is derived from both A and B using multiple inheritance. As a result, it inherits the methods methodA from class A and methodB from class B.
- In the main() function, we create an instance of the C class and can access methods from both base classes (A and B) as well as its own method (methodC).

Multiple inheritance can be a powerful tool, allowing you to model complex relationships between classes and reuse code effectively. However, it can also introduce ambiguity and the "diamond problem" (a situation where multiple base classes have a common ancestor) in more complex scenarios. Careful design and usage of multiple inheritance, along with virtual inheritance and explicit scope resolution, can help mitigate these issues when they arise.

Consider a scenario where you have a base class Animal, which provides common functionality like a method eat(). Two classes, Mammal and Bird, derive from Animal

and inherit its properties. Later, you design a class Bat that inherits from both Mammal and Bird since bats share characteristics of both groups. The inheritance hierarchy forms a diamond, as shown below:

```
       Animal
      /      \
  Mammal     Bird
      \      /
        Bat
```

Without special handling, the Bat class will contain two copies of the Animal class—one via Mammal and another via Bird. This can result in ambiguity; for instance, when calling eat() on a Bat object, the compiler cannot determine whether to use the Animal implementation inherited from Mammal or the one inherited from Bird. Moreover, if the Animal class contains data members, Bat will have redundant copies of these members, which can lead to increased memory usage and potential data inconsistency.

The core cause of the diamond problem lies in the way C++ implements multiple inheritance. Unlike single inheritance, where the inheritance chain is linear, multiple inheritance allows a class to inherit from more than one base class. When these base classes themselves have a common ancestor, each inheritance path typically results in a separate instance of the common base. Consequently, this redundancy complicates object construction and destruction, copy operations, and even runtime behavior.

The consequences of the diamond problem are both conceptual and practical. Conceptually, ambiguity in member access can confuse developers, leading to subtle bugs that are hard to track down. Practically, the duplicate data members can result in inefficient memory usage. The problem becomes even more critical in large-scale systems where multiple inheritance is heavily used, as it can compromise both performance and maintainability.

C++ provides a mechanism called virtual inheritance to address the diamond problem. When a class inherits virtually from a base class, it signals that the derived class should share a single instance of the virtual base class, regardless of how many intermediate classes inherit from it. Using virtual inheritance, the hierarchy is modified such that the most derived class, like Bat in our example, contains only one copy of the common base class Animal.

To apply virtual inheritance, you simply declare the inheritance as virtual in the intermediate classes:

```
class Animal {
public:
    void eat() {
        // Common eating behavior
    }
};

class Mammal : virtual public Animal {
    // Additional mammal-specific members
};
```

```
class Bird : virtual public Animal {
    // Additional bird-specific members
};

class Bat : public Mammal, public Bird {
    // Bat-specific members
};
```

With this setup, the Bat class contains only one instance of Animal. The ambiguity is resolved because there is now a single path to Animal::eat(), and data redundancy is eliminated. However, using virtual inheritance does introduce some additional overhead and complexity. The compiler needs to manage a pointer (often called a virtual base pointer) to the shared base class, and the construction of the base class becomes the responsibility of the most derived class. This means that when constructing a Bat object, the Animal constructor is called only once, by Bat, ensuring that the shared state is properly initialized.

While virtual inheritance is a powerful solution, it should be used judiciously. Overuse can lead to increased complexity in class hierarchies and may introduce performance penalties due to the extra level of indirection. Developers should carefully design their class hierarchies and consider alternative design patterns such as composition, which might avoid the pitfalls of multiple inheritance altogether.

Moreover, when virtual inheritance is employed, it is important to be aware of potential issues during object construction and destruction. The order in which constructors and destructors are called can be non-intuitive, and careful documentation and design can mitigate these risks. In real-world applications, thorough testing is essential to ensure that the single instance of the virtual base class behaves as expected, especially in scenarios involving complex initialization or resource management.

Polymorphism

Polymorphism is a fundamental concept in object-oriented programming, and it plays a significant role in C++. It allows objects of different classes to be treated as objects of a common base class, providing a way to work with objects in a more abstract and generic manner. There are two primary forms of polymorphism in C++: compile-time (or static) polymorphism and runtime (or dynamic) polymorphism.

Compile-Time Polymorphism (Static Polymorphism)

- Function Overloading: This is a form of compile-time polymorphism where multiple functions in the same scope have the same name but different parameter lists. The appropriate function to be called is determined at compile time based on the number and types of arguments provided.

```
class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }
```

```
    double add(double a, double b) {
        return a + b;
    }
};
```

- Operator Overloading: C++ allows you to redefine the behavior of operators for user-defined classes by overloading them. This enables you to use operators like +, -, *, etc., with user-defined types.

```
class Complex {
public:
    Complex operator+(const Complex& other) {
        Complex result;
        result.real = this->real + other.real;
        result.imaginary = this->imaginary + other.imaginary;
        return result;
    }
};
```

We will consider operator overloading in more details in next chapter.

Runtime Polymorphism (Dynamic Polymorphism)

- Function Overriding: This form of polymorphism occurs when a derived class provides a specific implementation for a function that is already defined in its base class. The function to be executed is determined at runtime based on the actual object's type.

```
class Shape {
public:
    virtual void draw() {
        // Common implementation in the base class
    }
};

class Circle : public Shape {
public:
    void draw() override {
        // Specific implementation for circles
    }
};

class Square : public Shape {
public:
    void draw() override {
        // Specific implementation for squares
    }
};
```

- Polymorphic Function Calls: You can use pointers or references to the base class to achieve polymorphism. When a base class pointer/reference is used to call a virtual function, the appropriate derived class function is invoked at runtime.

```
Shape* shapePtr;
Circle circle;
Square square;

shapePtr = &circle;
shapePtr->draw(); // Calls Circle's draw() method

shapePtr = &square;
shapePtr->draw(); // Calls Square's draw() method
```

- Dynamic Binding: The mechanism by which the correct function is called at runtime is called dynamic binding or late binding. It allows for more flexible and extensible code as new derived classes can be added without changing existing code that uses the base class interface.

Polymorphism is a powerful concept that enhances code flexibility and reusability in C++. It enables you to write more generic and maintainable code by working with objects based on their common interfaces rather than their specific types. It's a key feature of object-oriented programming and is widely used in C++ programs.

Shape drawing

A common real-world example of dynamic polymorphism in C++ is the use of polymorphic behavior through virtual functions when dealing with various shapes in a drawing application. Let's create a simplified example to illustrate dynamic polymorphism with shapes:

```
#include <iostream>
#include <vector>

class Shape {
public:
    virtual void draw() {
        std::cout << "Drawing a generic shape" << std::endl;
    }
};

class Circle : public Shape {
public:
    void draw() override {
        std::cout << "Drawing a circle" << std::endl;
    }
};

class Square : public Shape {
```

```
public:
    void draw() override {
        std::cout << "Drawing a square" << std::endl;
    }
};

int main() {
    std::vector<Shape*> shapes;

    Circle circle;
    Square square;

    shapes.push_back(&circle);
    shapes.push_back(&square);

    for (Shape* shape : shapes) {
        shape->draw(); // Calls the appropriate draw() method
           based on the actual object type
    }

    return 0;
}
```

In this example:

- We define a base class Shape with a virtual draw() method. This method has a common interface for all shapes but is meant to be overridden by derived classes.
- We create two derived classes, Circle and Square, each of which overrides the draw() method to provide a specific implementation for drawing a circle or square.
- In the main() function, we create instances of Circle and Square. We then store pointers to these objects in a vector of Shape*, which is possible because of the polymorphic relationship between the base class and derived classes.

We loop through the vector of Shape* and call the draw() method on each object. Because the draw() method is marked as virtual and overridden in the derived classes, the appropriate draw() method for each shape is called at runtime, demonstrating dynamic polymorphism.

Abstract Classes

Abstract classes are a fundamental concept in object-oriented programming, and they play an essential role in C++ and similar languages. An abstract class is a class that cannot be instantiated directly; instead, it is meant to serve as a blueprint for other classes. Abstract classes are often used to define a common interface and provide some default behavior that derived classes must implement.

Here are some key characteristics and uses of abstract classes in C++:

- Cannot be Instantiated: You cannot create objects of an abstract class. Attempting to do so will result in a compilation error. Abstract classes are meant to be used as base classes for other classes.
- May Contain Pure Virtual Functions: Abstract classes often contain at least one pure virtual function. A pure virtual function is declared with the virtual keyword and is followed by = 0;. It has no implementation in the abstract class and must be implemented by any concrete (i.e., non-abstract) derived class. This enforces that derived classes provide their own implementation of these functions.

```
class AbstractShape {
public:
    virtual void draw() = 0; // Pure virtual function
    virtual double area() const = 0; // Another pure virtual
       function
};
```

- Can Contain Concrete (Non-Virtual) Functions: Abstract classes can have regular (non-virtual) member functions that provide a default implementation that can be shared among derived classes. These functions can be overridden in derived classes if needed.

```
class AbstractShape {
public:
    // Concrete (non-virtual) function with default
       implementation
    void setColor(const std::string& color) {
        this->color = color;
    }

    // Pure virtual functions
    virtual void draw() = 0;
    virtual double area() const = 0;

protected:
    std::string color;
};
```

- Derived Classes Must Implement Pure Virtual Functions: Any class derived from an abstract class must implement all the pure virtual functions declared in the base class. Failure to do so will also result in a compilation error.

```
class Circle : public AbstractShape {
public:
    void draw() override {
        // Implement drawing for circles
    }

    double area() const override {
```

```
        // Implement area calculation for circles
    }
};
```

- Useful for Creating Common Interfaces: Abstract classes are useful for defining common interfaces in situations where multiple classes share a set of functions but implement them differently. They help ensure consistency and conformity among derived classes.

In summary, abstract classes in C++ serve as a way to define a contract or interface that derived classes must adhere to. They allow you to create a hierarchy of related classes with a shared interface while providing some default behavior in the base class. Abstract classes are an essential part of C++'s support for polymorphism and design by contract.

Interface Classes

In C++, there is no direct language construct called "interface" as you might find in some other programming languages like Java or C#. However, you can achieve similar functionality using abstract classes and pure virtual functions, which serve as interfaces in C++. These classes act as contracts that derived classes must adhere to by providing implementations for all the pure virtual functions declared in the base class. This is often referred to as "interface-like" behavior in C++.

Here's how you can create an interface-like structure in C++:

```
class Shape {
public:
    virtual void draw() = 0; // Pure virtual function (interface
       method)
    virtual double area() const = 0; // Another pure virtual
       function (interface method)
    virtual ~Shape() {} // Virtual destructor (recommended for
       polymorphism)
};
```

In this example:

- Shape is an abstract class that defines an interface with two pure virtual functions, draw() and area(). Any class inheriting from Shape must provide implementations for these methods.
- The destructor Shape() is also declared as virtual. This is recommended when you're working with polymorphism to ensure that derived class destructors are called correctly when objects are deleted through a base class pointer or reference.

Here's how you can create a concrete class that implements the Shape interface:

```
class Circle : public Shape {
public:
    void draw() override {
        // Implement drawing for circles
    }

    double area() const override {
        // Implement area calculation for circles
    }
};
```

In this example, the Circle class inherits from Shape and provides implementations for both draw() and area(), adhering to the interface defined by the Shape class.

You can create other classes like Rectangle, Triangle, etc., following the same pattern to implement the Shape interface.

Although C++ doesn't have a native keyword for defining interfaces like some other languages, this approach using abstract classes and pure virtual functions provides similar functionality. It allows you to define and enforce contracts for classes that implement specific interfaces, promoting code consistency and polymorphism.

Best practices and guidelines

1. Favor Composition Over Inheritance: Before using inheritance, consider whether composition (using objects of other classes as members) might be a better choice. Composition can often lead to more flexible and less tightly coupled designs.

2. Use Public Inheritance Sparingly: Public inheritance establishes an "is-a" relationship. Use it only when derived classes truly inherit the interface and behavior of the base class.

3. Prefer Protected or Private Inheritance: If you don't want to expose the interface of the base class to the outside world, consider using protected or private inheritance. This is often a better choice for implementation inheritance.

4. Avoid Deep Inheritance Hierarchies: Deep hierarchies can lead to complexity and make code harder to understand and maintain. Aim for shallow hierarchies with a clear and focused purpose.

5. Declare Virtual Functions Properly: Declare virtual functions in the base class with a clear understanding of which functions should be overridden by derived classes.

6. Always Provide a Virtual Destructor: When using inheritance with polymorphism, always declare a virtual destructor in the base class. This ensures that derived class destructors are called correctly when objects are deleted through base class pointers.

7. Use the override Keyword: In derived classes, use the override keyword when overriding virtual functions. This helps catch errors at compile time and makes the code more readable.

8. Avoid Hiding Inherited Functions: When inheriting functions with the same name as those in the derived class, use the using keyword to bring them into scope rather than hiding them. Hiding can lead to confusion.

9. Implement Pure Virtual Functions Completely: Derived classes must provide complete implementations for pure virtual functions. Failure to do so results in a compilation error.

10. Don't Rely on the Order of Constructor Calls: The order in which constructors are called for base and derived classes can be complex. Avoid relying on a specific order and design your classes to be independent of constructor order.

11. Use Static Polymorphism Where Appropriate: Not all polymorphism requires dynamic dispatch. If performance is critical, consider using function overloading and templates for static polymorphism.

12. Document Your Inheritance Hierarchy: Provide clear documentation for your inheritance hierarchy, including the purpose of each class and its relationship with others.

13. Consider the Liskov Substitution Principle (LSP): Ensure that derived classes can be substituted for their base classes without altering the correctness of the program.

14. Test and Verify: Thoroughly test your derived classes to ensure they behave as expected when inheriting from a base class.

15. Refactor When Necessary: Be open to refactoring your inheritance hierarchy if you discover design flaws or changing requirements.

16. Use Access Specifiers Thoughtfully: Choose access specifiers (public, protected, private) carefully to control the visibility of inherited members based on your design intent.

    Adhering to these best practices and guidelines when using inheritance in C++ will help you create maintainable, extensible, and robust code while minimizing potential pitfalls and design issues.

Summary

Inheritance and polymorphism are two fundamental concepts in C++ that enable the creation of robust, reusable, and extensible object-oriented code. They play key roles in structuring class hierarchies and enabling dynamic behavior in C++ programs.

- Inheritance:

    1. Definition: Inheritance is a mechanism in C++ that allows a class (called the derived or child class) to inherit properties and behaviors from another class (called the base or parent class). It establishes an "is-a" relationship between classes.

2. Key Points:

   Base classes provide a common template for derived classes to build upon. Derived classes inherit attributes and methods from their base classes. Inheritance promotes code reuse, hierarchy, and modularity in software design. Access specifiers (public, protected, private) control the visibility of inherited members in derived classes. Constructors and destructors are inherited and can be explicitly called in derived classes.

- Polymorphism

  1. Definition: Polymorphism is a concept that allows objects of different classes to be treated as objects of a common base class. It enables dynamic method invocation and flexible code that adapts to the actual type of an object.
  2. Key Points: Polymorphism comes in two forms: compile-time (static) polymorphism and runtime (dynamic) polymorphism. Compile-time polymorphism is achieved through function overloading and operator overloading. Runtime polymorphism is achieved through function overriding and virtual functions. Virtual functions are declared in base classes with the virtual keyword and must be overridden in derived classes. Polymorphism enables the development of generic code that can work with objects of different types through a common interface. It promotes code extensibility and flexibility, as new classes can be added without modifying existing code that relies on the base class interface.

- Inheritance and Polymorphism Together:

  1. Inheritance and polymorphism often go hand in hand. Inheritance allows you to create class hierarchies where derived classes inherit from a common base class.
  2. Polymorphism enables dynamic behavior by allowing different derived classes to provide their own implementations of base class methods.
  3. Polymorphic function calls, using base class pointers or references, ensure that the correct method is executed based on the actual object type at runtime.
  4. Together, inheritance and polymorphism facilitate the creation of flexible and maintainable code, making C++ a powerful language for object-oriented programming.

Lab 4: Inheritance in C++

1. Task 1: Create Base Class (Shape)

   Create a C++ program that defines a base class called Shape.

   In the Shape class, declare the following attributes: name (string): To store the name of the shape. color (string): To store the color of the shape.

Declare a constructor in the Shape class that initializes name and color.

Declare a virtual function named display() in the Shape class. This function should display the name and color of the shape.

2. Task 2: Create Derived Classes (Circle and Rectangle) Create two derived classes, Circle and Rectangle, that inherit from the Shape class.

   In the Circle class, declare an attribute radius (double) and a constructor that initializes it.

   In the Rectangle class, declare attributes length and width (both double) and a constructor that initializes them.

   Override the display() function in both the Circle and Rectangle classes to display specific information about each shape, including its name, color, and relevant attributes (radius for circles and length/width for rectangles).

3. Task 3: Create Objects and Test

   In the main() function, create objects of both the Circle and Rectangle classes.

   Set the name, color, and specific attributes (radius and length/width) for each object.

   Call the display() function for each object to verify that the correct information is displayed based on the object's type.

Questions

1. What is inheritance in C++? Explain its significance in object-oriented programming.
2. How do you declare a base class in C++? Provide an example of a simple base class.
3. What is a derived class in C++? How does it relate to the base class?
4. Explain the "is-a" relationship between base and derived classes. Provide an example to illustrate this relationship.
5. What is a virtual function in C++? Why is it used in inheritance?
6. How is method overriding achieved in C++? Provide an example of a base class with a virtual function and a derived class that overrides it.
7. Describe the access specifiers (public, protected, private) in C++ inheritance. How do they affect the visibility of inherited members?
8. What is the purpose of the virtual keyword in C++ when used with functions in a base class?
9. How do you prevent a base class function from being overridden in a derived class in C++?

10. Explain the concept of multiple inheritance in C++. What challenges and ambiguities can arise when using it?
11. What is the "diamond problem" in the context of multiple inheritance, and how can it be resolved in C++?
12. How can you create an instance of a derived class in C++, and what happens when you call a base class method on it?
13. What is dynamic polymorphism, and how does it relate to virtual functions and inheritance in C++?
14. Explain the role of destructors in C++ inheritance. When should a base class destructor be declared as virtual?
15. Give an example of a real-world scenario where inheritance in C++ can be applied to model relationships between objects.

Lecture 5: Operator Overloading

Introduction to Operator Overloading

Operator overloading is a feature in C++ that allows you to redefine the behavior of operators when applied to user-defined data types, such as classes and structures. In other words, it enables you to give new meanings to standard C++ operators when used with your custom objects. This feature allows you to write more intuitive and expressive code by extending the functionality of operators to work with user-defined types.

When you overload an operator, you provide a specific implementation for that operator in the context of your custom class or data type. This means you can define how operators like `+`, `-`, `*`, `/`, `==`, `!=`, and others behave when used with objects of your class.

For example, if you have a Complex class to represent complex numbers, you can overload the + operator to add two Complex objects together, making the code more natural and intuitive:

```
 Complex operator+(const Complex& lhs, const Complex& rhs) {
    Complex result;
    result.real = lhs.real + rhs.real;
    result.imaginary = lhs.imaginary + rhs.imaginary;
    return result;
}
```

Operator overloading is a useful feature in C++ for several reasons:

- Intuitive Syntax: Operator overloading allows you to use familiar operators (`e.g.`, `+`, `-`, `*`, `==`) with user-defined types. This makes code more intuitive and readable, as it resembles natural mathematical or logical expressions.
- Simplifies Complex Operations: For complex data types or mathematical structures like matrices, complex numbers, or custom containers, operator overloading can simplify complex operations, reducing the need for verbose function calls.
- Code Clarity: Overloaded operators can lead to code that closely resembles the problem domain, making it easier for developers to understand and maintain. This can improve code clarity and reduce the chances of errors.
- Consistency: Operator overloading allows you to maintain consistency in your code. When you use operators like +, ==, or « with your custom types, you apply the same logical meaning as with built-in types.
- Reusability: Well-designed operator overloads can be reused across different projects and codebases. This reusability saves time and effort when implementing similar functionality.

- Custom Behavior: You can tailor the behavior of operators to your specific needs. For example, you can redefine the + operator to perform vector addition for a physics simulation or concatenate strings in a custom way.
- Expressiveness: Operator overloading can make code more expressive. For example, overloading the « operator for output can allow you to print complex data structures with a single line of code, enhancing the expressiveness of your code.
- Elegant Syntax: Operator overloading can lead to elegant and concise code, which can be especially beneficial in math-heavy or scientific applications.
- Compatibility with Standard Library: When you overload operators consistently with standard library types, you can seamlessly integrate your custom types into the existing ecosystem of C++ libraries and algorithms.
- Object-Oriented Design: Operator overloading aligns well with object-oriented programming (OOP) principles, where objects encapsulate both data and behavior. It allows objects to interact with each other in a more natural and intuitive manner.

However, it's important to use operator overloading judiciously and maintain clear and consistent semantics. Overloading operators in a non-standard or confusing way can lead to code that is difficult to understand and debug. Therefore, it's crucial to follow best practices
and conventions when employing operator overloading in your C++ code.

Overloading Unary Operators

Overloading unary operators in C++ allows you to redefine the behavior of unary operators like +, -, ++, –, and ! when applied to objects of your custom classes. This gives you the flexibility to make your classes work with these operators in a way that is meaningful for your specific application. To overload a unary operator, you need to define a special member function or a non-member function that corresponds to the operator.

Here's a step-by-step guide on how to overload unary operators in C++:

1. Choose the Operator to Overload: Decide which unary operator you want to overload based on the requirements of your class and the desired functionality.
2. Define the Operator Function:

   To overload a unary operator, you need to define a special function with the following syntax:

   ```
   return_type operator<operator_symbol>(parameters);
   ```

   For example, to overload the unary minus operator -, you would define the function as follows:

   ```
   MyClass operator-() {
       // Define custom behavior for the unary minus operator
   }
   ```

3. Implement the Custom Behavior: Inside the operator function, implement the custom behavior you want for the unary operator. This behavior should make sense in the context of your class.

4. Return the Result: Ensure that the operator function returns an object of the same type as your class. This typically involves creating a new object with the modified values or returning the current object with its values changed.

5. Example - Overloading Unary Minus Operator (-): Here's an example of overloading the unary minus operator for a Complex class representing complex numbers:

```
 class Complex {
private:
    double real;
    double imag;

public:
    Complex(double r, double i) : real(r), imag(i) {}

    // Overload the unary minus operator (-)
    Complex operator-() {
        return Complex(-real, -imag);
    }

    // Other member functions and data members...
};
```

   In this example, when you apply the unary minus operator to a Complex object, it negates both the real and imaginary parts of the complex number.

6. Usage Example:

```
Complex a(3.0, 4.0);
Complex b = -a; // Now b contains (-3.0, -4.0)
```

Remember to follow the appropriate naming conventions for operator overloading functions, and ensure that the operator is overloaded in a way that maintains the semantic meaning of the operator in the context of your class.

Overloading Binary Operators

Overloading binary operators in C++ allows you to redefine the behavior of binary operators like `+`, `-`, `*`, `/`, `==`, and many others when applied to objects of your custom classes. This enables you to make your classes work seamlessly with these operators, providing meaningful functionality for your specific application. To overload a binary operator, you need to define a special member function or a non-member function that corresponds to the operator.

Here's a step-by-step guide on how to overload binary operators in C++:

1. Choose the Operator to Overload: Decide which binary operator you want to overload based on the requirements of your class and the desired functionality. 2. Define the Operator Function: To overload a binary operator, you need to define a special function with the following syntax:

```
return_type operator<operator_symbol>(const YourClass& rhs)
    const;
```

For example, to overload the addition operator +, you would define the function as follows:

```
YourClass operator+(const YourClass& rhs) const {
    // Define custom behavior for the addition operator
}
```

1. Implement the Custom Behavior: Inside the operator function, implement the custom behavior you want for the binary operator. This behavior should make sense in the context of your class and provide meaningful results when applied to objects of your class.

2. Return the Result: Ensure that the operator function returns an object of the same type as your class. This typically involves creating a new object with the modified values or returning the current object with its values changed based on the operation.

3. Example - Overloading Addition Operator (+): Here's an example of overloading the addition operator for a Vector class representing mathematical vectors:

```
class Vector {
private:
    double x;
    double y;

public:
    Vector(double xVal, double yVal) : x(xVal), y(yVal) {}

    // Overload the addition operator (+)
    Vector operator+(const Vector& rhs) const {
        return Vector(x + rhs.x, y + rhs.y);
    }

    // Other member functions and data members...
};
```

In this example, when you apply the addition operator to two Vector objects, it performs vector addition by adding their respective x and y components.

4. Usage Example:

```
Vector v1(1.0, 2.0);
Vector v2(2.0, 3.0);
Vector v3 = v1 + v2; // Now v3 contains (3.0, 5.0)
```

5. Overloading Assignment Operator (=)

   When overloading binary operators, it's often a good practice to overload the assignment operator = as well, so you can assign the result of the operator overloads to objects of your class.

```
 YourClass& operator=(const YourClass& rhs) {
    // Define custom assignment behavior
    if (this != &rhs) {
        // Copy values from rhs to this object
        // Don't forget to handle resource management if
           necessary
    }
    return *this;
}
```

Overloading Comparison Operators

Overloading comparison operators in C++ allows you to redefine the behavior of comparison operators like ==, !=, <, >, <=, and >= when applied to objects of your custom classes. This enables you to make your classes work seamlessly with these operators, providing meaningful functionality for comparing objects of your class. To overload a comparison operator, you need to define a special member function or a non-member function that corresponds to the operator.

Here's a step-by-step guide on how to overload comparison operators in C++:

1. Choose the Operator to Overload: Decide which comparison operator you want to overload based on the requirements of your class and the desired functionality.

2. Define the Operator Function:

   To overload a comparison operator, you need to define a special function with the following syntax:

```
  bool operator<operator_symbol>(const YourClass& rhs) const;
```

   For example, to overload the equality operator ==, you would define the function as follows:

```
 bool operator==(const YourClass& rhs) const {
    // Define custom behavior for the equality operator
}
```

3. Implement the Custom Behavior: Inside the operator function, implement the custom behavior you want for the comparison operator. This behavior should make sense in the context of your class and provide meaningful results when comparing objects of your class.

4. Return the Result: Ensure that the operator function returns a bool value (true or false) based on the result of the comparison.

5. Example - Overloading Equality Operator (==): Here's an example of overloading the equality operator for a Person class representing individuals:

```
 class Person {
private:
    std::string name;
    int age;

public:
    Person(const std::string& n, int a) : name(n), age(a) {}

    // Overload the equality operator (==)
    bool operator==(const Person& rhs) const {
        return (name == rhs.name) && (age == rhs.age);
    }

    // Other member functions and data members...
};
```

   In this example, the operator== function checks whether two Person objects have the same name and age.

6. Usage Example:

```
 Person person1("Alice", 30);
Person person2("Bob", 25);
Person person3("Alice", 30);

bool result1 = (person1 == person2); // false
bool result2 = (person1 == person3); // true
```

7. Overloading Other Comparison Operators: You can similarly overload other comparison operators like `!=`, `<`, `>`, `<=`, and `>=` following the same principles.

Overloading Subscript Operator `([])`

Overloading the subscript operator `([])` in C++ allows you to provide custom access to elements of an object, typically when the object represents a collection or container, like an array or a custom data structure. By overloading this operator, you can access elements of your class in a more intuitive and controlled manner.

Here's how to overload the subscript operator in C++:

1. Choose the Data Structure: First, determine the data structure that your class represents and define it internally. It could be an array, a vector, a linked list, or any other data structure you need.

2. Define the Operator Function:

   To overload the subscript operator, you need to define a special member function with the following syntax:

```
T& operator[](size_t index);
```

   Here, T is the type of the elements you want to access, and index is the position of the element you want to access.

3. Implement the Custom Behavior: Inside the operator function, implement the custom behavior for accessing elements based on the given index. Ensure that your implementation correctly handles out-of-range indices, if applicable.

4. Return the Result: The subscript operator should return a reference to the element you want to access. This allows you to both read and modify the element through the subscript operator.

5. Example - Overloading Subscript Operator ([]): Here's an example of overloading the subscript operator for a simple dynamic array class:

```
class DynamicArray {
private:
    int* data;
    size_t size;

public:
    DynamicArray(size_t sz) : size(sz) {
        data = new int[size];
    }

    // Overload the subscript operator []
    int& operator[](size_t index) {
        if (index >= size) {
            throw std::out_of_range("Index out of range");
        }
        return data[index];
    }

    // Destructor to release memory
    ~DynamicArray() {
        delete[] data;
    }
};
```

   In this example, the subscript operator allows you to access elements of the DynamicArray as if it were a regular C++ array.

6. Usage Example:

```
 DynamicArray arr(5);
arr[0] = 10; // Assign value to the first element
int value = arr[0]; // Retrieve the value from the first
   element
```

7. Error Handling: Ensure that you handle out-of-range indices gracefully, either by throwing an exception (as shown in the example) or using an alternative error-handling mechanism that suits your needs.

Overloading Function Call Operator (())

Overloading the function call operator () in C++ allows you to make an object of your class behave like a function when it is invoked. This is also known as creating a functor, which is an object that can be called like a function. Overloading the function call operator enables you to customize the behavior of your class when it is used as if it were a function.

Here's how to overload the function call operator in C++:

1. Define the Operator Function:

   To overload the function call operator, you need to define a special member function with the following syntax:

```
 return_type operator()(parameter_list);
```

   Here, return_type is the type of value you want the "function" to return, and parameter_list represents any parameters you want to pass to the "function."

2. Implement the Custom Behavior: Inside the operator function, implement the custom behavior that you want your object to exhibit when it is called like a function. The behavior should be meaningful and relevant to your class's purpose.

3. Return the Result: Ensure that the operator function returns a value of the specified return_type if applicable

4. Example - Overloading the Function Call Operator (()): Here's an example of overloading the function call operator for a Multiplier class, which multiplies an integer by a specified factor:

```
 class Multiplier {
private:
    int factor;

public:
    Multiplier(int f) : factor(f) {}

    // Overload the function call operator ()
    int operator()(int x) {
```

```
        return x * factor;
    }
};
```

In this example, an object of the Multiplier class can be used as a function that multiplies an integer by the factor specified during its construction.

5. Usage Example:

```
 Multiplier multiplyBy2(2);
int result = multiplyBy2(5); // Calls the Multiplier object
   as if it were a function
// result is now 10
```

6. Customization: Overloading the function call operator allows you to customize the behavior of your class objects in a way that makes sense for your specific application. This can be particularly useful in scenarios where you want to encapsulate complex or specialized functionality within an object.

7. Functor Use Cases: Functors are often used in algorithms that accept a function-like object as a parameter, such as standard library algorithms (std::sort, std::transform) or custom callback mechanisms.

Overloading Type Conversion Operators

Overloading type conversion operators in C++ allows you to define custom type conversions between your user-defined class and other data types. This feature lets you control how objects of your class can be implicitly or explicitly converted to other data types, enhancing the usability and versatility of your class.

Here's how to overload type conversion operators in C++:

1. Choose the Target Data Type: Determine the data type to which you want to convert objects of your class. This could be a built-in type, another user-defined type, or even a standard library type.

2. Define the Conversion Operator Function:

   To overload a type conversion operator, you need to define a member function named operator target_type() where target_type is the desired target data type.

   For example, if you want to allow implicit conversion to an int, you would define the conversion operator as follows:

```
 operator int() {
    // Define custom conversion logic
}
```

3. Implement the Custom Conversion Logic: Inside the conversion operator function, implement the custom logic to convert an object of your class to the target data type. Ensure that the conversion makes sense in the context of your class.

4. Return the Result: Return the value of the target data type after the conversion

5. Example - Overloading Conversion to int: Here's an example of overloading the type conversion operator to allow objects of a Fraction class to be implicitly converted to int by returning the truncated integer part of the fraction

```
class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int num, int denom) : numerator(num),
       denominator(denom) {}

    // Overload the conversion operator to int
    operator int() {
        return numerator / denominator;
    }
};
```

   In this example, when an object of the Fraction class is used in a context that expects an int, the conversion operator is automatically invoked to return the integer part of the fraction.

6. Usage Example:

```
 Fraction fraction(5, 2);
int intValue = fraction; // Implicit conversion to int
// intValue is now 2
```

7. Explicit Conversion: In addition to implicit conversions, you can also provide explicit conversions by defining a constructor or a member function that performs the conversion when explicitly called by the user.

8. Be Careful: When overloading type conversion operators, be mindful of potential loss of data or precision and ensure that the conversion logic is well-defined and safe.

Common Pitfalls and Best Practices

When working with operator overloading in C++, including overloading unary, binary, and comparison operators, it's essential to follow best practices to ensure code correctness, maintainability, and readability. Here are some common pitfalls to avoid and best practices to follow:

Common Pitfalls:

1. Inconsistent Behavior: Ensure that the overloaded operators exhibit consistent behavior and maintain the expected properties of the operators. For example, if you overload the == operator, the != operator should be overloaded accordingly to maintain symmetry.

2. Changing Object State: Be cautious when overloading binary operators to modify the internal state of the objects involved in the operation. Modifying object state without clear documentation can lead to unexpected behavior.

3. Ambiguity: Avoid creating ambiguous situations where the compiler cannot determine which operator to use. This can occur when overloading multiple operators with similar parameter types.

4. Inefficient Implementations: Be mindful of efficiency when overloading operators. Inefficient implementations can lead to performance bottlenecks, especially in critical sections of code.

5. Overloading Too Many Operators: Overloading many operators for a class can lead to code that is difficult to understand and maintain. Focus on overloading operators that make the most sense for your class's purpose.

6. Incorrect Return Types: Ensure that the return type of the overloaded operator matches the expected type based on the operator's semantics.

7. Incomplete Overloading: Overload all relevant operators together. For example, if you overload +, overload -, *, and / as well to maintain consistency.

Best Practices:

1. Follow Standard Semantics: Overloaded operators should maintain the standard semantics of the operator as closely as possible. Users of your class should not be surprised by unexpected behavior.

2. Use Comments and Documentation: Clearly document the behavior of overloaded operators in your class documentation. Explain any non-standard behavior or edge cases.

3. Overload with Purpose: Overload operators with a clear purpose that aligns with the class's intended use. Avoid overloading operators just for the sake of doing so.

4. Keep Operator Overloads Simple: Overloaded operators should have concise and readable implementations. If the logic is complex, consider moving it to a separate function and calling that function from the operator overload.

5. Use const Correctness: Whenever possible, use const correctly to indicate that an operator overload does not modify the state of the object.

6. Test Thoroughly: Write comprehensive unit tests to ensure that overloaded operators work as expected in various scenarios.

7. Avoid Hidden Conversions: Be cautious when overloading operators that can lead to hidden type conversions, as these can lead to unexpected behavior.

8. Consider Overload as Member or Non-Member: Depending on your class's design, you can overload operators as member functions or non-member functions. Member functions provide access to private members, while non-member functions offer more flexibility.

9. Implement Related Operators Together: When overloading operators for a class, implement related operators together to maintain a coherent and consistent interface.

10. Use Existing Conventions: Follow established naming conventions for operator overloads to make your code more readable and maintainable.

By adhering to these best practices and avoiding common pitfalls, you can effectively and safely overload operators in your C++ classes, enhancing code clarity and user experience.

Summary

Operator overloading in C++ is a powerful feature that allows you to redefine the behavior of standard C++ operators for user-defined types, such as classes and structures. Here's a summary of key points regarding operator overloading:

- Purpose: Operator overloading enables you to provide custom functionality for operators when applied to objects of your classes. This makes your code more intuitive and expressive.

- Operators That Can Be Overloaded: Common operators that can be overloaded include arithmetic operators (`+, -, *, /`), `comparison operators (==, !=, <, >, <=, >=), logical operators (&&, ||, !), and many others.`

- Syntax: To overload an operator, you define a special member function or a non-member function with a specific name that corresponds to the operator you want to overload.

- Consistency: Maintain consistency when overloading operators. For example, if you overload == for equality, also overload != for inequality.

- Best Practices: Follow best practices, including clear documentation, efficient implementations, and adherence to standard semantics, to ensure readable and maintainable code.

- Use Cases: Operator overloading is commonly used to make user-defined classes work seamlessly with standard operators, such as overloading + for custom numeric types or == for custom comparison.

- Type Conversion: You can use operator overloading to define custom type conversions between your class and other data types.

- Testing: Thoroughly test overloaded operators to ensure they behave as expected in various scenarios.

- Hidden Conversions: Be cautious about potential hidden type conversions when overloading operators, as these can lead to unexpected behavior.
- Member vs. Non-Member Overloading: Choose whether to overload operators as member functions or non-member functions based on the design and accessibility of your class.
- Code Clarity: Properly overloaded operators can enhance code clarity by allowing you to write code that resembles natural mathematical or logical expressions.
- Considerations: Keep in mind the implications of overloading operators, as misuse can lead to code that is hard to read, debug, and maintain.

Practice Lab

1. Exercise 1: Overloading the Arithmetic Operators In this exercise, you will create a Complex class to represent complex numbers and overload arithmetic operators (+, -, *, /) for it. Implement the following:

   Define the Complex class with real and imaginary parts. Overload the arithmetic operators to perform complex number operations. Implement a display() method to print complex numbers.

2. Exercise 2: Overloading the Equality and Inequality Operators

   In this exercise, you will create a Person class to represent individuals and overload the equality (==) and inequality (!=) operators for it. Implement the following:

   Define the Person class with name and age attributes. Overload the equality and inequality operators to compare Person objects based on their attributes.

3. Exercise 3: Overloading the Index Operator In this exercise, you will create a Matrix class to represent 2D matrices and overload the index operator ([]) to access matrix elements. Implement the following:

   Define the Matrix class with a 2D array. Overload the index operator to access elements using row and column indices.

4. Exercise 4: Overloading the Stream Insertion Operator

   n this exercise, you will create a Student class to represent students' information and overload the stream insertion operator («) to display student details. Implement the following:

   Define the Student class with attributes like name, roll number, and GPA. Overload the stream insertion operator to display student details in a user-friendly format.

5. Exercise 5: Overloading the Function Call Operator

   In this exercise, you will create a Calculator class to perform mathematical operations and overload the function call operator () for it. Implement the following:

   Define the Calculator class with methods for addition, subtraction, multiplication, and division. Overload the function call operator to perform custom mathematical operations.

6. Exercise 6: Overloading the Type Conversion Operator

   In this exercise, you will create a Temperature class to represent temperatures in Fahrenheit and Celsius and overload the type conversion operator to convert temperatures between the two scales. Implement the following:

   Define the Temperature class with attributes for Fahrenheit and Celsius. Overload the type conversion operator to allow conversions between Fahrenheit and Celsius.

7. Exercise 7: Complex Scenario

   Create a complex scenario where you define a custom class, overload multiple operators, and use them in a meaningful way. For example, you could create a class for a banking system and overload operators to handle account transactions.

Questions

1. What is operator overloading in C++? Provide a brief explanation.
2. Why is operator overloading useful in C++? Give an example scenario where operator overloading can improve code readability.
3. What are the limitations or restrictions when it comes to operator overloading in C++?
4. Create a Complex class to represent complex numbers (with real and imaginary parts). Overload the addition operator (+) to perform complex number addition.
5. In the Complex class, overload the subtraction operator (-) to perform complex number subtraction.
6. Define a Date class to represent dates with day, month, and year. Overload the equality (==) operator to compare two Date objects for equality.
7. In the Date class, overload the less-than operator (<) to compare two Date objects to determine which date comes first.
8. Create a Book class to represent books with attributes like title, author, and ISBN. Overload the stream insertion operator («) to display book details.
9. Write a code snippet that demonstrates the use of the overloaded stream insertion operator to display book details.
10. Design a Matrix class to represent 2D matrices with a dynamic array. Overload the index operator ([]) to access matrix elements.
11. Provide an example of how to access an element in a Matrix object using the overloaded index operator.
12. Define a Counter class that keeps track of a count value. Overload the function call operator () to increment the count value by a specified amount.

13. Write a code snippet that demonstrates the use of the overloaded function call operator to increment the count value.

14. Create a Temperature class to represent temperatures in Fahrenheit. Overload the type conversion operator to convert temperatures to Celsius.

15. Provide an example where an object of the Temperature class is implicitly converted to Celsius.

16. Design a class representing a 2D vector and overload the multiplication operator (*) to implement scalar multiplication of vectors.

17. Create a class representing a complex number and overload the division operator (/) to perform complex number division.

18. List three best practices to follow when overloading operators in C++.

19. Why is it essential to maintain consistency when overloading operators? Provide an example scenario.

Lecture 6: C++ Templates: Generic and Reusable Programming

Introduction

One of the most powerful features of the C++ programming language is its support for **generic programming**. Generic programming allows the creation of algorithms and data structures that are independent of the data types they operate on. C++ implements this idea using **templates**.

Templates enable programmers to write flexible, reusable, and type-safe code. Instead of writing multiple versions of a function or class for different data types, a single template definition can handle all of them. This is the foundation of the C++ Standard Template Library (STL).

> *"Templates turn C++ into a language for building languages."* — Bjarne Stroustrup

Why Templates? Motivation and Benefits

Consider writing a simple function that returns the maximum of two values:

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

This works only for integers. To compare floating-point numbers, we must write another function:

```
double max(double a, double b) {
    return (a > b) ? a : b;
}
```

The two functions are nearly identical except for the data type. Templates solve this duplication problem by allowing us to define a single, generic function.

Function Templates

A **function template** defines a family of functions that can operate on different data types. Its general form is:

```
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

Now, `max()` can work with any comparable type:

```
cout << max(3, 5) << endl;          // int
cout << max(2.5, 4.1) << endl;      // double
cout << max('a', 'z') << endl;      // char
```

Template Parameters

The keyword `typename` introduces a *template type parameter*. It can also be written as `class` — both are equivalent in this context:

```
template <class T> // same as template <typename T>
```

Class Templates

A **class template** defines a family of classes parameterized by one or more types. This is extremely useful for data structures such as stacks, queues, and linked lists.

```
template <typename T>
class Stack {
    T data[100];
    int top;
public:
    Stack() : top(-1) {}
    void push(T value) { data[++top] = value; }
    T pop() { return data[top--]; }
    bool isEmpty() const { return top == -1; }
};
```

Usage:

```
Stack<int> s1;
s1.push(10);
s1.push(20);
cout << s1.pop() << endl; // 20

Stack<string> s2;
s2.push("Hello");
s2.push("World");
cout << s2.pop() << endl; // World
```

Template Specialization

Sometimes, we need to customize the behavior of a template for a specific type. This is called **template specialization**.

```
template <typename T>
class Printer {
public:
    void print(T value) {
```

```
        cout << "Generic: " << value << endl;
    }
};

// Specialization for const char*
template <>
class Printer<const char*> {
public:
    void print(const char* value) {
        cout << "String: " << value << endl;
    }
};
```

Usage:

```
Printer<int> p1;
p1.print(100);

Printer<const char*> p2;
p2.print("Mostafa");
```

Non-Type Template Parameters

Templates can also accept constant values as parameters.

```
template <typename T, int size>
class Array {
    T data[size];
public:
    void fill(T value) {
        for (int i = 0; i < size; ++i)
            data[i] = value;
    }
};
```

```
Array<int, 5> a1;
a1.fill(3);
```

Templates and Inheritance

Templates can be used with inheritance to create flexible class hierarchies:

```
template <typename T>
class Base {
protected:
    T value;
public:
    Base(T v) : value(v) {}
    void show() { cout << "Base: " << value << endl; }
```

```
};

template <typename T>
class Derived : public Base<T> {
public:
    Derived(T v) : Base<T>(v) {}
    void showDerived() { cout << "Derived: " << this->value <<
       endl; }
};
```

Templates and Friends

Friend functions can be declared inside templates as well:

```
template <typename T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    friend void show(Box<T> b) {
        cout << "Box: " << b.value << endl;
    }
};
```

Templates and Static Members

Each template instantiation has its own copy of static data members:

```
template <typename T>
class Counter {
public:
    static int count;
    Counter() { ++count; }
};

template <typename T>
int Counter<T>::count = 0;

int main() {
    Counter<int> c1, c2;
    Counter<double> d1;
    cout << Counter<int>::count << endl;    // 2
    cout << Counter<double>::count << endl; // 1
}
```

Common Compilation Errors

- Missing `template` keyword.

- Forgetting to use `typename` in dependent names.
- Template definition not visible at link time (must be in header file).

Modern Extensions in C++ Template Programming

C++ templates have evolved significantly since their introduction in the early 1990s. Modern standards, beginning with C++11 and continuing through C++20 and C++23, introduced a wide range of new features that simplify syntax, improve type safety, and enhance compile-time expressiveness. This section presents a detailed exploration of these extensions, grouped by C++ standard version.

Templates in C++11: The Foundation of Modern Generic Programming

The C++11 standard marks the first major evolution of template programming. It introduced several fundamental concepts that serve as the building blocks for modern generic programming.

Variadic Templates

A **variadic template** allows a function or class template to accept an arbitrary number of template parameters. This generalizes previous designs that required multiple overloads.

Listing 1: Example of a variadic template function.

```
template <typename... Args>
void printAll(Args... args) {
    (std::cout << ... << args) << std::endl;  // Fold expression
       (C++17)
}
```

The ellipsis ('...') denotes a *parameter pack*, which can contain zero or more template arguments.

decltype, auto, and Trailing Return Types

The `decltype` keyword enables type deduction based on an expression, which allows templates to automatically infer result types.

```
template <typename T, typename U>
auto add(T a, U b) -> decltype(a + b) {
    return a + b;
}
```

This feature allows safer and more flexible template definitions.

constexpr Templates

The `constexpr` specifier enables compile-time computation inside templates.

```
template <int N>
constexpr int factorial() {
    return (N <= 1) ? 1 : N * factorial<N - 1>();
}
```

When invoked with constant arguments, the compiler evaluates the function at compile time.

Type Traits and Meta-Programming

The `<type_traits>` library introduced in C++11 provides utilities for compile-time type introspection.

```
#include <type_traits>

static_assert(std::is_integral<int>::value, "int must be integral
    ");
```

These utilities allow compile-time type validation and dispatching (SFINAE).

C++14: Simplicity and Refinement

C++14 further improved template programming by simplifying syntax and enhancing `constexpr` functions.

Generic Lambdas

Lambdas can now have template-like parameters using the `auto` keyword.

```
auto add = [](auto a, auto b) { return a + b; };
```

This feature eliminates the need for explicitly writing function templates for small generic operations.

Relaxed constexpr Rules

C++14 permits local variables and loops inside `constexpr` template functions, making compile-time computation more expressive.

C++17: Expressive Compile-Time Programming

C++17 enhanced template expressiveness and reduced boilerplate code.

Class Template Argument Deduction

C++17 enables deduction of template parameters from constructor arguments.

```
std::pair p(1, 3.14);  // deduces std::pair<int, double>
```

Fold Expressions

Fold expressions simplify operations on parameter packs.

```
template <typename... Args>
auto sum(Args... args) {
    return (args + ... + 0);
}
```

Compile-Time Branching with if constexpr

The `if constexpr` statement allows compile-time conditional logic, eliminating invalid code paths during instantiation.

```
template <typename T>
void printType(const T& value) {
    if constexpr (std::is_integral_v<T>)
        std::cout << "Integral: " << value << std::endl;
    else
        std::cout << "Other type\n";
}
```

Inline Template Variables

The `inline` specifier can be used for static variables in template headers, preventing multiple definition errors.

C++20: Concepts and Constraints

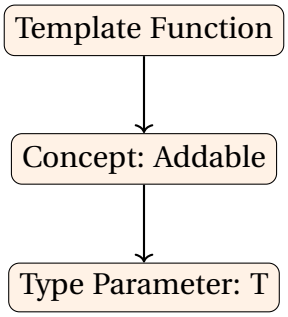
C++20 revolutionized template programming with the introduction of **concepts** and **requires expressions**. These provide a declarative syntax for specifying template constraints, leading to clearer error messages and improved readability.

Concepts

A **concept** defines a set of requirements that a type must satisfy to be used with a given template.

```
template <typename T>
concept Addable = requires(T a, T b) {
    a + b;   // must support addition
};

template <Addable T>
T add(T a, T b) { return a + b; }
```



Requires Expressions

The `requires` expression provides fine-grained control over constraints.

```
template <typename T>
concept HasBegin = requires(T t) {
    { t.begin() } -> std::input_iterator;
};
```

Abbreviated Function Templates

Function templates can now use `auto` directly in their parameter list.

```
auto multiply(auto a, auto b) { return a * b; }
```

Templated Lambdas

Lambdas can now accept explicit template parameters.

```
auto add = []<typename T>(T a, T b) { return a + b; };
```

C++23: Refinement and Polishing

The C++23 standard focuses on improving the usability of existing template mechanisms.

Explicit Object Parameters

You can now explicitly declare the `this` object in a member function template.

```
struct Point {
    int x, y;
    auto distance(this auto self, Point other) {
        return std::hypot(self.x - other.x, self.y - other.y);
    }
};
```

Extended constexpr and Concept Support

C++23 extends `constexpr` to more standard library functions and introduces new concept refinements for range and iterator operations.

Summary of Modern Template Features

Standard	Feature	Description
C++11	Variadic Templates	Functions and classes with arbitrary number of parameters.
	decltype / auto	Type deduction and trailing return types.
	constexpr	Compile-time evaluation.
	type traits	Compile-time type introspection.
C++14	Generic Lambdas	Template-like lambdas with `auto` parameters.
	Relaxed constexpr	Allows loops and local variables.
C++17	if constexpr	Compile-time branching.
	Fold expressions	Simplified parameter pack expansion.
	Deduction guides	Automatic template argument deduction.
C++20	Concepts	Declarative type constraints.
	Requires expressions	Fine-grained type validation.
	Abbreviated templates	Simplified syntax using `auto`.
	Templated lambdas	Generic lambdas with explicit template parameters.
C++23	Explicit object parameters	Explicit `this` in member templates.
	Extended constexpr	Broader compile-time evaluation support.

Lab: Template Programming in C++

Objectives

The goal of this lab is to help students understand the concept of templates in C++, including:

- Writing and using function and class templates.
- Understanding type parameterization and generic programming.
- Using non-type template parameters.
- Applying templates to implement reusable data structures.
- Experimenting with modern extensions like `auto` and `if constexpr`.

Pre-Lab Reading

Before starting this lab, review the following topics:

1. Function templates and class templates.
2. Template instantiation.
3. Type deduction and template specialization.
4. Introduction to C++11/14 template improvements.

Lab Setup

- Programming Language: **C++17** or higher.
- Recommended IDE: Visual Studio Code / CLion / Code::Blocks.
- Compiler: g++ or `clang++` with `-std=c++17`.

Part 1: Function Templates

Start by creating a simple generic function that can work with multiple data types.

Listing 2: Simple function template for maximum value.

```
#include <iostream>
using namespace std;

template <typename T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << "Max of 3 and 7 is " << myMax(3, 7) << endl;
    cout << "Max of 2.5 and 1.1 is " << myMax(2.5, 1.1) << endl;
    cout << "Max of A and Z is " << myMax('A', 'Z') << endl;
    return 0;
}
```

Practice Questions:

1. What happens if you call `myMax(3, 4.5)`? Why?
2. Can you modify the function to support comparison between different types?
3. Rewrite this function using a `concept` (C++20).

Part 2: Class Templates

Templates can be used to design generic classes, such as containers or pairs.

Listing 3: Generic class template for Pair.

```
#include <iostream>
using namespace std;

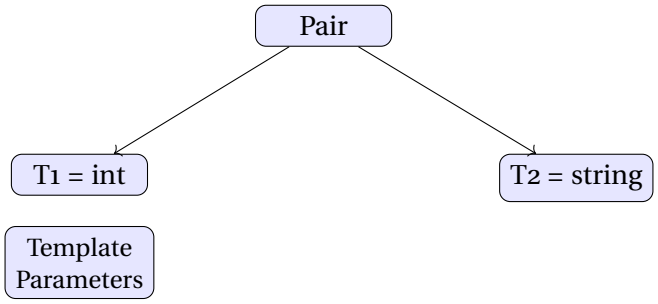
template <typename T1, typename T2>
class Pair {
private:
    T1 first;
    T2 second;
public:
```

```
    Pair(T1 a, T2 b) : first(a), second(b) {}
    void display() const {
        cout << "(" << first << ", " << second << ")" << endl;
    }
};

int main() {
    Pair<int, string> student(101, "Ahmed");
    Pair<string, double> product("Book", 29.5);
    student.display();
    product.display();
}
```

Practice Tasks:

- Add a member function `swap()` to interchange the values of two `Pair` objects.
- Create a `Pair<int, Pair<int, int>>` to represent a nested pair.



—

Part 3: Non-Type Template Parameters

Templates can take values as parameters, not just types.

Listing 4: Non-type template parameter example.

```
#include <iostream>
using namespace std;

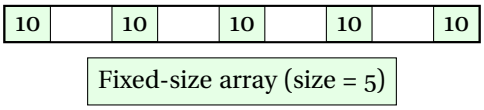
template <typename T, int size>
class Array {
private:
    T data[size];
public:
    void fill(T value) {
        for (int i = 0; i < size; ++i)
            data[i] = value;
    }
    void display() const {
        for (int i = 0; i < size; ++i)
```

```
            cout << data[i] << " ";
        cout << endl;
    }
};

int main() {
    Array<int, 5> arr;
    arr.fill(10);
    arr.display();
}
```

Tasks:

1. Change the template to support a default size.
2. Add a member function `at(int index)` with range checking.



—

Part 4: Template Specialization

You can customize a template for specific data types using **specialization**.

Listing 5: Class template specialization for char*.

```
#include <iostream>
#include <cstring>
using namespace std;

template <typename T>
class Compare {
public:
    static bool isEqual(T a, T b) {
        return a == b;
    }
};

// Specialization for char*
template <>
class Compare<char*> {
public:
    static bool isEqual(const char* a, const char* b) {
        return strcmp(a, b) == 0;
    }
};
```

```cpp
int main() {
    cout << Compare<int>::isEqual(5, 5) << endl;           // true
    cout << Compare<char*>::isEqual("Ahmed", "Mostafa") << endl;
       // false
}
```

Practice Questions:

- Why is `strcmp()` used instead of ==?
- What happens if we do not define this specialization?

—

Part 5: Modern Template Features (C++17 and Beyond)

This part introduces newer C++ features that simplify template code.

Listing 6: Using if constexpr for compile-time branching.

```cpp
#include <iostream>
#include <type_traits>
using namespace std;

template <typename T>
void describe(const T& x) {
    if constexpr (is_integral_v<T>)
        cout << x << " is an integer type." << endl;
    else if constexpr (is_floating_point_v<T>)
        cout << x << " is a floating-point type." << endl;
    else
        cout << "Unknown type." << endl;
}

int main() {
    describe(10);
    describe(3.14);
    describe("Hello");
}
```

Questions:

1. What advantage does `if constexpr` provide over normal `if`?
2. Modify the function to print a message for `std::string`.

Lab Report and Discussion

Deliverables:

- Source code for all exercises.

- Screenshots of successful program outputs.
- Short discussion of how templates improve code reusability.

Discussion Questions:

1. When should a template be preferred over function overloading?
2. How does template specialization improve program flexibility?
3. What are the main differences between compile-time polymorphism (templates) and runtime polymorphism (inheritance)?

Practice Questions on C++ Templates

Templates are a cornerstone of generic programming in C++, allowing functions and classes to operate with generic types. The following questions test both conceptual understanding and practical implementation of template programming in C++.

Basic Level Questions

1. What is a function template? Write a simple function template to return the maximum of two values.
2. Explain the difference between `typename` and `class` in a template parameter list.
3. Write a C++ function template called `swapValues()` that swaps two variables of any data type.
4. What will happen if you instantiate a template with a type that does not support the required operation inside the template body?
5. Why are templates called “compile-time polymorphism”?

Intermediate Level Questions

1. Write a class template `Box<T>` that stores a value of any type and provides `get()` and `set()` functions.
2. Define a template function `sumArray()` that calculates the sum of all elements in an array of any numeric type.
3. What is template instantiation? How does the compiler generate code for different template arguments?
4. Demonstrate how to use default template arguments in a function or class template.
5. Write a program that uses a function template to compare two objects of user-defined type using the > operator.

Advanced Level Questions

1. Define a class template `Pair<T1, T2>` that holds two values of possibly different types, and demonstrate how it can be used.
2. Explain what template specialization is, and provide an example where you specialize a template for `std::string`.
3. Discuss the advantages and disadvantages of using templates in large-scale software development.
4. Write an example of a function template that uses `enable_if` to restrict its use to integral types only.
5. What are template non-type parameters? Provide an example where they are useful.
6. How do variadic templates extend the power of function templates? Illustrate with an example.
7. Explain how templates contribute to the implementation of the C++ Standard Template Library (STL).

Scenario-Based Questions

1. You are designing a data processing library that must handle both integer and floating-point data types. How can templates help you design reusable, type-safe code for computing averages?
2. Suppose you are implementing a generic container class that must support user-defined types. What template features would you use to ensure that the type supports comparison and assignment operations?
3. Imagine you want to design a mathematical utility that supports multiple numeric types and array lengths known at compile-time. Which template mechanisms (type templates, non-type templates, or specialization) would you use, and why?
4. Your team is working on a high-performance graphics engine where performance-critical code must be inlined and type-generic. How would you leverage templates to achieve both flexibility and runtime speed?

Lecture 7: Design Patterns

Design patterns are reusable solutions to common software design problems that occur during the development of software applications. They provide best practices, guidelines, and templates for structuring code and solving recurring design challenges. Design patterns help improve code readability, maintainability, and extensibility while promoting good software engineering practices. Here are some important design patterns:

Creational Patterns

Singleton Pattern

Ensures that a class has only one instance and provides a global point of access to that instance.

The Singleton Pattern is a creational design pattern that ensures a class has only one instance and provides a global point of access to that instance. It is one of the simplest design patterns but can be quite powerful in scenarios where exactly one object should coordinate actions across the system, such as a configuration manager, a logging service, or a database connection pool.

Key characteristics of the Singleton Pattern:

- Single Instance: A Singleton class should have only one instance, which is created lazily or at the time of the first request.
- Global Access: It provides a way to access the single instance from any part of the application, typically through a static method.
- Initialization Control: The pattern allows for careful control over how and when the singleton instance is created, ensuring that it meets specific requirements.

Here's a typical implementation of the Singleton Pattern in C++:

```
class Singleton {
private:
    // Private constructor to prevent external instantiation
    Singleton() {}

    // Private static instance variable
    static Singleton* instance;

public:
    // Static method to access the singleton instance
    static Singleton* getInstance() {
        if (!instance) {
```

```
            instance = new Singleton();
        }
        return instance;
    }

    // Other member methods and data...
};

// Initialize the static instance variable
Singleton* Singleton::instance = nullptr;
```

In this example:

1. The constructor of the Singleton class is private, preventing external instantiation.
2. The getInstance method is used to get the single instance of the Singleton class. If an instance does not exist, it is created lazily.
3. The static instance variable holds the reference to the single instance, and it is initialized as nullptr.

Here's how you can use the Singleton:

```
int main() {
    Singleton* singleton1 = Singleton::getInstance();
    Singleton* singleton2 = Singleton::getInstance();

    // Both pointers point to the same instance
    if (singleton1 == singleton2) {
        std::cout << "Singleton instance is the same." << std::
           endl;
    } else {
        std::cout << "Singleton instance is different." << std::
           endl;
    }

    // Clean up (optional)
    delete singleton1;
    return 0;
}
```

It's important to note that the Singleton Pattern can be implemented in various ways, and the example provided above is a simple one. Depending on your requirements and the complexity of your application, you may need to consider thread safety, lazy initialization, or other aspects when implementing a Singleton. Additionally, some modern C++ standards and libraries provide alternative approaches to ensure thread safety and lazy initialization.

Factory Method Pattern

Defines an interface for creating an object, but leaves the choice of its type to subclasses, creating a framework for object creation.

The Factory Method Pattern is a creational design pattern that provides an interface for creating objects but allows subclasses to alter the type of objects that will be created. It is particularly useful when you have a class hierarchy of related classes, and you want to delegate the responsibility of object creation to subclasses, allowing for flexibility and customization.

Key characteristics of the Factory Method Pattern:

1. Abstract Creator: The pattern defines an abstract class (or interface) known as the "Creator" that declares a factory method. This method is responsible for creating objects of a related "Product" class.

2. Concrete Creators: Concrete subclasses of the Creator implement the factory method to produce specific instances of the Product class.

3. Product Class: The Product class is the common interface for all objects created by the factory method. It defines the properties and behaviors that all concrete product classes must implement.

   Here's a typical implementation of the Factory Method Pattern in C++:

```
// Product interface
class Product {
public:
    virtual void doSomething() = 0;
    virtual ~Product() {}
};

// Concrete Product A
class ConcreteProductA : public Product {
public:
    void doSomething() override {
        std::cout << "Product A does something." << std::endl
            ;
    }
};

// Concrete Product B
class ConcreteProductB : public Product {
public:
    void doSomething() override {
        std::cout << "Product B does something." << std::endl
            ;
    }
};

// Creator abstract class
```

```
class Creator {
public:
    virtual Product* factoryMethod() = 0;
};

// Concrete Creator A
class ConcreteCreatorA : public Creator {
public:
    Product* factoryMethod() override {
        return new ConcreteProductA();
    }
};

// Concrete Creator B
class ConcreteCreatorB : public Creator {
public:
    Product* factoryMethod() override {
        return new ConcreteProductB();
    }
};
```

In this example:

1. The Product interface defines the common interface for all products created by the factory method.
2. Concrete products, such as ConcreteProductA and ConcreteProductB, implement the Product interface.
3. The Creator abstract class declares the factory method, which returns a Product object. Concrete creators, such as ConcreteCreatorA and ConcreteCreatorB, implement this method to create specific products.

Here's how you can use the Factory Method Pattern:

```
int main() {
   Creator* creatorA = new ConcreteCreatorA();
   Product* productA = creatorA->factoryMethod();
   productA->doSomething();

   Creator* creatorB = new ConcreteCreatorB();
   Product* productB = creatorB->factoryMethod();
   productB->doSomething();

   delete creatorA;
   delete creatorB;
   delete productA;
   delete productB;
```

```
    return 0;
}
```

Abstract Factory Pattern

Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

The Abstract Factory Pattern is a creational design pattern that provides an interface for creating families of related or dependent objects without specifying their concrete classes. It allows you to create objects that are part of a common theme or family, ensuring that the created objects are compatible with each other.

Key characteristics of the Abstract Factory Pattern:

1. Abstract Factory: The pattern defines an abstract factory interface that declares a set of creation methods for creating related objects, often organized into families.
2. Concrete Factories: Concrete subclasses of the abstract factory implement the factory interface. Each concrete factory is responsible for creating a specific family of related objects.
3. Abstract Products: The pattern defines abstract product interfaces for each type of object that the factories can create. These interfaces declare the common methods that all concrete products must implement.
4. Concrete Products: Concrete product classes implement the abstract product interfaces. Each concrete factory creates its own set of concrete products.

Here's a typical implementation of the Abstract Factory Pattern in C++:

```
 // Abstract Product A
class AbstractProductA {
public:
    virtual void operationA() = 0;
    virtual ~AbstractProductA() {}
};

// Concrete Product A1
class ConcreteProductA1 : public AbstractProductA {
public:
    void operationA() override {
        std::cout << "Product A1 does operation A." << std::endl;
    }
};

// Concrete Product A2
class ConcreteProductA2 : public AbstractProductA {
public:
    void operationA() override {
```

```
        std::cout << "Product A2 does operation A." << std::endl;
    }
};

// Abstract Product B
class AbstractProductB {
public:
    virtual void operationB() = 0;
    virtual ~AbstractProductB() {}
};

// Concrete Product B1
class ConcreteProductB1 : public AbstractProductB {
public:
    void operationB() override {
        std::cout << "Product B1 does operation B." << std::endl;
    }
};

// Concrete Product B2
class ConcreteProductB2 : public AbstractProductB {
public:
    void operationB() override {
        std::cout << "Product B2 does operation B." << std::endl;
    }
};

// Abstract Factory
class AbstractFactory {
public:
    virtual AbstractProductA* createProductA() = 0;
    virtual AbstractProductB* createProductB() = 0;
    virtual ~AbstractFactory() {}
};

// Concrete Factory 1
class ConcreteFactory1 : public AbstractFactory {
public:
    AbstractProductA* createProductA() override {
        return new ConcreteProductA1();
    }

    AbstractProductB* createProductB() override {
        return new ConcreteProductB1();
    }
};

// Concrete Factory 2
class ConcreteFactory2 : public AbstractFactory {
```

```
public:
    AbstractProductA* createProductA() override {
        return new ConcreteProductA2();
    }

    AbstractProductB* createProductB() override {
        return new ConcreteProductB2();
    }
};
```

In this example:

1. There are two families of products, A and B, each with two concrete implementations (e.g., ConcreteProductA1, ConcreteProductA2, ConcreteProductB1, ConcreteProductB2).
2. The AbstractFactory interface declares the creation methods for product A and product B.
3. Concrete factories (ConcreteFactory1 and ConcreteFactory2) implement the AbstractFactory interface and create products from their respective families.

Here's how you can use the Abstract Factory Pattern:

```
int main() {
    AbstractFactory* factory1 = new ConcreteFactory1();
    AbstractProductA* productA1 = factory1->createProductA();
    AbstractProductB* productB1 = factory1->createProductB();

    AbstractFactory* factory2 = new ConcreteFactory2();
    AbstractProductA* productA2 = factory2->createProductA();
    AbstractProductB* productB2 = factory2->createProductB();

    // Use the created products...

    delete factory1;
    delete factory2;
    delete productA1;
    delete productB1;
    delete productA2;
    delete productB2;

    return 0;
}
```

Builder Pattern

Separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

The Builder Pattern is a creational design pattern that separates the construction of a complex object from its representation, allowing the same construction process to create different representations of the object. It is particularly useful when you need to create objects with many optional components or configurations.

Key characteristics of the Builder Pattern:

- Director: The pattern includes a director class that orchestrates the construction process. The director uses a builder interface to construct the object step by step.
- Builder Interface: The builder interface defines a set of methods for building various parts of the complex object. These methods typically include functions for setting attributes, adding components, and finalizing the object.
- Concrete Builders: Concrete builder classes implement the builder interface and provide specific implementations for constructing objects with different configurations.
- Product: The complex object being constructed is referred to as the "product." The builder constructs and returns the product once it's complete.
- Client: The client code interacts with the director to initiate the construction process, specifying which concrete builder to use. The client can also access the constructed product when the builder is done.

Here's a typical implementation of the Builder Pattern in C++:

```
 #include <iostream>
#include <string>

// Product class
class Computer {
public:
    void setCPU(const std::string& cpu) {
        cpu_ = cpu;
    }

    void setRAM(const std::string& ram) {
        ram_ = ram;
    }

    void setStorage(const std::string& storage) {
        storage_ = storage;
    }

    void displayInfo() {
        std::cout << "CPU: " << cpu_ << std::endl;
        std::cout << "RAM: " << ram_ << std::endl;
        std::cout << "Storage: " << storage_ << std::endl;
    }
```

```
private:
    std::string cpu_;
    std::string ram_;
    std::string storage_;
};

// Builder interface
class ComputerBuilder {
public:
    virtual void buildCPU() = 0;
    virtual void buildRAM() = 0;
    virtual void buildStorage() = 0;
    virtual Computer* getResult() = 0;
};

// Concrete builder for a gaming computer
class GamingComputerBuilder : public ComputerBuilder {
public:
    void buildCPU() override {
        computer_->setCPU("High-end Gaming CPU");
    }

    void buildRAM() override {
        computer_->setRAM("16GB Gaming RAM");
    }

    void buildStorage() override {
        computer_->setStorage("512GB SSD");
    }

    Computer* getResult() override {
        return computer_;
    }

    GamingComputerBuilder() {
        computer_ = new Computer();
    }

private:
    Computer* computer_;
};

// Director
class ComputerDirector {
public:
    Computer* build(ComputerBuilder* builder) {
        builder->buildCPU();
        builder->buildRAM();
        builder->buildStorage();
```

```
        return builder->getResult();
    }
};

int main() {
    ComputerDirector director;
    GamingComputerBuilder gamingBuilder;

    Computer* gamingComputer = director.build(&gamingBuilder);
    gamingComputer->displayInfo();

    delete gamingComputer;

    return 0;
}
```

In this example:

1. We have a Computer class representing the complex object to be built.
2. The ComputerBuilder interface defines methods for building various parts of the computer.
3. GamingComputerBuilder is a concrete builder that implements the builder interface to create a gaming computer with specific components.
4. The ComputerDirector orchestrates the construction process by using a builder to create a computer.
5. The client code (in main) interacts with the director to create a gaming computer.

Structural Patterns

Adapter Pattern

Allows the interface of an existing class to be used as another interface.

The Adapter Pattern is a structural design pattern that allows objects with incompatible interfaces to work together. It acts as a bridge between two interfaces, enabling them to collaborate without changing their source code. The Adapter Pattern is particularly useful when integrating existing classes or libraries with new code or when dealing with legacy systems.

Key characteristics of the Adapter Pattern:

- Target: The "target" is the interface that the client code expects to work with. It defines the operations or methods that the client code uses.
- Adaptee: The "adaptee" is the class or component that has a different interface from the target. It is the class that needs to be integrated or adapted to work with the client code.

- Adapter: The "adapter" is a class that implements the target interface while containing an instance of the adaptee. It acts as an intermediary, translating requests from the client code into calls to the adaptee's methods.

Here's a typical implementation of the Adapter Pattern in C++:

```
 #include <iostream>

// Target interface
class Target {
public:
    virtual void request() = 0;
};

// Adaptee
class Adaptee {
public:
    void specificRequest() {
        std::cout << "Adaptee's specific request." << std::endl;
    }
};

// Adapter
class Adapter : public Target {
private:
    Adaptee* adaptee_;

public:
    Adapter(Adaptee* adaptee) : adaptee_(adaptee) {}

    void request() override {
        std::cout << "Adapter is adapting the request." << std::
           endl;
        adaptee_->specificRequest();
    }
};

int main() {
    Adaptee* adaptee = new Adaptee();
    Target* adapter = new Adapter(adaptee);

    adapter->request();

    delete adaptee;
    delete adapter;

    return 0;
}
```

In this example:

- Target is the interface that the client code expects, and it defines the request method.
- Adaptee is the class with a different interface that needs to be integrated. It has a specificRequest method.
- Adapter is a class that implements the Target interface and contains an instance of Adaptee. It adapts the request method to call specificRequest internally.
- In the main function, an Adaptee object is created, and then an Adapter object is created, passing the Adaptee as a parameter. The Adapter is used to make the request.

Decorator Pattern

Attaches additional responsibilities to an object dynamically, providing a flexible alternative to subclassing.

The Decorator Pattern is a structural design pattern that allows you to attach additional responsibilities to an object dynamically. It is used to extend the functionality of objects in a flexible and reusable way without altering their source code. This pattern is based on the principle of composition rather than inheritance, which promotes open-closed principle compliance (classes are open for extension but closed for modification).

Key characteristics of the Decorator Pattern:

- Component: The "component" is an abstract class or interface that defines the common interface for concrete components and decorators.
- Concrete Component: A "concrete component" is a class that implements the component interface. It represents the base object to which additional responsibilities can be added.
- Decorator: The "decorator" is an abstract class that also implements the component interface. It has an association with the component and can add or override behaviors.
- Concrete Decorators: "Concrete decorators" are classes that extend the decorator class. They provide specific implementations for adding responsibilities to the component.

Here's a typical implementation of the Decorator Pattern in C++:

```
 #include <iostream>

// Component interface
class Coffee {
public:
    virtual std::string getDescription() const = 0;
    virtual double cost() const = 0;
};
```

```
// Concrete Component
class Espresso : public Coffee {
public:
    std::string getDescription() const override {
        return "Espresso";
    }

    double cost() const override {
        return 1.99;
    }
};

// Decorator abstract class
class CoffeeDecorator : public Coffee {
protected:
    Coffee* coffee;

public:
    CoffeeDecorator(Coffee* coffee) : coffee(coffee) {}

    std::string getDescription() const override {
        return coffee->getDescription();
    }

    double cost() const override {
        return coffee->cost();
    }
};

// Concrete Decorator 1
class MilkDecorator : public CoffeeDecorator {
public:
    MilkDecorator(Coffee* coffee) : CoffeeDecorator(coffee) {}

    std::string getDescription() const override {
        return coffee->getDescription() + ", Milk";
    }

    double cost() const override {
        return coffee->cost() + 0.50;
    }
};

// Concrete Decorator 2
class SugarDecorator : public CoffeeDecorator {
public:
    SugarDecorator(Coffee* coffee) : CoffeeDecorator(coffee) {}
```

```
    std::string getDescription() const override {
        return coffee->getDescription() + ", Sugar";
    }

    double cost() const override {
        return coffee->cost() + 0.25;
    }
};

int main() {
    Coffee* espresso = new Espresso();
    std::cout << "Order: " << espresso->getDescription()
              << ", Cost: $" << espresso->cost() << std::endl;

    Coffee* milkEspresso = new MilkDecorator(espresso);
    std::cout << "Order: " << milkEspresso->getDescription()
              << ", Cost: $" << milkEspresso->cost() << std
                 ::endl;

    Coffee* sugarMilkEspresso = new SugarDecorator(milkEspresso);
    std::cout << "Order: " << sugarMilkEspresso->getDescription()
          << ", Cost: $" << sugarMilkEspresso->cost() << std::
             endl;

    delete espresso;
    delete milkEspresso;
    delete sugarMilkEspresso;

    return 0;
}
```

In this example:

- Coffee is the component interface with getDescription and cost methods.
- Espresso is a concrete component that implements the Coffee interface.
- CoffeeDecorator is an abstract decorator class that also implements the Coffee interface and has an association with a Coffee object.
- MilkDecorator and SugarDecorator are concrete decorators that extend CoffeeDecorator. They add milk and sugar to the coffee description and cost.

In the main function, decorators are used to enhance an Espresso object with additional responsibilities.

Composite Pattern

Composes objects into tree structures to represent part-whole hierarchies. Clients can treat individual objects and compositions of objects uniformly.

The Composite Pattern is a structural design pattern that allows you to compose objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects and compositions of objects uniformly. The Composite Pattern is particularly useful when you need to work with complex structures that have a recursive or hierarchical nature.

Key characteristics of the Composite Pattern:

1. Component: The "component" is an abstract class or interface that defines the common interface for both leaf and composite objects. It usually includes methods for operations that can be performed on both types.

2. Leaf: A "leaf" is a concrete class that implements the component interface. Leaf objects represent the individual objects that make up the composite.

3. Composite: A "composite" is a concrete class that also implements the component interface but can contain child components. Composite objects represent containers that can hold both leaf and other composite objects.

Here's a typical implementation of the Composite Pattern in C++:

```
#include <iostream>
#include <vector>

// Component interface
class Component {
public:
    virtual void operation() const = 0;
    virtual ~Component() {}
};

// Leaf
class Leaf : public Component {
public:
    void operation() const override {
        std::cout << "Leaf does something." << std::endl;
    }
};

// Composite
class Composite : public Component {
private:
    std::vector<Component*> children;

public:
    void add(Component* component) {
        children.push_back(component);
    }

    void operation() const override {
```

```
        std::cout << "Composite does something." << std::endl;
        for (const auto& child : children) {
            child->operation();
        }
    }

    ~Composite() {
        for (auto& child : children) {
            delete child;
        }
    }
};

int main() {
    Leaf* leaf1 = new Leaf();
    Leaf* leaf2 = new Leaf();
    Composite* composite = new Composite();

    composite->add(leaf1);
    composite->add(leaf2);

    composite->operation();

    delete composite;

    return 0;
}
```

In this example:

1. Component is the component interface with an operation method.
2. Leaf is a concrete leaf class that implements the Component interface. It represents individual objects.
3. Composite is a concrete composite class that also implements the Component interface and contains a collection of child components. It can represent complex structures by recursively calling operation on its children.
4. In the main function, leaf and composite objects are created and organized into a hierarchical structure. The composite's operation method traverses the entire hierarchy, executing the operation method of each component.

Proxy Pattern

Provides a surrogate or placeholder for another object to control access to it.

The Proxy Pattern is a structural design pattern that provides a surrogate or placeholder for another object to control access to it. It allows you to add an extra layer of

control over the access to an object, typically to implement lazy initialization, access control, logging, monitoring, or other functionality.

Key characteristics of the Proxy Pattern:

1. Real Subject: The "real subject" is the actual object that the proxy represents. It is the object to which access is being controlled or augmented.
2. Proxy: The "proxy" is an interface or class that provides the same interface as the real subject. It acts as an intermediary between the client and the real subject, controlling access and adding behavior as needed.
3. Lazy Initialization: Proxies are often used for lazy initialization, meaning that the real subject is only created or initialized when it is actually needed, not when the proxy is created.
4. Access Control: Proxies can enforce access control rules, such as checking permissions or authentication, before allowing the client to access the real subject.
5. Monitoring and Logging: Proxies can be used to monitor and log the interactions between the client and the real subject for debugging or auditing purposes.

Here's a typical implementation of the Proxy Pattern in C++:

```
#include <iostream>

// Subject interface
class Subject {
public:
    virtual void request() = 0;
    virtual ~Subject() {}
};

// Real Subject
class RealSubject : public Subject {
public:
    void request() override {
        std::cout << "RealSubject handles the request." << std::
           endl;
    }
};

// Proxy
class Proxy : public Subject {
private:
    RealSubject* realSubject;

public:
    Proxy() : realSubject(nullptr) {}

    void request() override {
```

```
        // Lazy initialization: Create the RealSubject only when
           needed
        if (!realSubject) {
            realSubject = new RealSubject();
        }

        // Additional behavior, if necessary, can be added here
        std::cout << "Proxy handles the request." << std::endl;

        // Delegate the request to the RealSubject
        realSubject->request();
    }

    ~Proxy() {
        delete realSubject;
    }
};

int main() {
    Proxy proxy;

    // The client interacts with the Proxy, which controls access
        to the RealSubject
    proxy.request();

    return 0;
}
```

In this example:

Subject is the subject interface that both RealSubject and Proxy implement. It defines the request method.

RealSubject is the actual object that the proxy represents. It implements the Subject interface and performs the real operation when requested.

Proxy is a proxy class that also implements the Subject interface. It contains an instance of RealSubject and controls access to it. It performs lazy initialization and additional behavior (printing a message) before delegating the request to the RealSubject.

In the main function, the client interacts with the Proxy object, which in turn controls access to the RealSubject.

Behavioral Patterns

Observer Pattern

Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

The Observer Pattern is a behavioral design pattern that defines a one-to-many dependency between objects. It allows multiple objects (observers) to be notified and updated automatically when the state of a subject (the object being observed) changes.

This pattern is commonly used in software architectures where changes to one object need to be reflected in other objects without them being tightly coupled.

Key characteristics of the Observer Pattern:

1. Subject: The "subject" (also known as the "observable") is the object being observed. It maintains a list of observers and notifies them when its state changes.
2. Observer: An "observer" is an object that wants to be notified when the subject's state changes. Observers register themselves with the subject and implement an update method to respond to notifications.
3. Registration and Deregistration: Observers can dynamically register and deregister themselves with a subject to start or stop receiving notifications.
4. Notification: When the subject's state changes, it notifies all registered observers by calling their update methods.
5. Loose Coupling: The Observer Pattern promotes loose coupling between subjects and observers. Subjects don't need to know the specific classes of their observers, and observers are not tightly coupled to the subject they observe.

Here's a typical implementation of the Observer Pattern in C++:

```
 #include <iostream>
#include <vector>

// Observer interface
class Observer {
public:
    virtual void update(const std::string& message) = 0;
};

// Concrete Observer
class ConcreteObserver : public Observer {
private:
    std::string name;

public:
    ConcreteObserver(const std::string& _name) : name(_name) {}

    void update(const std::string& message) override {
        std::cout << "Observer " << name << " received message: "
            << message << std::endl;
    }
};

// Subject interface
class Subject {
private:
    std::vector<Observer*> observers;
```

```
    std::string state;

public:
    void attach(Observer* observer) {
        observers.push_back(observer);
    }

    void detach(Observer* observer) {
        // Implement observer removal logic here (not shown in
           this simple example)
    }

    void setState(const std::string& newState) {
        state = newState;
        notify();
    }

    void notify() {
        for (Observer* observer : observers) {
            observer->update(state);
        }
    }
};

int main() {
    Subject subject;

    ConcreteObserver observer1("Observer 1");
    ConcreteObserver observer2("Observer 2");

    subject.attach(&observer1);
    subject.attach(&observer2);

    subject.setState("New state!");

    return 0;
}
```

In this example:

1. Observer is the observer interface that defines the update method.
2. ConcreteObserver is a concrete observer class that implements the Observer interface. It responds to notifications by printing a message.
3. Subject is the subject class that maintains a list of observers and notifies them when its state changes. It provides methods for attaching observers, detaching observers (not shown in this simple example), setting its state, and notifying observers.

4. In the main function, two observers are created, attached to the subject, and notified when the subject's state changes.

Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable. It allows the algorithm to vary independently from clients that use it.

The Strategy Pattern is a behavioral design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable. It allows a client to choose an algorithm from a family of algorithms at runtime, providing flexibility in selecting the appropriate strategy without altering the client code.

Key characteristics of the Strategy Pattern:

1. Context: The "context" is the class that contains the strategy and maintains a reference to the selected strategy. The context is responsible for invoking the strategy.

2. Strategy: The "strategy" is an interface or abstract class that defines a set of methods or operations. Concrete strategy classes implement these methods to provide different algorithmic behaviors.

3. Concrete Strategies: "Concrete strategies" are the actual implementations of the algorithms. Each concrete strategy class implements the strategy interface and provides a specific algorithmic behavior.

4. Dynamic Switching: The Strategy Pattern allows the client (context) to switch between different strategies at runtime, making it possible to choose the most appropriate strategy for a given situation.

Here's a typical implementation of the Strategy Pattern in C++:

```
 #include <iostream>

// Strategy interface
class PaymentStrategy {
public:
    virtual void pay(int amount) = 0;
};

// Concrete Strategy 1
class CreditCardPayment : public PaymentStrategy {
public:
    void pay(int amount) override {
        std::cout << "Paid " << amount << " via Credit Card." <<
           std::endl;
    }
};

// Concrete Strategy 2
```

```
class PayPalPayment : public PaymentStrategy {
public:
    void pay(int amount) override {
        std::cout << "Paid " << amount << " via PayPal." << std::
           endl;
    }
};

// Context
class ShoppingCart {
private:
    PaymentStrategy* paymentStrategy;

public:
    void setPaymentStrategy(PaymentStrategy* strategy) {
        paymentStrategy = strategy;
    }

    void checkout(int amount) {
        paymentStrategy->pay(amount);
    }
};

int main() {
    ShoppingCart cart;

    PaymentStrategy* creditCardStrategy = new CreditCardPayment()
       ;
    PaymentStrategy* paypalStrategy = new PayPalPayment();

    cart.setPaymentStrategy(creditCardStrategy);
    cart.checkout(100);

    cart.setPaymentStrategy(paypalStrategy);
    cart.checkout(200);

    delete creditCardStrategy;
    delete paypalStrategy;

    return 0;
}
```

In this example:

1. PaymentStrategy is the strategy interface that defines the pay method.
2. CreditCardPayment and PayPalPayment are concrete strategy classes that implement the PaymentStrategy interface with specific payment methods.

3. ShoppingCart is the context class that maintains a reference to the selected payment strategy and uses it to perform the payment when the checkout method is called.

4. In the main function, two different payment strategies are created, and the ShoppingCart object switches between them at runtime to complete the checkout process.

Command Pattern

Encapsulates a request as an object, thereby allowing for parameterization of clients with queues, requests, and operations.

The Command Pattern is a behavioral design pattern that encapsulates a request as an object, thereby allowing for parameterization of clients with queues, requests, and operations. It enables the separation of the sender (invoker) of a request from the receiver (handler) of that request, providing flexibility and extensibility in how requests are processed.

Key characteristics of the Command Pattern:

1. Command: The "command" is an object representing an action or request. It typically includes a method (e.g., execute) that is called to perform the action.

2. Concrete Command: A "concrete command" is a class that implements the command interface and defines the specific operation to be performed. It holds a reference to the receiver.

3. Receiver: The "receiver" is the object that knows how to perform the action. It has the necessary methods and state to carry out the request.

4. Invoker: The "invoker" is responsible for storing and invoking commands. It does not know the specifics of how a command is executed; it simply calls the command's execute method.

5. Client: The "client" creates and assembles the commands, receivers, and invokers, thereby defining how the system's behavior should be configured.

Here's a typical implementation of the Command Pattern in C++:

```
 #include <iostream>
#include <vector>

// Command interface
class Command {
public:
    virtual void execute() = 0;
};

// Receiver
class Light {
```

```
public:
    void turnOn() {
        std::cout << "Light is ON." << std::endl;
    }

    void turnOff() {
        std::cout << "Light is OFF." << std::endl;
    }
};

// Concrete Command 1
class TurnOnCommand : public Command {
private:
    Light* light;

public:
    TurnOnCommand(Light* l) : light(l) {}

    void execute() override {
        light->turnOn();
    }
};

// Concrete Command 2
class TurnOffCommand : public Command {
private:
    Light* light;

public:
    TurnOffCommand(Light* l) : light(l) {}

    void execute() override {
        light->turnOff();
    }
};

// Invoker
class RemoteControl {
private:
    Command* command;

public:
    void setCommand(Command* cmd) {
        command = cmd;
    }

    void pressButton() {
        command->execute();
    }
```

```
};

int main() {
    Light livingRoomLight;
    Light kitchenLight;

    TurnOnCommand livingRoomOn(&livingRoomLight);
    TurnOffCommand livingRoomOff(&livingRoomLight);

    TurnOnCommand kitchenOn(&kitchenLight);
    TurnOffCommand kitchenOff(&kitchenLight);

    RemoteControl remote;

    // Configure the remote control
    remote.setCommand(&livingRoomOn);
    remote.pressButton();

    remote.setCommand(&kitchenOn);
    remote.pressButton();

    remote.setCommand(&livingRoomOff);
    remote.pressButton();

    return 0;
}
```

In this example:

1. Command is the command interface that defines the execute method.
2. Light is the receiver that knows how to perform the actions, in this case, turning on and off the light.
3. TurnOnCommand and TurnOffCommand are concrete command classes that implement the Command interface and specify which action to execute. They hold references to the appropriate Light object.
4. RemoteControl is the invoker, which holds a reference to the command and can press the button to execute the command.
5. In the main function, we create instances of Light, TurnOnCommand, TurnOffCommand, and RemoteControl to configure and execute commands to control the lights.

State Pattern

Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

The State Pattern is a behavioral design pattern that allows an object to alter its behavior when its internal state changes. The pattern represents each possible state as an object and delegates the state-specific behavior to these objects. This enables an object to appear as if it changes its class when its internal state changes.

Key characteristics of the State Pattern:

1. Context: The "context" is the class that contains the state and maintains a reference to the current state object. The context delegates state-specific behavior to the current state object.

2. State: "State" objects are classes that represent specific states and implement the behavior associated with those states. Each state typically provides methods for handling events or operations.

3. Context Interface: The context defines an interface that allows clients to request operations. These operations are typically delegated to the current state object.

4. Concrete States: "Concrete states" are classes that implement the state interface. They provide the specific behavior associated with a particular state.

Here's a typical implementation of the State Pattern in C++:

```
 #include <iostream>

// State interface
class State {
public:
    virtual void handle() = 0;
};

// Concrete State 1
class State1 : public State {
public:
    void handle() override {
        std::cout << "State 1 is handling the request." << std::
           endl;
    }
};

// Concrete State 2
class State2 : public State {
public:
    void handle() override {
        std::cout << "State 2 is handling the request." << std::
           endl;
    }
};

// Context
```

```
class Context {
private:
    State* currentState;

public:
    Context(State* initialState) : currentState(initialState) {}

    void setState(State* newState) {
        currentState = newState;
    }

    void request() {
        currentState->handle();
    }
};

int main() {
    State1 state1;
    State2 state2;

    Context context(&state1);

    context.request();

    // Change the state dynamically
    context.setState(&state2);
    context.request();

    return 0;
}
```

In this example:

1. State is the state interface that defines the handle method for handling requests.
2. State1 and State2 are concrete state classes that implement the State interface. They provide specific behavior associated with different states.
3. Context is the context class that maintains a reference to the current state and delegates requests to the current state object.
4. In the main function, we create instances of State1, State2, and Context to demonstrate how the context can change its behavior dynamically by switching between different states.

Practice Lab

1. Exercise 1: Singleton Pattern

   Implement a Singleton pattern to create a configuration manager that loads application settings from a file. Ensure that only one instance of the configuration manager exists.

2. Exercise 2: Factory Method Pattern

   Create a simple factory method for creating different shapes (e.g., Circle, Rectangle, Triangle). Implement classes for each shape and use the factory to create instances.

3. Exercise 3: Observer Pattern

   Implement an observer pattern for a weather monitoring system. Create a subject that generates random weather data and observers that display the weather.

4. Exercise 4: Decorator Pattern

   Create a text editor application with a Text class. Implement decorators for adding different formatting options (e.g., bold, italic, underline) to the text.

5. Exercise 5: Command Pattern

   Design a remote control for a smart home system. Create command objects for controlling lights, thermostat, and other devices. Use the command pattern to execute actions.

6. Exercise 6: State Pattern

   Implement a vending machine that dispenses different items based on its state. Use the state pattern to represent the vending machine's states (e.g., no coin, has coin, dispensing).

7. Exercise 7: Strategy Pattern

   Develop a payment processing system. Implement payment strategies (e.g., Credit Card, PayPal, Bitcoin) that can be selected at runtime to process payments.

8. Exercise 8: Composite Pattern

   Create a file system representation with directories and files. Implement a composite pattern to manage the hierarchy and perform operations like calculating the total size of a directory.

9. Exercise 9: Builder Pattern

   Build a custom burger ordering system. Implement a builder pattern to create complex burger orders with various components (e.g., patty type, toppings, sauces).

10. Exercise 10: Proxy Pattern

    Design a proxy to restrict access to a sensitive database. Only authorized users should be able to perform database operations through the proxy.

11. Exercise 11: Abstract Factory Pattern

    Extend the factory method exercise (Exercise 2) to use the abstract factory pattern to create families of related objects, such as different styles of shapes.

12. Exercise 12: Template Method Pattern

    Create a template for generating reports (e.g., PDF report, HTML report). Implement subclasses that provide specific report content and formatting.

Lecture 8: Standard Template Library (STL)

What is the Standard Template Library (STL)?

The Standard Template Library (STL) is a powerful and versatile library in C++ that provides a collection of generic template classes and functions to simplify common programming tasks. It is a fundamental part of the C++ Standard Library, offering reusable and efficient algorithms and data structures. The STL aims to promote code reusability, maintainability, and performance in C++ applications.

Key Components of the STL

The STL consists of several key components, each designed to address specific programming needs:

Containers

Containers are classes that hold collections of objects or data. STL provides a variety of container classes, including:

- Vectors: Dynamic arrays that can grow or shrink in size.
- Lists: Doubly linked lists that allow efficient insertions and deletions.
- Deques: Double-ended queues that offer fast insertions and deletions at both ends.
- Sets: Containers that store unique elements in a sorted order.
- Maps: Containers that store key-value pairs in a sorted order.

Algorithms

Algorithms in the STL are a set of reusable functions for performing various operations on containers or sequences. These include sorting, searching, and manipulation operations. Some commonly used algorithms include sort(), find(), for_each(), and more.

Iterators

Iterators are objects that provide a way to access elements in a container in a consistent and abstract manner. They act as a bridge between algorithms and containers, allowing you to traverse and manipulate container elements. Common types of iterators include begin(), end(), rbegin(), and rend().

Function Objects (Functors)

Function objects, also known as functors, are objects that behave like functions. They are often used as arguments to algorithms to customize their behavior. Functors are implemented as classes with overloaded operator().

Allocators

Allocators are used to manage memory allocation and deallocation for containers. While they are typically abstracted away in most cases, advanced users can customize memory management using allocators.

Benefits of Using the STL

Using the STL offers several advantages to C++ programmers:

Code Reusability

The STL provides a wide range of generic classes and functions, reducing the need to write custom data structures and algorithms. Developers can leverage pre-implemented solutions for common tasks.

Standardization

The STL is part of the C++ Standard Library, ensuring that it is available and consistent across different C++ compilers and platforms. This standardization simplifies cross-platform development.

Performance

STL containers and algorithms are optimized for efficiency and speed. The library is designed to be highly efficient, making it suitable for various applications, including performance-critical ones.

Safety

STL containers and algorithms are designed to be safe to use. They perform bounds checking and ensure memory safety, helping to prevent common programming errors.

Using the STL in Your Code

To utilize the STL in your C++ programs, include the appropriate headers and make use of the provided classes and functions. Here's a simple example that demonstrates the usage of a vector, an algorithm, and an iterator:

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```cpp
int main() {
    // Create a vector of integers
    std::vector<int> numbers = {5, 2, 9, 1, 5, 6};

    // Use an STL algorithm (sort) to sort the vector
    std::sort(numbers.begin(), numbers.end());

    // Use an STL iterator to print the sorted vector
    for (const int& num : numbers) {
        std::cout << num << " ";
    }

    return 0;
}
```

Example of STL Containers

Vectors

Imagine std::vector as a magical resizable array in C++. It's like having a container that can automatically grow or shrink as you add or remove elements, without you needing to worry about the underlying memory management.

- Auto-Sizing: When you insert elements into a std::vector, it expands itself as needed to accommodate the new data. You don't have to specify the size upfront; the vector adapts on the fly.
- Easy Access: You can access elements by their position just like in a regular array, using indices. It's like having a collection of numbered drawers, and you can open any drawer to get what you want.
- Efficient Packing: std::vector packs its elements tightly in memory, making it efficient for tasks like looping through all the items. Elements are neighbors in memory, so it's like flipping through pages in a book.
- Self-Cleaning: When you're done with a std::vector, it takes care of cleaning up the memory it used. It's like having a magical butler who cleans up your room when you leave.
- Customizable: You can tell the vector how much space you expect to need (without putting things in it), so it can prepare in advance. It's like reserving seats in a restaurant before you arrive.
- Adaptable: If you decide you need more room, you can add more elements, and the vector expands to fit. If you want to slim it down, you can remove elements, and it shrinks accordingly.

In essence, std::vector is your dynamic, self-adjusting, and memory-savvy array companion in C++, always ready to accommodate your data as your program runs.

Here is an example:

```
#include <iostream>
#include <vector>

int main() {
    // Create a std::vector to store a list of integers
    std::vector<int> numbers;

    // Add some numbers to the vector
    numbers.push_back(5);
    numbers.push_back(2);
    numbers.push_back(9);
    numbers.push_back(1);
    numbers.push_back(5);
    numbers.push_back(6);

    // Display the contents of the vector
    std::cout << "Numbers in the vector: ";
    for (const int& num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    // Calculate the sum of the numbers
    int sum = 0;
    for (const int& num : numbers) {
        sum += num;
    }
    std::cout << "Sum of the numbers: " << sum << std::endl;

    // Find and display the maximum element
    int max = numbers[0];
    for (const int& num : numbers) {
        if (num > max) {
            max = num;
        }
    }
    std::cout << "Maximum number: " << max << std::endl;

    // Remove the first element from the vector
    numbers.erase(numbers.begin());

    // Display the updated vector
    std::cout << "Numbers after erasing the first element: ";
    for (const int& num : numbers) {
        std::cout << num << " ";
```

```
    }
    std::cout << std::endl;

    return 0;
}
```

In this example:

- We include the necessary headers for std::vector.
- We create a std::vector called numbers to store a list of integers.
- We use push_back() to add numbers to the vector.
- We iterate through the vector to display its contents, calculate the sum, and find the maximum number.
- We remove the first element using erase().
- Finally, we display the updated vector.

This example showcases the basic operations you can perform with std::vector, such as adding, accessing, iterating, and modifying elements. std::vector is a flexible and convenient container for working with dynamic lists of data in C++.

Arrays

std::array is a container class in C++ that represents a fixed-size, static array with several advantages over traditional arrays. It is part of the Standard Library and provides type and bounds safety, making it a safer alternative to C-style arrays. Here are some key characteristics and features of std::array:

- Fixed Size: std::array has a fixed size determined at compile time. Once you create an std::array, its size cannot be changed. This feature ensures compile-time safety and predictability
- Type and Bounds Safety: std::array provides type safety, meaning you can't accidentally mix different data types. It also performs bounds checking, preventing you from accessing elements outside the valid range.
- Efficient: Like traditional arrays, std::array elements are stored in contiguous memory, making element access and iteration efficient.
- Iterators: You can use iterators with std::array just like with other STL containers, allowing for a consistent way to traverse and manipulate elements.
- Standard Library Compatibility: std::array conforms to the SequenceContainer concept, making it compatible with many STL algorithms and functions.

- Initialization: std::array provides various ways to initialize its elements, including aggregate initialization (similar to plain arrays), fill constructors, and copy constructors.

Here's a practical example of using std::array to store and manipulate a list of integers:

```
#include <iostream>
#include <array>

int main() {
    // Declare and initialize an std::array of integers
    std::array<int, 5> numbers = {1, 2, 3, 4, 5};

    // Access elements by index
    std::cout << "Element at index 2: " << numbers[2] << std::
       endl;

    // Modify an element
    numbers[3] = 10;

    // Calculate the sum of elements
    int sum = 0;
    for (const int& num : numbers) {
        sum += num;
    }
    std::cout << "Sum of elements: " << sum << std::endl;

    // Size and iteration
    std::cout << "Size of the array: " << numbers.size() << std::
       endl;
    std::cout << "Elements in the array: ";
    for (const int& num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

In this example:

- We include the necessary headers for std::array.
- We declare and initialize an std::array called numbers with a fixed size of 5.
- We access and modify elements using the subscript operator ([]).
- We calculate the sum of elements using a loop.
- We use size() to get the size of the array.

- We iterate through the array using a range-based for loop.

std::array combines the safety of a fixed-size array with the convenience and compatibility of STL containers, making it a valuable tool for managing arrays in C++.

List

std::list is a doubly linked list container class provided by the C++ Standard Library as part of the Standard Template Library (STL). Unlike std::vector, which uses dynamic arrays, std::list uses linked nodes to store its elements. Here are some key characteristics and features of std::list:

- Doubly Linked List: std::list is implemented as a doubly linked list, which means that each element in the list has pointers to both the next and the previous elements. This allows for efficient insertions and deletions at both the beginning and the end of the list, as well as at arbitrary positions.
- Efficient Insertions and Deletions: Inserting or removing an element at the beginning or end of a std::list is a constant-time operation, O(1). Insertions and deletions at arbitrary positions also have good time complexity, O(N), where N is the distance from the beginning or end of the list.
- Iterators: std::list provides bidirectional iterators, which allow you to traverse the list in both forward and backward directions. This makes it suitable for scenarios where you need to access elements in reverse order.
- Dynamic Sizing: Unlike std::array, which has a fixed size, or std::vector, which dynamically resizes itself, std::list can dynamically grow or shrink as elements are inserted or removed. This makes it useful for scenarios where the number of elements is not known in advance.
- Memory Overhead: Due to its doubly linked nature, std::list has higher memory overhead compared to arrays and vectors. Each element in the list requires additional memory for two pointers (next and previous).
- No Contiguous Memory: Elements in a std::list are not stored in contiguous memory locations, which can impact cache locality and random access performance. It's optimized for insertions and deletions, not for quick indexing.

Here's a basic example of using std::list to store and manipulate a list of integers:

```
#include <iostream>
#include <list>

int main() {
    // Declare and initialize an std::list of integers
    std::list<int> numbers = {1, 2, 3, 4, 5};

    // Insert an element at the beginning of the list
```

```
    numbers.push_front(0);

    // Insert an element at the end of the list
    numbers.push_back(6);

    // Iterate through the list using an iterator
    std::cout << "Elements in the list: ";
    for (const int& num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    // Remove the first and last elements
    numbers.pop_front();
    numbers.pop_back();

    // Display the updated list
    std::cout << "Elements after removal: ";
    for (const int& num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

In this example:

- We include the necessary headers for std::list.
- We declare and initialize an std::list called numbers.
- We use push_front() and push_back() to insert elements at the beginning and end of the list.
- We use an iterator to iterate through the list and display its contents.
- We use pop_front() and pop_back() to remove the first and last elements.
- Finally, we display the updated list.

std::list is a useful container when you need efficient insertions and deletions, especially at both ends, and when you don't require constant-time random access.

Set

std::set is a container class provided by the C++ Standard Library as part of the Standard Template Library (STL). It represents a collection of unique, sorted elements. Here are some key characteristics and features of std::set:

- Unique Elements: In a std::set, each element is unique. Duplicate values are automatically eliminated, ensuring that the set contains only distinct values.
- . Sorted Order: Elements in a std::set are automatically sorted in ascending order. This sorting is performed using a comparison function, which can be customized.
- Efficient Search: Searching for an element in a std::set is an efficient operation, typically performed in logarithmic time (O(log N)), where N is the number of elements in the set. This makes it suitable for tasks that require quick lookups.
- nsertions and Deletions: Inserting and deleting elements in a std::set are also efficient operations, performed in logarithmic time. The set automatically maintains its sorted order after these operations.
- Iterators: std::set provides bidirectional iterators, allowing you to traverse the set in both forward and backward directions. This is useful for tasks that involve iterating through elements.
- . Custom Comparisons: You can customize the comparison function used for sorting elements in the set. By default, elements are sorted using the less-than (<) operator, but you can provide your own comparison function for more complex data types.
- Memory Overhead: std::set has a memory overhead due to the need to maintain unique and sorted elements. Each element is typically associated with additional memory for internal data structures.

Here's a basic example of using std::set to store and manipulate a set of integers:

```
#include <iostream>
#include <set>

int main() {
    // Declare and initialize an std::set of integers
    std::set<int> numbers = {5, 2, 9, 1, 5, 6};

    // Insert an element into the set
    numbers.insert(8);

    // Check if an element is present in the set
    if (numbers.count(5) > 0) {
        std::cout << "5 is in the set." << std::endl;
    } else {
        std::cout << "5 is not in the set." << std::endl;
    }

    // Iterate through the set using an iterator
    std::cout << "Elements in the set: ";
    for (const int& num : numbers) {
        std::cout << num << " ";
```

```
    }
    std::cout << std::endl;

    // Remove an element from the set
    numbers.erase(1);

    // Display the updated set
    std::cout << "Elements after removal: ";
    for (const int& num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

In this example:

- We include the necessary headers for std::set.
- We declare and initialize an std::set called numbers.
- We use insert() to add an element to the set.
- . We use count() to check if an element exists in the set.
- We use an iterator to iterate through the set and display its contents.
- We use erase() to remove an element from the set.
- Finally, we display the updated set.

std::set is a valuable container when you need a collection of unique, sorted elements and efficient search operations.

Maps

std::map is a container class provided by the C++ Standard Library as part of the Standard Template Library (STL). It represents an associative container that stores a collection of key-value pairs, where each key is unique, and the keys are sorted in ascending order. Here are some key characteristics and features of std::map:

- Associative: std::map is an associative container, meaning it associates keys with values. You can look up values by their corresponding keys.
- Unique Keys: In a std::map, each key is unique. This ensures that there are no duplicate keys in the map.
- Sorted Keys: Keys in a std::map are automatically sorted in ascending order. This allows for efficient key-based searches and traversals.

- Efficient Lookup: Searching for a key in a std::map is an efficient operation, typically performed in logarithmic time (O(log N)), where N is the number of key-value pairs in the map.
- Insertions and Deletions: Inserting, updating, and deleting key-value pairs in a std::map are also efficient operations, performed in logarithmic time. The map maintains its sorted order after these operations.
- Custom Comparisons: You can customize the comparison function used for sorting keys in the map. By default, keys are sorted using the less-than (<) operator, but you can provide your own comparison function for more complex data types.
- Iterators: std::map provides bidirectional iterators, allowing you to traverse the map in both forward and backward directions. This is useful for tasks that involve iterating through key-value pairs.
- Memory Overhead: std::map has a memory overhead due to the need to maintain unique and sorted keys. Each key-value pair is typically associated with additional memory for internal data structures.

Here's a basic example of using std::map to store and manipulate a map of names and ages:

```
#include <iostream>
#include <map>

int main() {
    // Declare and initialize an std::map of names (keys) and
       ages (values)
    std::map<std::string, int> ageMap;

    // Insert key-value pairs into the map
    ageMap["Alice"] = 25;
    ageMap["Bob"] = 30;
    ageMap["Charlie"] = 22;

    // Look up the age of a person by name
    std::string nameToLookup = "Bob";
    if (ageMap.find(nameToLookup) != ageMap.end()) {
        std::cout << nameToLookup << "'s age is " << ageMap[
           nameToLookup] << " years." << std::endl;
    } else {
        std::cout << nameToLookup << " not found in the map." <<
           std::endl;
    }

    // Iterate through the map using an iterator
    std::cout << "Contents of the map:" << std::endl;
    for (const auto& entry : ageMap) {
```

```
        std::cout << entry.first << ": " << entry.second << "
            years" << std::endl;
    }

    return 0;
}
```

In this example:

- We include the necessary headers for std::map.
- We declare and initialize an std::map called ageMap to store names as keys and ages as values.
- We use the subscript operator (`[]`) to insert key-value pairs into the map.
- We use find() to check if a key exists in the map before accessing its value.
- We use an iterator to iterate through the map and display its contents.

std::map is a versatile container when you need to maintain a collection of key-value pairs with unique keys and sorted keys for efficient lookups.

Pairs

std::pair is a simple but versatile utility class provided by the C++ Standard Library. It allows you to create a pair of two objects, which can be of different types. This class is frequently used for various purposes, especially in scenarios where you need to group two values together. Here are some key characteristics and features of std::pair:

- Two-Element Container: std::pair can hold two values, referred to as "first" and "second" elements. These elements can be of different types.
- Value Semantics: Like most C++ objects, std::pair exhibits value semantics, meaning that copies of pairs are created with distinct copies of the stored elements.
- Flexible Types: You can use std::pair to store values of any data type, including user-defined types. For example, you can use it to store a pair of integers, a pair of strings, or even a pair of custom objects.
- Comparison Operations: std::pair supports comparison operations, so you can compare pairs using operators like `<`, `>`, `<=`, `and` `>=` based on their first elements and then their second elements.
- Convenient for Functions: Pairs are often used to return multiple values from functions. Instead of defining a custom struct or class for this purpose, you can use std::pair to bundle the values together.

Here's a basic example of using std::pair to store and manipulate a pair of integers:

```cpp
#include <iostream>
#include <utility> // Required for std::pair

int main() {
    // Declare and initialize a pair of integers
    std::pair<int, int> coordinates = std::make_pair(3, 4);

    // Access and modify the elements of the pair
    std::cout << "Coordinates: (" << coordinates.first << ", " <<
        coordinates.second << ")" << std::endl;

    // Compare pairs based on their first elements
    std::pair<int, int> otherCoordinates = std::make_pair(2, 5);
    if (coordinates < otherCoordinates) {
        std::cout << "These coordinates are closer to the origin.
           " << std::endl;
    } else {
        std::cout << "These coordinates are farther from the
           origin." << std::endl;
    }

    return 0;
}
```

In this example:

- We include the necessary header for std::pair.
- We declare and initialize an std::pair called coordinates with two integers.
- We access and modify the elements of the pair using .first and .second.
- We compare pairs based on their first elements to determine which set of coordinates is closer to the origin.

std::pair is a convenient and widely used tool in C++ for working with pairs of values. It's often employed in a wide range of scenarios, from simple data grouping to returning multiple values from functions.

Tuples

std::tuple is a versatile container provided by the C++ Standard Library that allows you to store a fixed-size collection of elements. Each element in a tuple can be of a different type, making it useful for scenarios where you need to bundle together values of different data types. Tuples are indexed, ordered, and allow you to access their elements by position.

Key Features of std::tuple

- Heterogeneous Elements: One of the primary advantages of std::tuple is that it can hold elements of different types. This is in contrast to arrays or vectors, where all elements must have the same data type.

- Fixed Size: The size of a tuple is determined at compile time. Once you define the number and types of elements in a tuple, it cannot be changed without altering the code.
- Ordered: Elements in a tuple are ordered and can be accessed by their position within the tuple. This order is established at compile time and remains consistent.
- Value Semantics: Tuples have value semantics, meaning they create copies of their elements when copied. Modifications to one tuple do not affect another tuple.
- Structured Binding (C++17 and later): C++17 introduced structured binding, which allows you to easily decompose a tuple into individual variables. This makes working with tuples more convenient.
- Compile-Time Type Safety: The types of elements in a tuple are known at compile time, providing type safety. This means you can't accidentally access elements with the wrong data type.
- Comparison Operators: Tuples support comparison operators (e.g., ==, <, >), making it possible to compare tuples for equality or order.

Here's how you can create and access a std::tuple:

```
#include <iostream>
#include <tuple>

int main() {
    // Creating a tuple with elements of different types
    std::tuple<int, double, std::string> myTuple(42, 3.14, "Hello
       ");

    // Accessing elements by position
    int intValue = std::get<0>(myTuple);
    double doubleValue = std::get<1>(myTuple);
    std::string stringValue = std::get<2>(myTuple);

    std::cout << "Tuple elements: " << intValue << ", " <<
       doubleValue << ", " << stringValue << std::endl;

    return 0;
}
```

In this example:

- We include the necessary header for std::tuple.
- We create a tuple myTuple containing an integer, a double, and a string.
- We access elements using std::get<index>(tuple), where index is the position of the element in the tuple.

Queues

std::queue is a container adapter provided by the C++ Standard Library that represents a queue data structure. A queue follows the First-In-First-Out (FIFO) principle, meaning that the element added first is the one that will be removed first. std::queue is typically implemented using other standard containers, such as std::deque (double-ended queue) or std::list. Here are some key characteristics and features of std::queue:

- FIFO Behavior: Elements are inserted at the back (enqueue) and removed from the front (dequeue) of the queue. This ensures that the element that has been in the queue the longest is the first to be removed.
- Container Adapter: std::queue is a container adapter, which means that it is built on top of an underlying container (usually std::deque by default). The choice of the underlying container can be customized if needed.
- Limited Access: Access to elements in a queue is limited. You can only access the front element (the one to be dequeued next) and the back element (the one most recently enqueued).
- No Random Access: Unlike arrays or vectors, you cannot directly access elements at arbitrary positions within a queue. It's optimized for efficient enqueuing and dequeuing.
- Constant-Time Operations: Enqueuing and dequeuing operations are typically performed in constant time, O(1), making queues efficient for managing a collection of elements where order matters.
- . Empty Queue: It provides methods like empty() to check if the queue is empty, which is useful for handling boundary cases.

Here's a basic example of using std::queue to manage a queue of integers:

```
#include <iostream>
#include <queue>

int main() {
    // Declare a queue of integers
    std::queue<int> myQueue;

    // Enqueue elements
    myQueue.push(10);
    myQueue.push(20);
    myQueue.push(30);

    // Dequeue and print elements
    while (!myQueue.empty()) {
        std::cout << "Dequeued: " << myQueue.front() << std::endl
            ;
        myQueue.pop(); // Remove the front element
```

```
    }

    return 0;
}
```

std::queue is commonly used in various applications where elements are processed in a specific order, such as task scheduling, breadth-first search algorithms, and managing a print job queue in a printer.

Optional

std::optional is a relatively new addition to the C++ Standard Library, introduced in C++17. It provides a way to represent values that may or may not be present, and it's particularly useful in situations where you need to handle optional or nullable values without resorting to null pointers or special sentinel values. Here are some key characteristics and features of std::optional:

- Optional Values: std::optional allows you to represent values that may or may not exist. It encapsulates the idea of optionality in a type-safe manner.
- Value Semantics: Instances of std::optional have value semantics, meaning that they own their values. Copying an optional creates a new instance with a copy of the value.
- Type Safety: The type of the value contained in the optional is known at compile time, providing strong type safety. This eliminates common runtime errors associated with nullable pointers.
- Efficient Storage: std::optional is designed to be efficient in terms of storage. It only holds a value when it's present, which means no extra memory overhead is incurred when the value is absent.
- Accessing Values: You can access the value within an optional using the * dereference operator or the -> arrow operator, similar to working with pointers.
- Checking for Presence: You can check if an optional has a value using the has_value() member function or the bool conversion operator.
- Value Initialization: You can initialize an optional without a value, making it empty. This can be useful when you don't have a valid value to provide initially.
- Use Cases: std::optional is useful for function return values that may fail, configuration settings that may or may not be present, and handling nullable values in general.

Here's a basic example of using std::optional to represent an optional integer value:

```cpp
#include <iostream>
#include <optional>

int main() {
    // Declare an optional integer
    std::optional<int> optionalValue;

    // Check if the optional has a value
    if (optionalValue.has_value()) {
        std::cout << "Value is present: " << *optionalValue <<
           std::endl;
    } else {
        std::cout << "Value is not present." << std::endl;
    }

    // Assign a value to the optional
    optionalValue = 42;

    // Access the value within the optional
    if (optionalValue.has_value()) {
        std::cout << "Value is present: " << *optionalValue <<
           std::endl;
    } else {
        std::cout << "Value is not present." << std::endl;
    }

    return 0;
}
```

In this example:

- We include the necessary header for std::optional.
- . We declare an std::optional called optionalValue to hold an integer.
- We check if the optional has a value using has_value().
- We assign a value to the optional using the assignment operator.
- We access the value within the optional using *optionalValue.

std::optional is a powerful tool for handling optional values in a type-safe and expressive manner, reducing the risk of null pointer-related errors in your code.

Example of STL Algorithms

Find

std::find is a C++ Standard Library algorithm that's part of the <algorithm> header. It's used to search for a specific value within a range, such as an array, a container, or

any sequence that provides iterators. std::find is a simple yet versatile tool for finding elements in a collection. Here are its key features:

- . Search for Value: std::find is used to locate a specific value within a given range. It searches for the first occurrence of the value.
- Range-Based: It operates on any sequence-like data structure that provides iterators. This includes arrays, vectors, lists, sets, maps, and more.
- Type-Agnostic: std::find is not tied to a specific data type. You can search for values of any type that supports comparison using ==.
- Returns an Iterator: When the value is found, std::find returns an iterator pointing to the first occurrence of the value. If the value is not found, it returns the end iterator of the range.
- Linear Search: The algorithm performs a linear search, comparing each element in the range with the target value until it finds a match or reaches the end.

Here's a basic example of using std::find to search for an integer within a vector:

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};

    int target = 3;

    // Use std::find to search for the target value
    auto result = std::find(numbers.begin(), numbers.end(),
       target);

    if (result != numbers.end()) {
        std::cout << "Value " << target << " found at position "
           << std::distance(numbers.begin(), result) << std::endl
           ;
    } else {
        std::cout << "Value " << target << " not found." << std::
           endl;
    }

    return 0;
}
```

In this example:

- We include the necessary headers for vectors, algorithms, and I/O.
- We create a vector of integers called numbers and initialize it with some values.

- We specify the target value we want to find (in this case, 3).
- We use std::find to search for the target value within the range defined by numbers.begin() and numbers.end().
- If the value is found (result is not equal to numbers.end()), we print its position using std::distance.

std::find is a fundamental algorithm in C++ that's commonly used for searching within collections, and it's a building block for more complex operations.

Find_if

std::find_if is another C++ Standard Library algorithm provided by the <algorithm> header. It's an extension of std::find and is used for searching a range based on a specified predicate or condition. std::find_if allows you to search for an element that satisfies a specific condition rather than just looking for an exact value. Here are its key features:

- Search with a Predicate: std::find_if searches for the first element in a range that satisfies a given predicate or condition.
- Range-Based: Similar to std::find, it operates on sequences that provide iterators, including arrays, vectors, lists, sets, maps, and custom containers.
- Type-Agnostic: You can use std::find_if to search for elements of any data type as long as you define a suitable predicate.
- Returns an Iterator: When the predicate is satisfied, std::find_if returns an iterator pointing to the first element that meets the condition. If the predicate is never satisfied, it returns the end iterator of the range.
- Linear Search: Like std::find, this algorithm performs a linear search, checking each element in the range against the predicate until a match is found or the end of the range is reached.

Here's a basic example of using std::find_if to search for an element greater than a specific threshold in a vector of integers:

```
 #include <iostream>
#include <vector>
#include <algorithm>

bool isGreaterThanThreshold(int value, int threshold) {
    return value > threshold;
}

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};

    int threshold = 3;
```

```
    // Use std::find_if with a custom predicate
    auto result = std::find_if(numbers.begin(), numbers.end(), [
       threshold](int value) {
        return value > threshold;
    });

    if (result != numbers.end()) {
        std::cout << "First value greater than " << threshold <<
           " found: " << *result << std::endl;
    } else {
        std::cout << "No value greater than " << threshold << "
           found." << std::endl;
    }

    return 0;
}
```

In this example:

- We include the necessary headers for vectors, algorithms, and I/O.
- We define a custom predicate function isGreaterThanThreshold that checks if a value is greater than a threshold.
- . We create a vector of integers called numbers and initialize it with some values.
- We specify the threshold value we want to use for comparison.
- We use std::find_if to search for the first element greater than the threshold, and we provide the custom predicate as a lambda function.

std::find_if is a powerful algorithm for searching based on custom conditions, making it versatile for a wide range of applications where you need to find elements that meet specific criteria.

for_each

std::for_each is a C++ Standard Library algorithm provided by the <algorithm> header. It is used for performing a specified operation on each element within a given range, such as an array, a container, or any sequence that provides iterators. std::for_each is often used when you need to apply a function or perform an action on every element in a collection. Here are its key features:

- Apply a Function: std::for_each applies a given function or callable object to each element in the specified range.
- Range-Based: It operates on sequences that provide iterators, including arrays, vectors, lists, sets, maps, and custom containers.

- Type-Agnostic: You can use std::for_each to process elements of any data type as long as you define a suitable function or callable object.
- Returns Nothing: std::for_each does not return any value. It is typically used for its side effects, such as modifying elements in place or performing some action on each element.
- Sequential Processing: The function is applied sequentially to each element in the order defined by the iterator range.

Here's a basic example of using std::for_each to print each element of a vector of integers:

```
#include <iostream>
#include <vector>
#include <algorithm>

void printValue(int value) {
    std::cout << value << " ";
}

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};

    // Use std::for_each to apply the printValue function to each
        element
    std::for_each(numbers.begin(), numbers.end(), printValue);

    std::cout << std::endl;

    return 0;
}
```

In this example:

- We include the necessary headers for vectors, algorithms, and I/O.
- We define a custom function printValue that takes an integer and prints it.
- We define a custom function printValue that takes an integer and prints it.
- We create a vector of integers called numbers and initialize it with some values.
- We use std::for_each to apply the printValue function to each element within the range defined by numbers.begin() and numbers.end().

The result of running this code will be the values in the vector printed to the console.

std::for_each is a convenient algorithm when you want to perform a consistent operation on each element in a collection without manually writing loops. It promotes a more declarative and functional style of programming.

Count

std::count is a C++ Standard Library algorithm provided by the <algorithm> header. It is used for counting the occurrences of a specific value within a given range, such as an array, a container, or any sequence that provides iterators. std::count is a simple yet powerful tool for finding out how many times a particular value appears in a collection. Here are its key features:

- Count Occurrences: std::count counts the number of times a specific value appears within a given range.
- Range-Based: It operates on sequences that provide iterators, including arrays, vectors, lists, sets, maps, and custom containers.
- Type-Agnostic: You can use std::count to count occurrences of elements of any data type that supports comparison using ==.
- Returns an Integer: std::count returns an integer representing the count of occurrences of the specified value within the range.
- Linear Search: The algorithm performs a linear search, comparing each element in the range with the target value and incrementing the count when a match is found.

Here's a basic example of using std::count to count the number of occurrences of a specific integer value in a vector:

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numbers = {1, 2, 2, 3, 2, 4, 2, 5};

    int target = 2;

    // Use std::count to count the occurrences of the target
       value
    int count = std::count(numbers.begin(), numbers.end(), target
       );

    std::cout << "Value " << target << " appears " << count << "
       times." << std::endl;

    return 0;
}
```

In this example:

- We include the necessary headers for vectors, algorithms, and I/O.

- We create a vector of integers called numbers and initialize it with some values.
- We specify the target value we want to count (in this case, 2).
- We use std::count to count the occurrences of the target value within the range defined by numbers.begin() and numbers.end().

The result of running this code will be the number of times the target value appears in the vector.

std::count is a convenient algorithm when you need to quickly determine how many times a specific value occurs in a collection, and it's commonly used in tasks like data analysis and statistics.

`std::all_of`

The algorithm `std::all_of` returns `true` if the predicate holds for *every* element in the range. It short-circuits, meaning it stops as soon as an element violates the predicate.

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> nums = {2, 4, 6, 8};

    bool allEven = std::all_of(nums.begin(), nums.end(),
                               [](int x){ return x % 2 == 0; });

    std::cout << std::boolalpha << "All even? " << allEven << "\n
       ";
}
```

Output:

```
All even? true
```

`std::any_of`

The algorithm `std::any_of` returns `true` if *at least one* element in the range satisfies the predicate.

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 3, 5, 10};

    bool hasEven = std::any_of(nums.begin(), nums.end(),
                               [](int x){ return x % 2 == 0; });
```

```
    std::cout << std::boolalpha << "Contains even? " << hasEven
       << "\n";
}
```

Output:

```
Contains even? true
```

std::none_of

The algorithm `std::none_of` returns `true` if *no* elements satisfy the predicate. It is logically equivalent to negating `std::any_of`.

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> nums = {1, 3, 5};

    bool noneEven = std::none_of(nums.begin(), nums.end(),
                                 [](int x){ return x % 2 == 0; })
                                    ;

    std::cout << std::boolalpha << "No even numbers? " <<
       noneEven << "\n";
}
```

Output:

```
No even numbers? true
```

std::copy

The algorithm `std::copy` copies all elements in a given iterator range to a destination range.

```
std::copy(source.begin(), source.end(), destination.begin());
```

Key points:

- Performs element-by-element copying.
- Requires that the destination has enough space.
- Uses copy assignment for each element.

Example:

```
std::vector<int> a = {1, 2, 3, 4};
std::vector<int> b(4);

std::copy(a.begin(), a.end(), b.begin());
// b becomes {1, 2, 3, 4}
```

std::copy_if

The algorithm `std::copy_if` copies only those elements that satisfy a given predicate.

```
std::copy_if(first, last, dest, predicate);
```

Key points:

- Useful for filtering.
- Predicate must return `true` to copy an element.
- Destination must have enough capacity.

Example:

```
std::vector<int> data = {1, 2, 3, 4, 5, 6};
std::vector<int> evens;

std::copy_if(data.begin(), data.end(),
             std::back_inserter(evens),
             [](int x){ return x % 2 == 0; });
// evens = {2, 4, 6}
```

std::move

The algorithm `std::move` applies move semantics to each element in a range.

```
std::move(first, last, dest);
```

Key points:

- Transfers resources instead of copying them.
- After the move, the source elements remain in a valid but unspecified state.
- More efficient than `std::copy` for movable types (e.g., `std::string`, `std::vector`).

Example:

```
std::vector<std::string> src = {"one", "two", "three"};
std::vector<std::string> dst(3);

std::move(src.begin(), src.end(), dst.begin());
// src now contains unspecified values
```

`std::partition`

The algorithm `std::partition` rearranges the elements of a range so that all elements satisfying a predicate come before those that do not.

```
std::partition(first, last, predicate);
```

Key points:

- Does not preserve relative order (unstable).
- Returns an iterator to the first element of the second group.
- Often used to implement quicksort-like partitioning.

Example:

```
std::vector<int> nums = {9, 2, 7, 4, 1, 6};

auto it = std::partition(nums.begin(), nums.end(),
                         [](int x){ return x % 2 == 0; });

// Elements before 'it' are even; after are odd.
```

Summary of STL Containers and Algorithms

The C++ Standard Template Library (STL) provides a rich set of generic containers, iterators, and algorithms. These components enable developers to write efficient, reusable, and expressive code while relying on well-tested and optimized implementations.

STL Containers

STL containers store and organize collections of objects. They are implemented as templates, offering type safety and flexibility.

Sequence Containers

Sequence containers store elements in a linear order.

- `std::vector`: Dynamic array with fast random access and amortized constant-time insertion at the end.
- `std::deque`: Double-ended queue with efficient insertions and removals at both ends.
- `std::list`: Doubly linked list with fast insertion and deletion anywhere in the list.
- `std::forward_list`: Singly linked list, more memory-efficient than `std::list`.
- `std::array`: Fixed-size array with stack allocation.

Associative Containers

Associative containers store elements in sorted order and are typically implemented using self-balancing binary search trees.

- `std::set`: Stores unique keys in sorted order.
- `std::multiset`: Allows duplicate keys.
- `std::map`: Stores key–value pairs with unique keys.
- `std::multimap`: Stores key–value pairs with duplicate keys.

Operations such as insertion, deletion, and search typically have $\mathcal{O}(\log n)$ complexity.

Unordered (Hash-Based) Containers

Unordered containers store elements in hash tables, providing average constant-time access.

- `std::unordered_set`
- `std::unordered_multiset`
- `std::unordered_map`
- `std::unordered_multimap`

These are ideal for fast lookups where ordering is not important.

Container Adaptors

Container adaptors provide restricted interfaces built on top of existing containers.

- `std::stack`: LIFO structure.
- `std::queue`: FIFO structure.
- `std::priority_queue`: Stores elements ordered by priority.

STL Algorithms

STL algorithms operate on iterator ranges, making them flexible and container-independent. They support functional-style programming using lambdas and functors.

Non-Modifying Algorithms

These algorithms examine elements without modifying them.

- `std::for_each`
- `std::find`, `std::find_if`
- `std::count`, `std::count_if`
- `std::all_of`, `std::any_of`, `std::none_of`
- `std::equal`
- `std::search`

Modifying Algorithms

These algorithms modify the values or arrangement of elements.

- `std::copy`, `std::copy_if`
- `std::move`
- `std::fill`, `std::generate`
- `std::replace`, `std::replace_if`
- `std::transform`
- `std::partition`, `std::stable_partition`
- `std::remove`, `std::remove_if`

Sorting and Related Algorithms

These algorithms provide efficient sorting and ordering utilities.

- `std::sort`
- `std::stable_sort`
- `std::partial_sort`
- `std::nth_element`
- `std::is_sorted`

  Most sorting operations run in $\mathcal{O}(n \log n)$ time.

Numeric Algorithms

Provided by the `<numeric>` header:

- `std::accumulate`
- `std::inner_product`
- `std::iota`
- `std::partial_sum`
- `std::adjacent_difference`

Parallel Algorithms (C++17)

Using execution policies, some algorithms can run in parallel:

```
std::sort(std::execution::par, v.begin(), v.end());
```

Available execution policies:

- `std::execution::seq` – Sequential execution.
- `std::execution::par` – Parallel execution.
- `std::execution::par_unseq` – Parallel + vectorization.

Iterators

Iterators act as generalized pointers, enabling algorithms to work uniformly with various containers. Iterator categories include:

- Input and Output iterators
- Forward iterators
- Bidirectional iterators
- Random-access iterators

The iterator category determines which algorithms can be used.

Advantages of STL

- Encourages **generic programming** through templates.
- Provides **reusable**, well-tested data structures and algorithms.
- Guarantees **predictable performance**.
- Ensures **interoperability** via iterators.
- Improves **code clarity** and safety.
- Supports **extensibility** with custom comparators, iterators, and allocators.

Best Practices

- Prefer `std::vector` unless another container is more suitable.
- Use STL algorithms instead of manual loops when possible.
- Choose containers based on their complexity guarantees.
- Understand iterator invalidation rules for safe modification.
- Avoid premature optimization; rely on profiling.

Lab 9: Exploring C++ Standard Library Containers

1. Objective To gain hands-on experience with C++ Standard Library containers and understand their basic operations.
2. Task 1: Vector Operations
    - Create a C++ program that includes the necessary headers (nclude <iostream> and <vector>).
    - Declare a std::vector<int> called numbers and initialize it with the integers 1 to 5.
    - Use a for loop to print the contents of the numbers vector.
    - Add two more integers to the numbers vector using the push_back method.
    - Print the modified numbers vector.
3. Task 2: List Operations
    - Include the <list> header in your program.
    - Declare a std::list<char> called characters and initialize it with the characters 'A' to 'E'.
    - Use an iterator to print the contents of the characters list in reverse order (from 'E' to 'A').
    - Insert a new character 'F' at the beginning of the characters list.
    - Print the modified characters list.
4. Task 3: Map Operations
    - Include the <map> header.
    - Declare a std::map<std::string, int> called grades.
    - Add at least five key-value pairs representing student names and their corresponding grades.
    - Use an iterator to print the student names and grades from the grades map.
    - Remove one entry from the grades map.
    - Print the modified grades map.

5. Task 4: Set Operations
    - Include the <set> header.
    - Declare a std::set<double> called scores and initialize it with some random scores.
    - Print the scores in ascending order using a for loop.
    - Remove a specific score from the scores set.
    - Print the modified scores set.
6. Task 5: Container Algorithms
    - Include the <algorithm> header.
    - Use the std::find algorithm to search for a specific value within one of your containers.
    - Use the std::count algorithm to count the occurrences of an element in another container.

Practice Questions

1. Conceptual Questions

1. What does the algorithm `std::sort` require from its input range? Explain the iterator and comparator requirements.
2. Explain the difference between `std::find` and `std::find_if`. When would you choose one over the other?
3. Explain the difference between `std::remove` and `erase`. Why does `std::remove` not actually remove elements from a container?
4. What is the purpose of `std::transform`? Give two possible use cases.
5. Describe what `std::accumulate` does. Which header must be included to use it? How is it different from a manual loop?
6. What is the difference between `std::all_of`, `std::any_of`, and `std::none_of`? Provide examples where each is useful.
7. Why are some algorithms such as `std::sort` and `std::binary_search` restricted to random-access iterators?
8. Explain the concept of **stable** algorithms. Give an example of a stable algorithm and one that is not stable.

2. Code Tracing Questions

9. What is the output of the following code?

```
std::vector<int> v = {5, 3, 9, 1};
std::sort(v.begin(), v.end());
auto it = std::lower_bound(v.begin(), v.end(), 4);
std::cout << (it - v.begin());
```

10. Predict the output:

```
std::vector<int> v = {1, 2, 3, 4, 5};
std::reverse(v.begin(), v.end());
std::rotate(v.begin(), v.begin() + 2, v.end());
for (int n : v) std::cout << n << " ";
```

11. What will be printed?

```
std::vector<int> v = {2, 4, 6, 8, 9};
bool result = std::all_of(v.begin(), v.end(),
    [](int x){ return x % 2 == 0; });

std::cout << std::boolalpha << result;
```

3. Write-the-Code Questions

12. Write a function that uses `std::count_if` to count how many numbers in a vector are divisible by 3.

13. Write a one-line statement using `std::accumulate` to compute the product of all elements in a vector.

14. Given a vector of strings, use `std::sort` to sort them by length (shortest first).

15. Use `std::transform` to convert all characters in a string to uppercase.

16. Write a program that uses `std::remove_if` to remove all negative values from a vector.

4. Problem-Solving Questions

17. You are given a vector of integers. Write code using standard algorithms to:

    - Remove duplicates
    - Sort the remaining values in descending order
    - Print them

18. Given two vectors of equal size, compute their element-wise sum using an STL algorithm. Which algorithm is most appropriate and why?
19. Explain how to check if two containers contain the same elements in the same order using a single STL algorithm.
20. Write a program that:
    - Reads a list of words
    - Converts them to lowercase
    - Removes duplicates
    - Sorts them alphabetically

    Use only `std::algorithm` operations.

5. Advanced Questions

21. Explain the difference between:
    - `std::find_if`
    - `std::find_if_not`
22. Compare:
    - `std::copy`
    - `std::copy_if`
    - `std::move`
23. When would you prefer `std::partition` over `std::stable_partition`?
24. What are the performance guarantees of `std::sort`? Why is it usually faster than many other sorting methods?
25. How does `std::for_each` differ from a regular range-based `for` loop? When would you choose to use it?

Lecture 9: Error and Exception Handling in C++

Introduction

Error and exception handling is a cornerstone of robust and reliable software development in C++. Regardless of how carefully software is designed, it will inevitably face situations such as invalid input, missing resources, or unexpected logical conditions. How these situations are handled determines the stability, maintainability, and user experience of the program.

C++ provides two main mechanisms for error handling:

1. Traditional error codes and return values.
2. Exception handling through `try`, `throw`, and `catch`.

While return codes are efficient and suitable for low-level systems, exception handling provides a more structured and object-oriented approach, allowing separation between regular and exceptional logic.

This chapter provides a comprehensive overview of error and exception handling in C++, discussing traditional techniques, exception handling mechanisms, assertions, exception safety, best practices, and performance considerations.

Understanding Errors in C++

An *error* is any event that prevents a program from executing as intended. Errors can be categorized as:

- **Compile-time errors:** Detected by the compiler, such as syntax or type errors.
- **Link-time errors:** Occur due to missing symbols or unresolved external references.
- **Run-time errors:** Arise during execution, such as invalid memory access or division by zero.

This chapter focuses on run-time errors, since they cannot be predicted at compile time and require explicit handling.

Traditional Error Handling in C

Before the advent of exceptions, C programmers handled errors using return values, global flags like `errno`, or manual condition checks.

```
#include <iostream>
#include <cstdio>
```

```
int divide(int a, int b, int* result) {
    if (b == 0) return -1; // error code for division by zero
    *result = a / b;
    return 0; // success
}

int main() {
    int res;
    if (divide(10, 0, &res) != 0) {
        std::cerr << "Error: Division by zero.\n";
        return 1;
    }
    std::cout << "Result = " << res << "\n";
}
```

This approach is simple but can quickly lead to complex and unreadable code as the program size grows.

Limitations of Traditional Error Handling

Although functional, the traditional C-style approach has several limitations:

1. **Code clutter:** Every function call must be followed by an error check.
2. **Error propagation:** Errors must be manually passed up the call chain.
3. **Omission risk:** Developers might forget to check for errors.
4. **Ambiguity:** Return codes may not clearly indicate the cause of failure.
5. **Difficult resource cleanup:** Managing multiple acquired resources becomes complex.

To overcome these limitations, C++ introduced exception handling, offering an elegant and object-oriented way to separate normal and error-handling code.

The Exception Handling Model in C++

C++ exceptions allow errors to be represented as objects that can be thrown and caught. The core keywords are:

- `try`: Marks a block of code that might throw an exception.
- `throw`: Signals that an exceptional event has occurred.
- `catch`: Handles the exception based on its type.

```
#include <iostream>
#include <stdexcept>

int divide(int a, int b) {
```

```
    if (b == 0)
        throw std::runtime_error("Division by zero");
    return a / b;
}

int main() {
    try {
        std::cout << divide(10, 0) << "\n";
    } catch (const std::runtime_error& e) {
        std::cerr << "Error: " << e.what() << "\n";
    }
}
```

When an exception is thrown, normal execution stops, and the control transfers to the nearest matching `catch` block.

Mechanics of Exception Handling

When an exception is thrown:

1. The runtime searches for the nearest matching `try` block.
2. If a matching `catch` block exists, control is transferred there.
3. If none exists, the stack is unwound — local objects are destroyed.
4. If no handler is found in the call hierarchy, `std::terminate()` is called.

This process, known as *stack unwinding*, ensures safe resource cleanup.

The `throw` Statement

The `throw` statement signals an exceptional condition. It can throw objects of any type, though it is best practice to throw types derived from `std::exception`.

```
throw 10;                          // integer exception
throw std::string("Error");        // string exception
throw std::runtime_error("Bad"); // preferred approach
```

Using `try` and `catch`

A `try` block must be followed by one or more `catch` handlers.

```
try {
    // risky operation
}
catch (const std::out_of_range& e) {
    std::cerr << "Out of range: " << e.what() << "\n";
}
catch (const std::exception& e) {
    std::cerr << "Standard exception: " << e.what() << "\n";
```

```
}
catch (...) {
    std::cerr << "Unknown exception\n";
}
```

The ellipsis (`...`) acts as a catch-all for any exception not matched earlier.

Exception Class Hierarchy

The C++ Standard Library defines a rich hierarchy of exceptions derived from `std::exception`. Common classes include:

- `std::logic_error`: For programming logic errors.
- `std::runtime_error`: For errors detected only during execution.
- `std::bad_alloc`, `std::bad_cast`, `std::out_of_range`.

These predefined classes encourage consistent error handling and descriptive reporting.

User-Defined Exception Classes

Developers can define custom exception types for domain-specific errors.

```
#include <exception>
#include <string>

class FileNotFound : public std::exception {
    std::string message;
public:
    FileNotFound(const std::string& filename)
        : message("File not found: " + filename) {}
    const char* what() const noexcept override {
        return message.c_str();
    }
};
```

This approach ensures semantic clarity and precise error identification.

Stack Unwinding and RAII

When an exception is thrown, all automatic variables in scope are destroyed. This process, known as *stack unwinding*, ensures resources are released even when execution flow changes abruptly.

The **Resource Acquisition Is Initialization (RAII)** principle binds resource lifetime to object lifetime, simplifying cleanup.

```
#include <fstream>
#include <stdexcept>
```

```cpp
void processFile(const std::string& path) {
    std::ifstream file(path);
    if (!file.is_open())
        throw std::runtime_error("Cannot open file");
    // file is automatically closed at scope end
}
```

Catching by Value, Reference, or Pointer

- **By value:** May cause slicing; avoid if polymorphism is needed.
- **By reference:** Preserves type hierarchy and avoids copies.
- **By pointer:** Rarely used; requires manual memory management.

Best practice: `catch (const std::exception& e)`.

Exception Safety

Exception safety guarantees describe how code behaves when exceptions occur:

- **No-throw guarantee:** Function never throws.
- **Strong guarantee:** State remains unchanged if an exception occurs.
- **Basic guarantee:** State remains valid but may change.
- **No guarantee:** No assumptions about state validity.

Well-designed code should at least provide the basic guarantee.

The `noexcept` Specifier

The `noexcept` keyword indicates that a function does not throw exceptions.

```cpp
void safeFunction() noexcept {
    // guaranteed not to throw
}
```

Using `noexcept` improves performance and enables compiler optimizations such as move semantics.

Rethrowing Exceptions

A caught exception can be rethrown to higher levels:

```cpp
catch (const std::exception& e) {
    log(e.what());
    throw; // rethrow same exception
}
```

Rethrowing allows separation between logging and recovery responsibilities.

Exceptions in Constructors and Destructors

If a constructor throws, the destructor for that object is not called, but already constructed members are destroyed. This behavior ensures no resource leaks.

Destructors should **never throw** exceptions. If cleanup fails, log the issue instead.

```
~FileHandler() noexcept {
    try { close(); }
    catch (...) { /* log and suppress */ }
}
```

Assertions in C++

Assertions are a defensive programming mechanism for detecting programmer errors rather than runtime failures. They are typically used to verify assumptions made by the code during development.

The C++ standard library provides the `assert` macro in `<cassert>`:

```
#include <cassert>

int factorial(int n) {
    assert(n >= 0); // precondition check
    int result = 1;
    for (int i = 1; i <= n; ++i)
        result *= i;
    return result;
}
```

If the condition inside `assert()` evaluates to false, the program prints an error message and aborts.

Assertions are typically disabled in production builds by defining the `NDEBUG` macro. For example:

```
#define NDEBUG
#include <cassert>
```

When to Use Assertions

- To check for conditions that should never occur.
- To validate preconditions and postconditions during development.
- To document assumptions made in code logic.

Assertions differ from exceptions:

- Assertions detect programming errors.
- Exceptions handle recoverable runtime errors.

Thus, assertions complement exceptions rather than replace them.

Standard Library and Exceptions

Many C++ library functions throw exceptions. Examples include:

- `std::vector::at()` throws `std::out_of_range`.
- `new` throws `std::bad_alloc` on memory failure.
- `std::stoi()` throws `std::invalid_argument`.

Understanding these behaviors helps prevent unexpected crashes.

Performance Considerations

Exception handling introduces minor overhead in terms of binary size and runtime metadata, but most compilers optimize for the case where no exceptions occur. Therefore, the performance impact is negligible unless exceptions are frequently thrown.

Best practices:

- Use exceptions only for exceptional situations.
- Avoid throwing exceptions in inner loops.
- Mark non-throwing functions as `noexcept`.

Alternatives to Exceptions

In some environments, such as embedded systems, exceptions are disabled. Alternatives include:

- **Error codes and `std::expected` (C++23).**
- **Assertions.**
- `std::optional` to represent optional results.

Example with `std::expected`:

```
#include <expected>

std::expected<int, std::string> safe_divide(int a, int b) {
    if (b == 0)
        return std::unexpected("Division by zero");
    return a / b;
}
```

Best Practices

- Throw by value, catch by reference.
- Use standard exceptions when possible.
- Do not throw exceptions from destructors.
- Guarantee resource cleanup using RAII.
- Apply `noexcept` to non-throwing functions.
- Avoid using exceptions for normal control flow.

Summary

Exception handling in C++ provides a structured, object-oriented approach to dealing with run-time errors while ensuring proper cleanup and recoverability. Assertions serve as an additional safeguard to verify program logic during development.

Key points:

- Use exceptions for exceptional events, not flow control.
- Assertions detect logical errors early.
- Always manage resources via RAII.
- Use `noexcept` for guaranteed safe operations.
- Prefer clarity, maintainability, and predictability in error-handling design.

Together, assertions, exceptions, and modern C++ constructs form a powerful foundation for building safe and reliable software systems.

Practice Lab: Exception Handling in C++

This lab introduces the principles and practical applications of **exception handling in C++**. You will learn to detect, throw, and catch runtime errors using the `try`, `throw`, and `catch` constructs. You will also practice working with standard exception classes, creating custom exceptions, and writing exception-safe code.

Objectives

After completing this lab, you should be able to:

- Explain the purpose of exception handling.
- Apply `try`, `throw`, and `catch` to manage runtime errors.
- Use standard exceptions such as `std::runtime_error` and `std::invalid_argument`.
- Define and use custom exception classes.
- Demonstrate stack unwinding and RAII (Resource Acquisition Is Initialization).
- Apply assertions to validate assumptions during program execution.

Prerequisites

- Basic understanding of C++ syntax, control structures, and input/output.
- A working C++ compiler (e.g., g++, `clang++`, or MSVC).

Required Tools

- Operating System: Linux, macOS, or Windows.
- Compiler: Any C++17 or later compiler.
- Text Editor or IDE: Visual Studio Code, CLion, or any preferred environment.

Background Theory

In C++, exceptions handle unexpected or erroneous runtime events. The main keywords are:

- **throw**: Signals that an error or exceptional condition has occurred.
- **try**: Wraps code that might generate exceptions.
- **catch**: Defines how to handle an exception once it is thrown.

Example:

```
try {
    throw std::runtime_error("Example error");
}
catch (const std::exception& e) {
    std::cerr << "Caught: " << e.what() << '\n';
}
```

Exercises

Exercise 1: Basic Exception Handling

Goal: Throw and catch a simple exception.

```
#include <iostream>
#include <stdexcept>

int divide(int a, int b) {
    if (b == 0)
        throw std::runtime_error("Division by zero");
    return a / b;
}

int main() {
    try {
```

```
        int x, y;
        std::cin >> x >> y;
        std::cout << divide(x, y) << '\n';
    }
    catch (const std::runtime_error& e) {
        std::cerr << "Error: " << e.what() << '\n';
    }
}
```

Try dividing by zero and observe the output.

Exercise 2: Multiple Catch Blocks

Goal: Handle different types of exceptions.

```
#include <iostream>
#include <stdexcept>

void check(int value) {
    if (value < 0)
        throw std::invalid_argument("Negative not allowed");
    if (value == 0)
        throw std::runtime_error("Zero not allowed");
}

int main() {
    try {
        int n;
        std::cin >> n;
        check(n);
        std::cout << "OK\n";
    }
    catch (const std::invalid_argument& e) {
        std::cerr << e.what() << '\n';
    }
    catch (const std::runtime_error& e) {
        std::cerr << e.what() << '\n';
    }
}
```

Exercise 3: Custom Exception Class

Goal: Define and throw a user-defined exception.

```
#include <iostream>
#include <exception>
#include <string>

class FileNotFound : public std::exception {
```

```
    std::string msg;
public:
    explicit FileNotFound(const std::string& file)
        : msg("File not found: " + file) {}
    const char* what() const noexcept override { return msg.c_str
       (); }
};

void openFile(const std::string& name) {
    if (name != "data.txt")
        throw FileNotFound(name);
}

int main() {
    try {
        openFile("test.txt");
    }
    catch (const FileNotFound& e) {
        std::cerr << e.what() << '\n';
    }
}
```

Exercise 4: Stack Unwinding

Goal: Observe object destruction when exceptions are thrown.

```
#include <iostream>
#include <stdexcept>

class Demo {
    std::string name;
public:
    Demo(std::string n) : name(n) { std::cout << "Construct " <<
       name << '\n'; }
    ~Demo() { std::cout << "Destruct " << name << '\n'; }
};

void test() {
    Demo a("A"), b("B");
    throw std::runtime_error("Error occurred");
}

int main() {
    try {
        test();
    }
    catch (const std::exception& e) {
        std::cerr << e.what() << '\n';
```

```
    }
}
```

Observe how destructors are called automatically as the stack unwinds.

Exercise 5: RAII and Resource Safety

Goal: Demonstrate automatic cleanup of resources.

```
#include <iostream>
#include <fstream>
#include <stdexcept>

void readFile(const std::string& file) {
    std::ifstream f(file);
    if (!f)
        throw std::runtime_error("Cannot open " + file);
    std::string line;
    while (std::getline(f, line))
        std::cout << line << '\n';
}

int main() {
    try {
        readFile("missing.txt");
    }
    catch (const std::exception& e) {
        std::cerr << e.what() << '\n';
    }
}
```

Even if an exception is thrown, the file resource is safely closed due to RAII.

Exercise 6: Assertions

Goal: Use assertions to validate program assumptions.

```
#include <cassert>
#include <iostream>

int factorial(int n) {
    assert(n >= 0 && "Negative input not allowed");
    return (n <= 1) ? 1 : n * factorial(n - 1);
}

int main() {
    std::cout << factorial(5) << '\n';
    std::cout << factorial(-2) << '\n';  // triggers assertion
}
```

Assertions are used to detect logic errors during development, not for runtime error handling.

Discussion Questions

1. Why use exceptions instead of return codes?
2. What happens if an exception is not caught?
3. How does stack unwinding prevent resource leaks?
4. When should assertions be preferred over exceptions?

Challenge Task

Design a `BankAccount` class that:

- Throws an `InsufficientFunds` exception when withdrawing too much money.
- Uses RAII to manage any resource safely.
- Demonstrates exception propagation through multiple function calls.

Summary

In this lab, you practiced:

- Using `try`, `throw`, and `catch`.
- Handling multiple types of exceptions.
- Defining and using custom exception classes.
- Observing stack unwinding and RAII in action.
- Applying assertions for program correctness.

Exception handling helps create reliable and maintainable C++ programs that handle errors gracefully without resource leaks.

Questions

Practice Questions on Exception Handling in C++

This section provides a set of theoretical and practical questions to test your understanding of exception handling concepts in C++. They include short-answer, code analysis, and problem-solving questions.

Part A: Conceptual Questions

1. Define the term **exception** in the context of C++ programming.
2. What are the three main keywords used in C++ exception handling? Briefly explain their roles.
3. Differentiate between `try` blocks and `catch` blocks.
4. What happens when an exception is thrown but not caught by any `catch` block?
5. Explain the concept of **stack unwinding**. Why is it important?
6. Describe the difference between compile-time errors, run-time errors, and exceptions.
7. What is the purpose of the `std::exception` class in C++?
8. Why is it recommended to catch exceptions by reference rather than by value?
9. What is the difference between standard exceptions and user-defined exceptions?
10. Can destructors throw exceptions safely? Why or why not?

Part B: Code Analysis Questions

For each of the following code snippets, determine the output or explain what happens when the code is executed.

1.
```
#include <iostream>
#include <stdexcept>
int main() {
    try {
        throw std::runtime_error("Error occurred");
        std::cout << "After throw\n";
    }
    catch (const std::runtime_error& e) {
        std::cout << "Caught: " << e.what() << '\n';
    }
    std::cout << "Program continues\n";
}
```

2.
```
#include <iostream>
void func() {
    throw 10;
}
int main() {
    try {
        func();
    }
```

```
        catch (int x) {
            std::cout << "Caught int: " << x << '\n';
        }
        catch (...) {
            std::cout << "Caught unknown exception\n";
        }
    }
```

3.
```
    #include <iostream>
    class A {
    public:
        A() { std::cout << "Constructing A\n"; }
        ~A() { std::cout << "Destructing A\n"; }
    };
    void test() {
        A obj;
        throw std::runtime_error("Failure");
    }
    int main() {
        try {
            test();
        } catch (...) {
            std::cout << "Exception caught\n";
        }
    }
```

Discuss the order of object destruction.

Part C: Programming Tasks

1. Write a program that reads two integers and performs division. Throw and handle an exception if the denominator is zero.

2. Implement a function `checkAge(int age)` that throws an `invalid_argument` exception if the input age is less than 18. Handle the exception in `main()` and print an appropriate message.

3. Define a custom exception class `NegativeBalance` derived from `std::exception`. Write a program that simulates a simple bank withdrawal operation and throws this exception if the balance becomes negative.

4. Write a program that demonstrates **rethrowing** exceptions using `throw;` inside a `catch` block.

5. Create a function that opens a text file for reading. If the file cannot be opened, throw a `std::runtime_error`. Use RAII to ensure the file stream is properly closed.

6. Write a short program that illustrates the use of `assert()` to check for invalid array indices.

7. Implement nested `try-catch` blocks where an exception thrown in an inner `try` block is caught and partially handled, then rethrown for further handling in the outer block.

Part D: Discussion and Reflection

1. In what scenarios should exceptions be preferred over error return codes?
2. What are the best practices for defining custom exception classes?
3. How does exception handling interact with resource management in C++?
4. Why is RAII a critical concept for writing exception-safe C++ code?
5. How do assertions complement exception handling in software development?

Part E: Challenge Problems

1. Create a class `SafeArray` that represents an integer array with boundary checking. If an invalid index is accessed, the class should throw an exception of type `std::out_of_range`.
2. Design a small library system that throws different exceptions such as `BookNotFound`, `InvalidUser`, and `OverdueBook`. Demonstrate how these exceptions can be caught and handled differently.
3. Implement a simple calculator program that uses a single `try` block and multiple `catch` blocks to handle input validation, division by zero, and overflow errors.

Lecture 10: Files and Streams in C++

In computer programming, dealing with data is a fundamental task. Data can be read from various sources, processed, and then written back to storage. C++ provides a versatile and powerful mechanism for working with external files through the concept of files and streams. Streams provide a unified interface for reading and writing data, whether it's coming from a file, the console, or any other source.

This chapter explores the basics of files and streams in C++, including file input and output operations, stream classes, and error handling. By the end of this chapter, you will have a solid understanding of how to work with files and streams in C++.

Understanding Streams

What Are Streams?

In C++, a stream is an abstraction that represents a sequence of characters or bytes. Streams are a flexible way to handle input and output operations. They can be associated with various devices, such as the standard input (keyboard), standard output (screen), and external files.

Streams can be categorized into two main types:

- Input Streams: These are used for reading data from a source, such as a file or the keyboard. The standard input stream is represented by cin.
- Output Streams: These are used for writing data to a destination, such as a file or the screen. The standard output stream is represented by cout.

C++ provides a set of predefined stream objects for common input and output operations. These include cin (standard input), cout (standard output), cerr (standard error output), and clog (standard logging output).

Stream Classes

Streams in C++ are implemented as classes within the Standard Library. The primary stream classes are:

- istream: This class is used for input operations and includes functions for reading data.
- ostream: This class is used for output operations and includes functions for writing data.
- ifstream: This class is derived from istream and is used specifically for file input operations.

- ofstream: This class is derived from ostream and is used specifically for file output operations.
- fstream: This class is derived from both ifstream and ofstream and is used for both input and output operations on files.

These stream classes provide member functions that allow you to perform various operations like reading and writing data, positioning the file pointer, and handling errors.

Stream States and Flags

Streams can be in various states depending on their current condition. Common stream states include:

- good(): Indicates that the stream is in a good state and ready for further operations.
- eof(): Indicates that the end of the stream has been reached.
- fail(): Indicates that a failure occurred during the last operation.
- bad(): Indicates that the stream is in an unrecoverable bad state.

Stream states are often checked using stream flags. The clear() member function is used to clear error flags, and the setstate() member function is used to set specific flags.

File Input and Output

Opening and Closing Files

Before you can read from or write to a file, you must open it. To open a file for reading or writing, you use an ifstream (for input) or ofstream (for output) object, respectively.

```
#include <fstream>
using namespace std;

int main() {
    // Open a file for reading
    ifstream inputFile("input.txt");

    // Open a file for writing (creates a new file if it doesn't
       exist)
    ofstream outputFile("output.txt");

    // Perform operations on the files...

    // Close the files
    inputFile.close();
    outputFile.close();

    return 0;
}
```

Always remember to close files when you're done with them to release system resources.

Reading from Files

To read data from a file, use the » operator with an `ifstream` object.

```
 #include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream inputFile("input.txt");
    int number;

    if (inputFile.is_open()) {
        while (inputFile >> number) {
            cout << "Read: " << number << endl;
        }
        inputFile.close();
    } else {
        cerr << "Failed to open the file." << endl;
    }

    return 0;
}
```

Writing to Files

To write data to a file, use the « operator with an ofstream object.

```
 #include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outputFile("output.txt");

    if (outputFile.is_open()) {
        outputFile << "Hello, File!" << endl;
        outputFile.close();
    } else {
        cerr << "Failed to open the file." << endl;
    }

    return 0;
}
```

Error Handling

Handling errors when working with files is crucial to ensure the reliability of your programs. C++ provides mechanisms to check and handle errors during file operations.

Checking for Errors

You can use the fail() member function to check if a stream operation has failed.

```
 ifstream inputFile("input.txt");
if (inputFile.fail()) {
    cerr << "Failed to read from the file." << endl;
}
```

Exception Handling

C++ also supports exception handling for more robust error management. You can use try, catch, and throw statements to handle exceptions that occur during file operations.

```
 try {
    ifstream inputFile("input.txt");
    if (!inputFile) {
        throw runtime_error("Failed to read from the file.");
    }
    // Read from the file...
} catch (const runtime_error& e) {
    cerr << "Error: " << e.what() << endl;
}
```

Working with Binary files

Binary files are files that contain non-text data, such as images, audio, or program executables. These files are not human-readable and must be handled differently than text files. In C++, you can work with binary files using file streams, specifically ifstream and ofstream, just like you do with text files. However, you need to be aware of the differences in handling binary data.

Here's a guide on how to work with binary files using C++ streams:

Writing Binary Data to a File

To write binary data to a file, you'll typically use an ofstream object in binary mode. Binary mode is essential because it prevents any character encoding translations that might occur in text mode.

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main() {
    // Open a binary file for writing
    ofstream outputFile("binary_data.dat", ios::out | ios::binary
       );

    if (outputFile.is_open()) {
        // Data to be written
        int integerData = 42;
        double doubleData = 3.14159;

        // Write binary data
        outputFile.write(reinterpret_cast<char*>(&integerData),
           sizeof(int));
        outputFile.write(reinterpret_cast<char*>(&doubleData),
           sizeof(double));

        // Close the file
        outputFile.close();
        cout << "Data written to binary file." << endl;
    } else {
        cerr << "Failed to open the file." << endl;
    }

    return 0;
}
```

In this example:

1. We open a file named "binary_data.dat" in binary mode using ios::out | ios::binary as the mode flags.
2. We prepare some binary data (an integer and a double) that we want to write to the file.
3. We use the write() method to write the binary data. Note the use of reinterpret_cast to convert the data pointers to char* pointers, allowing us to write raw bytes to the file.
4. Finally, we close the file.

Reading Binary Data from a File

Reading binary data from a file involves using an ifstream object in binary mode, similar to writing.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
```

```
    // Open the binary file for reading
    ifstream inputFile("binary_data.dat", ios::in | ios::binary);

    if (inputFile.is_open()) {
        // Data to store the read values
        int readInteger;
        double readDouble;

        // Read binary data
        inputFile.read(reinterpret_cast<char*>(&readInteger),
           sizeof(int));
        inputFile.read(reinterpret_cast<char*>(&readDouble),
           sizeof(double));

        if (!inputFile.fail()) {
            cout << "Read Integer: " << readInteger << endl;
            cout << "Read Double: " << readDouble << endl;
        } else {
            cerr << "Failed to read data from the file." << endl;
        }

        // Close the file
        inputFile.close();
    } else {
        cerr << "Failed to open the file." << endl;
    }

    return 0;
}
```

In this example:

1. We open the same file "binary_data.dat" for reading, using ios::in | ios::binary as the mode flags.
2. We declare variables to store the data we will read from the file.
3. We use the read() method to read the binary data, again using reinterpret_cast to convert the data pointers to char* pointers.
4. We check for errors using the fail() method. If the read was successful, we print the read values.
5. Finally, we close the file.

Binary File Considerations

- When working with binary files, it's crucial to specify the correct data types and sizes when reading and writing. The use of sizeof() ensures that the correct number of bytes is processed.

- Endianness (byte order) can be an issue when reading/writing binary data across different systems. Be cautious when sharing binary files between systems with different endianness.
- Binary files may not be portable between different compilers or platforms due to differences in data representation and padding.
- Binary files are not human-readable, so ensure proper error handling and version control when working with them.

Working with binary files in C++ allows you to handle non-text data efficiently. However, it requires careful management of data types, sizes, and endianness to ensure data integrity when reading and writing binary files.

Lab 10: C++ Streams: Text and Binary File Operations

- Lab Objectives:
  1. Understand the basics of C++ streams.
  2. Learn how to work with text files.
  3. Learn how to work with binary files.
  4. Practice error handling when working with files.
- Text File Operations Create a program that opens a text file named "text_file.txt" for writing and writes the following lines to the file:

```
Hello, World!
This is a text file.
C++ streams are awesome!
```

  Ensure proper error handling for file opening. Create another program that opens "text_file.txt" for reading and displays its contents on the console.

- Binary File Operations
  1. Create a program that opens a binary file named "binary_data.dat" for writing. Write the following data to the file:n integer with the value 42. A double with the value 3.14159.
  2. Create another program that opens "binary_data.dat" for reading. Read the data and display it on the console
- Error Handling
- Modify the programs in Task 1 and Task 2 to include proper error handling for file opening, reading, and writing. Ensure that error messages are displayed if any errors occur.
- User Interaction

1. Create a program that allows the user to enter text interactively and write it to a text file named "user_text.txt." Allow the user to enter multiple lines of text, and write them to the file. Use a loop to continue taking input until the user decides to stop.
2. Create another program that reads the contents of "user_text.txt" and displays them on the console.

- Binary File Extension

1. Extend the binary file program from Task 2 to write and read additional data types such as float, char, and bool.
2. Ensure proper error handling for all file operations, and display the read data on the console.

- Lab Completion:

1. Test each program thoroughly to ensure proper functionality and error handling.
2. Verify that text and binary files are created, written to, and read from correctly.
3. Ensure that all error messages are informative and helpful.
4. Demonstrate understanding of both text and binary file operations and error handling.

Practice Questions

1. What is the purpose of C++ streams, and why are they essential in programming?
2. What is the difference between text mode and binary mode when working with C++ streams?
3. How do you open a text file for reading in C++ using streams? Provide an example.
4. How can you check if a file stream (ifstream or ofstream) was opened successfully? What happens if the stream fails to open?
5. Describe the process of writing text to a file in C++ using an ofstream object. Provide an example.
6. How do you read text from a file in C++ using an ifstream object? Provide an example.
7. Explain the concept of end-of-file (EOF) when reading from a file. How do you detect EOF in C++?
8. What are the common stream states that can be checked using member functions like good(), eof(), fail(), and bad()? How are they used?

9. How do you open a binary file for writing in C++ using streams? Provide an example.
10. What is the purpose of write() and read() methods when working with binary files in C++? Provide examples of using these methods.
11. Explain the importance of proper error handling when working with file streams in C++. What mechanisms can be used for error handling?
12. Describe how you can use C++ streams to read user input from the console and write it to a text file. Provide an example.
13. What precautions should be taken when sharing binary files between systems with different endianness (byte order)?
14. How do you close a file stream in C++? Why is it important to close files properly?
15. What are the advantages of using C++ streams over traditional C-style file handling functions (e.g., fopen, fread, fwrite)?

Lecture 11: Writing Big Programs

Header and Implementation Files

Header and implementation files are essential components in C++ programming for organizing and structuring your code. They help separate the declaration (interface) of classes, functions, and variables from their actual implementation, promoting modularity, reusability, and maintainability of code. Let's explore header and implementation files in C++:

Header Files (.h or .hpp)

Header files contain the declarations (signatures) of classes, functions, and variables that are meant to be used by other parts of your program, including other source files. Here are some key points about header files:

- Purpose: Header files serve as the interface to your code. They provide a way for other parts of your program to access and use the functionality you've defined.
- Contents: Typically, header files include class declarations, function prototypes, constant definitions, and type aliases. They should not contain the actual implementation code.
- Extensions: Header files often have extensions like .h or .hpp. The choice of extension is not standardized, so you can use what you prefer.
- Include Guards: To prevent multiple inclusions of the same header file, you should use include guards. These are preprocessor directives that ensure the header is included only once in a source file. For example:

```
#ifndef MYHEADER_H
#define MYHEADER_H

// ... Contents of the header file ...

#endif // MYHEADER_H
```

Implementation Files (.cpp)

Implementation files contain the actual code that defines the functionality declared in header files. Here are some key points about implementation files:

- Purpose: Implementation files provide the details of how functions and classes work. They contain the executable code that implements the behavior declared in the header files.
- Contents: Implementation files include the function definitions, method implementations, variable initializations, and any other code that isn't part of the interface.
- Extensions: Implementation files often use the .cpp extension, although it's not strictly required.

Example

Let's say you have a class called Calculator that you want to use in your program. You would typically organize your code as follows:

Calculator.h (Header File):

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

class Calculator {
public:
    Calculator(); // Constructor
    int add(int a, int b);
    int subtract(int a, int b);
    int multiply(int a, int b);
    double divide(int a, int b);
};

#endif // CALCULATOR_H
```

Calculator.cpp (Implementation File):

```
#include "Calculator.h"

Calculator::Calculator() {
    // Constructor implementation
}

int Calculator::add(int a, int b) {
    return a + b;
}

int Calculator::subtract(int a, int b) {
    return a - b;
}

int Calculator::multiply(int a, int b) {
    return a * b;
}
```

```
double Calculator::divide(int a, int b) {
    if (b == 0) {
        throw "Division by zero!";
    }
    return static_cast<double>(a) / b;
}
```

Main.cpp (Source File):

```
#include <iostream>
#include "Calculator.h"

int main() {
    Calculator calc;
    int result = calc.add(5, 3);
    std::cout << "5 + 3 = " << result << std::endl;

    return 0;
}
```

In this example:

- Calculator.h defines the class Calculator and its member functions' declarations.
- Calculator.cpp contains the implementations of the Calculator class member functions.
- Main.cpp uses the Calculator class by including its header file and creating an instance of the class.

Forward Declarations in C++

In large-scale C++ projects, managing dependencies between classes and modules efficiently is crucial for reducing compilation time and improving maintainability. One of the most effective techniques to achieve this is the use of **forward declarations**. A forward declaration allows the programmer to declare the existence of a type, function, or variable before it is fully defined. This technique is often used in header files to avoid unnecessary inclusion of other headers and to break circular dependencies.

Concept of Forward Declaration

A **forward declaration** is a declaration of an identifier (such as a class, struct, function, or variable) that tells the compiler that this entity exists somewhere in the program, but its full definition will appear later. It allows the compiler to recognize the name and type of the symbol without needing to know its complete implementation at that point.

For example, the following line is a forward declaration of a class named `Student`:

```
class Student;
```

This declaration simply informs the compiler that a class named `Student` will be defined later, enabling other code to reference pointers or references to `Student` before its definition is seen.

Why Use Forward Declarations?

Forward declarations are primarily used for three reasons:

1. **To reduce compilation dependencies:** Including header files in multiple source files can cause unnecessary recompilation when one header changes. Forward declarations minimize the number of included headers, leading to faster builds.
2. **To resolve circular dependencies:** When two classes depend on each other, using forward declarations can help break the dependency cycle, allowing each class to be compiled independently.
3. **To improve encapsulation:** Forward declarations limit visibility of implementation details, keeping interfaces clean and reducing coupling between components.

Example: Using Forward Declarations Between Classes

Consider the case of two classes, `A` and `B`, where each class holds a pointer to the other. Without forward declarations, this situation would lead to circular inclusion errors.

```
// File: A.h
#ifndef A_H
#define A_H

class B; // Forward declaration

class A {
public:
    void setB(B* ptr);
private:
    B* b_ptr;
};

#endif
```

```
// File: B.h
#ifndef B_H
#define B_H

class A; // Forward declaration

class B {
public:
    void setA(A* ptr);
private:
    A* a_ptr;
};

#endif
```

```
// File: A.cpp
#include "A.h"
#include "B.h"

void A::setB(B* ptr) {
    b_ptr = ptr;
}
\end{verbatim}

\begin{lstlisting}
// File: B.cpp
#include "B.h"
#include "A.h"

void B::setA(A* ptr) {
    a_ptr = ptr;
}
```

In this example, each class only needs a pointer to the other, not the full definition. The actual definitions are included in the source files (`.cpp`), not the headers, preventing circular inclusion.

Forward Declaration of Functions

Forward declarations are also common for functions. For example:

```
#include <iostream>
using namespace std;

void greet(); // Forward declaration

int main() {
    greet();
    return 0;
}

void greet() {
    cout << "Hello, World!" << endl;
}
```

Here, the function `greet()` is declared before `main()`, allowing it to be called before its full definition appears later in the file.

Forward Declarations of Structs and Enums

In addition to classes and functions, C++ allows forward declarations for `structs`, `unions`, and `enums`. For example:

```
struct Node;   // Forward declaration
```

```
enum Color;     // Forward declaration (C++11 and later)
```

However, note that forward-declaring an `enum class` is not allowed before C++11, and the compiler must know its underlying type.

Limitations of Forward Declarations

Although forward declarations are powerful, they have some limitations:

- You can only use **pointers or references** to an incomplete type. You cannot access its members or create instances until the full definition is known.
- If a function or class method uses the type by value, you must include its full definition.
- Forward declarations can become invalid if the actual definition changes in a way that conflicts with the declaration (e.g., changing a class to a struct).

Best Practices

- Use forward declarations in header files whenever you only need to reference a type, not define it.
- Include the full definition (`#include`) in the source file where the type's members are accessed.
- Avoid unnecessary forward declarations that clutter headers.
- Always prefer forward declarations over including large header files in performance-critical builds.

Benefits in Large Projects

In large C++ programs, reducing header inclusion dependencies can significantly decrease build times and improve modularity. This is especially important when hundreds of source files depend on a common header file. By relying on forward declarations, developers can limit the impact of changes to specific components.

For example, if a header file defines several classes but other modules only need pointers to them, using forward declarations ensures that changes to one class definition do not trigger unnecessary recompilation of unrelated modules.

Summary

Forward declarations are a fundamental tool in professional C++ development. They allow programmers to:

- Break circular dependencies.
- Reduce compile-time dependencies.

- Simplify header management.
- Keep interfaces clean and maintainable.

By understanding and correctly applying forward declarations, C++ developers can build more efficient, modular, and scalable software systems.

Precompiled Headers (PCH)

When developing large C++ projects, one of the most significant challenges is the long compilation time caused by repeatedly including and processing the same header files across multiple source files. The C++ compilation model requires that every source file (`.cpp`) be compiled independently, which means that common header files are parsed and compiled repeatedly. To address this inefficiency, most modern compilers support the concept of **Precompiled Headers (PCH)**.

Concept of Precompiled Headers

A **precompiled header** is a mechanism that allows the compiler to process a set of commonly used headers once, store the intermediate compiled state, and then reuse this state across multiple source files. This significantly reduces compilation time, especially in projects with large or complex header hierarchies.

The main idea is simple: Instead of parsing and compiling the same header files repeatedly, the compiler compiles them once into a special binary file (the precompiled header) and reuses it for subsequent compilations.

For example, if every source file in your project includes headers such as:

```
#include <iostream>
#include <vector>
#include <string>
#include <map>
```

the compiler can precompile these headers into a single precompiled header file, so that each source file can simply reference this precompiled version instead of reprocessing them every time.

Advantages of Using Precompiled Headers

Using precompiled headers offers several advantages:

1. **Reduced Compilation Time:** Large headers such as the Standard Template Library (STL) or Windows API headers are expensive to parse. Compiling them once and reusing the compiled result can drastically reduce build times.
2. **Consistency Across Source Files:** All source files that use the same precompiled header share the same set of included headers, which reduces the likelihood of subtle inconsistencies or mismatched macro definitions.
3. **Improved Developer Productivity:** Faster compilation means faster iteration and testing, allowing developers to be more productive during the development cycle.

How Precompiled Headers Work

When a compiler processes source code, it first parses and expands all included header files into a large translation unit before compilation. For large projects with many dependencies, this process becomes slow because the same headers are reprocessed multiple times.

With precompiled headers, the compiler performs the following steps:

1. Parse the specified header file and all its includes once.
2. Store the parsed state (symbol tables, type information, macros, etc.) in a binary precompiled header file, typically with an extension such as `.pch` or `.gch`.
3. For subsequent compilations, the compiler loads this precompiled state instead of re-parsing the headers.

Creating and Using a Precompiled Header

The typical workflow for using precompiled headers in a project consists of two steps: creating the precompiled header and then referencing it in other source files.

Step 1: Create the Precompiled Header File

Create a header file (for example, `pch.h`) containing all the commonly used headers:

```
// File: pch.h
#ifndef PCH_H
#define PCH_H

#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <algorithm>

#endif
```

Then, create a corresponding source file (`pch.cpp`) that includes this header:

```
// File: pch.cpp
#include "pch.h"
```

Compile this file first to generate the precompiled header. The actual command depends on the compiler being used:

- **Microsoft Visual C++:** `cl /Yc"pch.h" pch.cpp`
- **GCC or Clang:** `g++ -x c++-header pch.h -o pch.h.gch`

Step 2: Use the Precompiled Header in Other Files

In other source files, include the precompiled header at the top:

```
// File: main.cpp
#include "pch.h"

int main() {
    std::vector<int> v = {1, 2, 3, 4};
    std::cout << "Vector size: " << v.size() << std::endl;
    return 0;
}
```

Then compile using the precompiled header:

- **Microsoft Visual C++:** `cl /Yu"pch.h" main.cpp`
- **GCC or Clang:** `g++ main.cpp -include pch.h -o main`

Best Practices for Precompiled Headers

Although precompiled headers can greatly improve build performance, they must be used carefully to avoid dependency or maintenance issues. Some best practices include:

- **Include only stable headers:** Precompiled headers should contain only headers that change infrequently. If a precompiled header changes, all dependent files must be recompiled.
- **Avoid project-specific headers:** Limit precompiled headers to standard and external library headers. Internal headers under active development should be excluded.
- **Include it first:** Always include the precompiled header as the first include in every source file to ensure consistency.
- **Maintain clean dependencies:** Keep the precompiled header organized and minimal. Avoid adding unnecessary includes.
- **Regenerate when environment changes:** Rebuild the precompiled header if the compiler version, build options, or platform changes.

Limitations and Potential Issues

While precompiled headers can speed up compilation, they also introduce certain challenges:

1. **Recompilation Overhead:** Any modification to the precompiled header file or its included headers forces a full recompilation of all dependent source files.
2. **Compiler-Specific Implementation:** The mechanism and file format for precompiled headers are compiler-dependent, reducing portability across toolchains.
3. **Inconsistent States:** If macros or preprocessor definitions differ across files, the precompiled header might produce inconsistent results.
4. **Hidden Dependencies:** Overuse of precompiled headers can make source dependencies less visible, leading to code that silently depends on unrelated includes.

Practical Recommendations

- Use precompiled headers in large projects with many external library includes (for example, GUI applications or frameworks such as Qt or MFC).
- Keep the precompiled header content as stable as possible.
- Periodically review and refactor your precompiled header to ensure it contains only necessary headers.
- Combine precompiled headers with modular code organization and incremental builds for maximum performance.

Example Build Times

For a project containing hundreds of source files and large standard library includes, enabling precompiled headers can reduce compilation time by as much as 50–80%, depending on the compiler and hardware. The performance gain grows with the number of files that include the same heavy headers.

Summary

Precompiled headers are an essential optimization mechanism in modern C++ build systems. They help:

- Reduce repetitive parsing of common headers.
- Shorten total build time.
- Enforce consistent include environments across compilation units.

While they introduce some complexity in build management, careful use of precompiled headers can dramatically improve the efficiency of large-scale C++ development. They are especially useful in projects that rely on heavy use of the C++ Standard Library or platform-specific APIs.

Static and Dynamic Libraries in C++

As software projects grow larger, it becomes inefficient and impractical to compile all source code into a single executable file. Instead, C++ developers organize code into reusable components known as **libraries**. Libraries provide a way to share and reuse code across multiple programs without duplicating source files. In C++, there are two main types of libraries: **static libraries** and **dynamic (shared) libraries**. Understanding how these libraries are created and linked is essential for effective modular software development.

What Is a Library?

A **library** is a collection of precompiled object files that provide functionality which can be linked to other programs. Instead of recompiling common code for every program, libraries allow developers to compile the code once and reuse it many times.

For example, the C++ Standard Library (`libstdc++`) provides precompiled implementations of data structures, algorithms, and utilities that all C++ programs can use without needing to recompile them.

C++ supports two main library linking methods:

- **Static linking** — the library code is copied directly into the executable during compilation.
- **Dynamic linking** — the library code remains separate and is loaded into memory at runtime.

Static Libraries

A **static library** is a collection of object files that are linked directly into the program's executable at compile time. On Unix-like systems, static libraries typically have the extension `.a` (archive), while on Windows they use the extension `.lib`.

When a program is statically linked with a library, the linker copies all required functions and data from the library into the final executable file. This results in a larger executable size but a self-contained program that does not depend on external library files at runtime.

Creating a Static Library

To create a static library in C++:

1. Write one or more source files containing the implementation.
2. Compile them into object files using the compiler.
3. Archive them into a static library file.

For example:

```cpp
// File: math_utils.cpp
#include "math_utils.h"

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

```
// File: math_utils.h
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b);
int multiply(int a, int b);

#endif
```

Compile and create the library using the following commands:

```
g++ -c math_utils.cpp       // Compile to object file
ar rcs libmath.a math_utils.o // Create static library
```

This creates a static library named `libmath.a`.

Linking a Static Library

To link the static library to another program:

```
// File: main.cpp
#include <iostream>
#include "math_utils.h"

int main() {
    std::cout << "Sum: " << add(2, 3) << std::endl;
    std::cout << "Product: " << multiply(2, 3) << std::endl;
    return 0;
}
```

Compile and link:

```
g++ main.cpp -L. -lmath -o app
```

Here, `-L.` tells the compiler to search for libraries in the current directory, and `-lmath` links with `libmath.a`.

Advantages of Static Libraries

- Faster program startup since all code is already in memory.
- No external dependencies at runtime.
- Easier distribution since only a single executable is needed.

Disadvantages of Static Libraries

- Larger executable file size because all library code is embedded.
- Updating the library requires recompiling all dependent programs.
- Memory waste if multiple programs include the same static library code.

Dynamic (Shared) Libraries

A **dynamic library** (also called a **shared library**) is not copied into the executable at compile time. Instead, it is linked at runtime when the program is executed. On Unix-like systems, shared libraries use the `.so` (shared object) extension, and on Windows they use `.dll` (dynamic-link library).

When a program using a dynamic library runs, the operating system loads the library into memory and connects its functions to the program through a process called **dynamic linking**.

Creating a Dynamic Library

To create a shared library, compile the code with the `-fPIC` (Position Independent Code) and `-shared` flags:

```
g++ -fPIC -c math_utils.cpp     // Generate position-independent object file
g++ -shared -o libmath.so math_utils.o // Create shared library
```

Linking a Dynamic Library

To link against a dynamic library:

```
g++ main.cpp -L. -lmath -o app
```

When running the program, ensure the shared library is in the library search path. You can run the program as follows:

```
export LD_LIBRARY_PATH=.
./app
```

Advantages of Dynamic Libraries

- Smaller executable size since library code is not copied.
- Easier updates — only the shared library needs to be replaced.
- Reduced memory usage when multiple programs share the same library in memory.

Disadvantages of Dynamic Libraries

- Slightly slower startup time due to runtime linking.
- Requires proper configuration of the library search path.
- Compatibility issues can arise if the library interface changes.

Static vs Dynamic Linking: Comparison

Feature	Static Library	Dynamic Library
Linking Time	Compile time	Runtime
File Extension	`.a` (Unix), `.lib` (Windows)	`.so` (Unix), `.dll` (Windows)
Executable Size	Larger	Smaller
Update Flexibility	Requires recompilation	Can be updated independently
Memory Usage	Duplicated per executable	Shared among processes
Performance	Slightly faster	May incur minor runtime overhead
Dependency	None (self-contained)	Depends on external files

When to Use Each Type

- Use **static libraries** for small programs, embedded systems, or environments where self-contained executables are preferred.
- Use **dynamic libraries** for large applications, plugins, or systems where code reuse and modular updates are important.

Best Practices

- Keep library interfaces stable to avoid breaking compatibility.
- Use versioning for shared libraries to manage updates safely.
- Avoid mixing static and dynamic linking for the same library unless necessary.
- Always document the dependencies of your program when using dynamic libraries.

Make and CMake

Make

A Makefile is a special text file used in Unix-like operating systems (including Linux) and other development environments to automate the build process of a project. It specifies a set of rules and dependencies for compiling source code files, linking them into executables, and performing other project-related tasks. Makefiles are often used in C and C++ projects, but they can be applied to other programming languages and projects as well.

Here's a basic overview of how a Makefile works and how to create one:

Makefile Structure

A Makefile typically consists of a set of rules, each defined in the following format:

```
target: dependencies
    commands
```

1. target: The target is the name of the file or action you want to build. It could be an executable file, an object file, or even just a label for a particular task.
2. dependencies: Dependencies are the files or prerequisites required to build the target. If any of the dependencies change, the target is rebuilt.
3. commands: Commands are the shell commands or instructions to execute in order to build the target from its dependencies.

Example Makefile

Here's a simple example of a Makefile for a C++ program with two source files, main.cpp and utils.cpp, and one header file, utils.h:

```
CXX = g++
CXXFLAGS = -std=c++11 -Wall
SRCDIR = src
BINDIR = bin

TARGET = myapp
SOURCES = $(SRCDIR)/main.cpp $(SRCDIR)/utils.cpp
OBJECTS = $(SOURCES:$(SRCDIR)/%.cpp=$(BINDIR)/%.o)

$(BINDIR)/$(TARGET): $(OBJECTS)
  $(CXX) $(CXXFLAGS) -o $@ $(OBJECTS)

$(BINDIR)/%.o: $(SRCDIR)/%.cpp
  $(CXX) $(CXXFLAGS) -c -o $@ $<

clean:
  rm -f $(BINDIR)/*.o $(BINDIR)/$(TARGET)
```

In this example:

- CXX specifies the C++ compiler to use (in this case, g++).
- CXXFLAGS specifies compiler flags, such as the C++ version and warning options.
- SRCDIR and BINDIR define source code and binary directories, respectively.
- TARGET specifies the name of the final executable.
- SOURCES lists the source files.
- OBJECTS lists the corresponding object files.
- The first rule ($(BINDIR)$/(TARGET)) specifies how to build the final executable from object files.
- The second rule specifies how to compile each source file into an object file.
- The clean rule allows you to clean up generated files.

Using the Makefile

To build your project using the Makefile, navigate to the directory containing the Makefile and your source code files, and then run make:

```
make
```

This will compile your program and create the executable. You can also use specific targets, such as clean, to perform other tasks:

```
make clean
```

CMake

CMake is an open-source cross-platform build system and project management tool widely used in the C and C++ programming communities (although it supports many other languages as well). CMake simplifies the process of configuring, building, testing, and packaging software projects. It generates build files and scripts (e.g., Makefiles, Visual Studio project files) for various build tools and platforms, allowing developers to work on their projects across different operating systems with ease.

Here are the key features and concepts of CMake:

- Cross-Platform Build System: CMake is designed to work across different platforms, including Unix/Linux, macOS, and Windows. It generates platform-specific build files tailored to the developer's system.
- CMakeLists.txt: The core of CMake-based projects is the CMakeLists.txt file. This text file describes project details, dependencies, source files, and build instructions. It's used to configure the build process.
- Out-of-Source Builds: CMake encourages out-of-source builds, where build artifacts (e.g., object files, executables) are generated in a separate directory from the source code. This keeps the source directory clean and allows for multiple build configurations without clutter.
- Multiple Generators: CMake can generate build files for various build systems, including Makefiles, Ninja, Visual Studio, and Xcode. Developers can choose the generator that best suits their development environment.
- Dependency Management: CMake provides mechanisms to find and configure dependencies, including libraries, header files, and external projects. This simplifies the process of integrating third-party code.
- Configurable Builds: CMake allows developers to configure build options and features through command-line arguments or a graphical user interface (CMake GUI). This makes it easy to adapt the build process to different requirements.
- Testing: CMake includes built-in support for testing. It allows you to define test cases, run tests, and generate test reports as part of the build process.

- Packaging: CMake supports packaging your project for distribution. It can generate installer packages, compressed archives, or other distribution formats.
- Integration with IDEs: Many integrated development environments (IDEs), including Visual Studio, CLion, and Qt Creator, have built-in support for CMake. This simplifies project setup and development.
- Extensibility: CMake can be extended with custom CMake modules and scripts to support specific project requirements.

Here's a simplified example of a CMakeLists.txt file for a C++ project:

```
cmake_minimum_required(VERSION 3.10)
project(MyProject)

set(CMAKE_CXX_STANDARD 17)

# Define source files
set(SOURCES
    main.cpp
    utils.cpp
)

# Define the executable
add_executable(myapp ${SOURCES})

# Specify dependencies (e.g., libraries)
target_link_libraries(myapp
    ${DEPENDENCY_LIBRARIES}
)

# Configuration options (e.g., compile flags)
target_compile_options(myapp PRIVATE
    -Wall
)
```

To build a project using CMake, you would typically run the following commands in your project directory:

```
mkdir build  # Create a build directory (out-of-source build)
cd build
cmake ..      # Generate build files
make         # Build the project (or use the appropriate build
   tool)
```

Further Reading in C++

C++ is a deep and evolving language that blends procedural, object-oriented, and generic programming paradigms. Mastering its core features—such as classes, templates, and move semantics—is only the first step toward writing efficient, safe, and maintainable software. To continue developing expertise, programmers should explore the advanced areas of the language and consult authoritative references.

Recommended Topics for Continued Study

The following topics build upon the foundations presented in this text and represent the next stages of professional C++ development:

- **Templates and Generic Programming:** Learn about template specialization, template metaprogramming, and concepts introduced in C++20 for type-safe generic code.
- **Advanced Memory Management:** Explore allocators, smart pointer customization, and memory resource management using the `<memory>` and `<memory_resource>` libraries.
- **Concurrency and Multithreading:** Study the C++ thread library, synchronization primitives, atomics, and lock-free data structures for high-performance parallel applications.
- **The Standard Template Library (STL):** Deepen understanding of algorithms, iterators, and container internals to write more idiomatic and efficient C++.
- **Modern C++ Features (C++11 to C++23):** Review lambda expressions, constexpr functions, structured bindings, ranges, modules, and coroutines.
- **Design Patterns and Idioms:** Learn classical object-oriented patterns (e.g., Singleton, Factory, Observer) and C++-specific idioms such as RAII, Pimpl, and CRTP.
- **Compile-Time Programming:** Investigate `constexpr`, `consteval`, and template metaprogramming for compile-time computation and code generation.
- **Error Handling and Contracts:** Study modern techniques for error propagation, exception safety, and the proposed contract programming model in future standards.

Influential Books and References

A number of books are considered classics in professional C++ development. These works provide practical insights into both language features and design philosophy.

- *The C++ Programming Language* by Bjarne Stroustrup — the definitive reference by the language's creator.
- *Effective Modern C++* by Scott Meyers — a concise guide to best practices in C++11 and C++14.
- *More Effective C++* and *Effective STL* by Scott Meyers — deeper dives into efficiency and library use.
- *C++ Primer* by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo — an accessible yet comprehensive introduction for intermediate learners.
- *A Tour of C++ (2nd Edition)* by Bjarne Stroustrup — a modern overview of C++17 and later.
- *C++ Concurrency in Action* by Anthony Williams — an in-depth exploration of multithreading and concurrency.
- *Design Patterns: Elements of Reusable Object-Oriented Software* by Gamma et al. — foundational material for object-oriented design, applicable to C++.

Online Resources

In addition to books, the following online materials provide up-to-date coverage of evolving C++ standards and best practices:

- The **ISO C++ Committee** website (`https://isocpp.org`) — the official portal for news, proposals, and standardization updates.
- The **C++ Reference** site (`https://en.cppreference.com`) — a comprehensive, community-maintained technical reference for every standard library feature.
- The **CppCoreGuidelines** project (`https://github.com/isocpp/CppCoreGuidelines`) — a collection of modern C++ coding guidelines curated by Bjarne Stroustrup and Herb Sutter.
- The **C++ Weekly** video series by Jason Turner — concise explanations of advanced C++ topics and new standard features.

Looking Ahead

As the C++ language continues to evolve through C++20, C++23, and beyond, developers should adopt a mindset of continuous learning. Understanding how new features interact with existing paradigms—especially in templates, concurrency, and compile-time computation—will ensure code that is both performant and maintainable. Mastery of C++ is a lifelong pursuit that rewards curiosity, experimentation, and adherence to best practices.

Suggested Projects

Project 1: Analog Clock in C++

Overview

This project focuses on designing and implementing a fully functional analog clock in C++ using the principles of Object-Oriented Programming (OOP). The clock displays the current system time through three rotating hands: the hour, minute, and second hands. The output may be rendered in ASCII or through a graphical library such as SFML, SDL, wxWidget, or Qt.

Objectives

- Apply OOP principles such as abstraction, encapsulation, inheritance, and polymorphism.
- Model clock components as independent and reusable classes.
- Use `std::chrono` for time retrieval and updates.
- Implement geometric rotation to draw the clock hands correctly.
- Maintain a clean, modular, and extendable architecture.

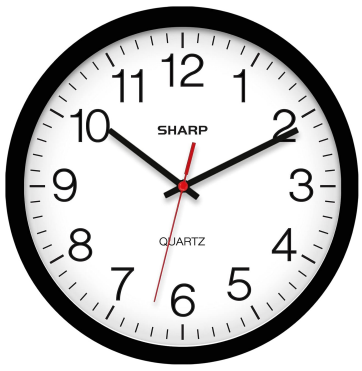


Figure 4: The clock project

System Components

Class `Time`

This class stores and updates the internal time state. It retrieves system time and exposes it to the rest of the clock.

Base Class `ClockHand`

This abstract class represents a generic clock hand. It stores:

- Hand length
- Angle of rotation

Derived classes override the method that computes the hand's angle:

- `HourHand`
- `MinuteHand`
- `SecondHand`

Class `AnalogClock`

This class aggregates the components of the clock:

- `Time time;`
- `HourHand hour;`
- `MinuteHand minute;`
- `SecondHand second;`

It is responsible for updating time, recomputing angles, and invoking the renderer.

Class `ClockRenderer`

An abstract class that defines functions to render clock elements. Implementations include:

- `AsciiClockRenderer`
- `SfmlClockRenderer`

Rotating a Point to Draw Clock Hands

To draw a clock hand, begin with a point that initially points straight upward:

$$(x, y) = (0, -\text{length})$$

To rotate a point by an angle θ around the origin, use the standard rotation equations:

$$x' = x\cos\theta - y\sin\theta$$

$$y' = x\sin\theta + y\cos\theta$$

In C++:

```
double angleRad = angleDegrees * M_PI / 180.0;

double x = 0;
double y = -handLength;

double xr = x * cos(angleRad) - y * sin(angleRad);
double yr = x * sin(angleRad) + y * cos(angleRad);

// Shift to clock center:
double finalX = centerX + xr;
double finalY = centerY + yr;
```

The rotated point becomes the endpoint of the hand.

Computing Hand Angles

The orientation of the hands is computed as follows:

$$\theta_{\text{second}} = \text{seconds} \times 6^\circ$$

$$\theta_{\text{minute}} = \text{minutes} \times 6^\circ + \frac{\text{seconds}}{10}$$

$$\theta_{\text{hour}} = \text{hours} \times 30^\circ + \frac{\text{minutes}}{2}$$

These angles are then converted into radians and used in the rotation equations.

Expected Learning Outcomes

By completing this project, the learner will:

- Develop a strong understanding of OOP design.
- Learn to manipulate and display time using `<chrono>`.
- Apply geometric transformations to draw rotated shapes.
- Create a renderer abstraction using polymorphism.
- Separate program logic from graphical or textual output.

Project 2: Slide Show Application in C++

Overview

This project focuses on developing a simple yet functional slide show application using C++. The program loads a set of image files from the file system and displays them one at a time, allowing the user to navigate forward and backward through the collection. The project emphasizes the use of Object-Oriented Programming (OOP) principles, modular design, file handling, and basic rendering techniques. The application may render images in ASCII format in the terminal, or through a graphical library such as SFML, SDL, or Qt.

Objectives

- Practice OOP concepts such as encapsulation, abstraction, composition, and polymorphism.
- Work with file system operations to load images from a directory.
- Learn how to manage collections of objects using standard containers.
- Implement a clean separation between logic (slideshow management) and output (rendering).
- Introduce optional transition effects for changing slides.

System Components

Class `Image`

This class stores information about a single image. Depending on the rendering method, it may:

- Store ASCII pixel data (for a console application).
- Store graphical texture data (for SFML or SDL).

It is responsible for loading the file from disk and providing access to its contents.

Class `SlideShow`

The core controller that manages the sequence of images. It maintains:

- A `std::vector` of loaded `Image` objects.
- The index of the currently displayed slide.

It exposes methods such as:

- `nextSlide()`
- `previousSlide()`
- `getCurrentSlide()`

Class `TransitionEffect`

An abstract base class representing a transition between two slides. It declares a function:

```
virtual void apply(const Image& from, const Image& to) = 0;
```

Possible derived classes:

- `FadeEffect`
- `SlideLeftEffect`
- `NoTransition` (default)

Class `Renderer`

This abstract class is responsible for drawing an image on the screen. It defines:

```
virtual void draw(const Image& img) = 0;
```

Derived renderers may include:

- `AsciiRenderer` for console-based output.
- `SfmlRenderer` or `SdlRenderer` for graphical output.

Program Workflow

When the program starts:

1. The user provides a directory path.
2. All image files in that directory are loaded into `Image` objects.
3. The first slide is rendered on the screen.
4. The user navigates using keyboard keys (e.g., Left/Right arrows).
5. Optional transition effects are applied between slides.

Possible Extensions

- Add image captions extracted from filenames or metadata.
- Add automatic slide playback with a fixed time interval.
- Add keyboard shortcuts for selecting transition types.
- Add scaling to fit images to different window sizes.

Project 3: Binary Parser in C++

Overview

The Binary Parser project aims to design and implement a C++ program capable of reading, interpreting, and analyzing the contents of binary files with a predefined internal structure. This project introduces students to low-level data representation, binary file operations, endianness handling, and techniques commonly used in parsers for compilers, network protocols, and operating systems.

Objectives

The main objectives of this project are as follows:

- Understand how binary data is stored, aligned, and accessed.
- Practice reading and writing files using `std::ifstream` and `std::ofstream` in binary mode.
- Parse structured data records from a binary stream.
- Validate file integrity through mechanisms such as magic numbers or checksums.
- Present the parsed data in clear, human-readable form.

Binary File Format Specification

The project uses a simple custom binary file format. A file begins with a header followed by a sequence of records. The header structure is shown in Table 2.

Field	Size (bytes)	Description
Magic Number	4	A constant value identifying the file type (e.g. `0xDEADBEEF`).
Version	2	File format version.
Record Count	4	Number of records contained in the file.

Table 2: Binary header format

Each record has a fixed structure consisting of:

- 4-byte integer ID
- 8-byte timestamp
- 2-byte status code
- 32-byte fixed-length text field

Core Functionalities

The parser will implement the following core features:

1. **Binary Loader:** Opens the file in binary mode, reads the header, and validates the magic number and version.
2. **Record Parser:** Extracts each record from the binary stream, converts byte order when needed, and stores the data in C++ structures.
3. **Error Handling:** Detects corrupted or incomplete files, invalid headers, and unexpected end-of-file conditions.
4. **Data Output:** Displays the parsed records in formatted text and optionally exports them to CSV or JSON for further analysis.

Optional Extensions

To enrich the project, several advanced features may be added:

- Support for variable-length fields using length-prefixed encoding.
- Checksum or hash verification to detect corrupted files.
- A companion binary writer to generate valid test files.
- Hex-dump visualization for debugging.
- Command-line options for selective parsing or exporting.
- Memory-mapped file parsing for handling very large binary files.

Project 4: Puzzle Game: Mind Maze

Project Overview

The **Mind Maze** project is a console-based puzzle game developed in **C++**, designed to challenge the player's logic and problem-solving skills. The game consists of a series of progressively difficult puzzles, where the player must arrange, match, or navigate elements to complete each level. The project focuses on applying **Object-Oriented Programming (OOP) principles** to create a modular, maintainable, and extensible codebase.

Objectives

- Design a scalable game architecture using OOP concepts.
- Implement multiple puzzle types, such as sliding tiles, number puzzles, and logic grids.
- Demonstrate encapsulation, inheritance, and polymorphism through class design.
- Provide an interactive console interface for gameplay and level selection.
- Enable easy expansion by adding new puzzles or game mechanics.

Key Features

- **Menu System:** Start a new game, continue a saved game, or view high scores.
- **Multiple Puzzle Levels:** Increasing difficulty, including sliding tile puzzles, number-matching puzzles, and maze navigation challenges.
- **Player Progress Tracking:** Records scores, attempts, and completion times.
- **Hints System:** Optional hints to assist players with challenging puzzles.
- **Save and Load Functionality:** Allows players to resume games.

OOP Design and Implementation

The project applies **core OOP principles** as follows:

Classes and Objects

- `Game` – Main class controlling game flow, menus, and user interactions.
- `Puzzle` – Abstract base class representing a generic puzzle.
- `SlidingPuzzle`, `NumberPuzzle`, `MazePuzzle` – Derived classes implementing specific puzzle logic.
- `Player` – Tracks player name, progress, and statistics.
- `ScoreBoard` – Maintains high scores and player rankings.

Encapsulation

All class data members are private or protected. Public member functions provide controlled access to modify and retrieve data. For example, the `Player` class provides `getScore()` and `updateScore(int points)` methods.

Inheritance

`Puzzle` acts as a base class. Specific puzzle types inherit common properties such as `display()`, `isSolved()`, and `reset()` while implementing specialized logic.

Polymorphism

Virtual functions allow dynamic binding for `display()` and `checkSolution()` across different puzzle types. This enables the game engine to manage different puzzles uniformly.

Abstraction

Users interact with simple methods like `play()` and `solvePuzzle()` without needing to know the internal puzzle logic.

Composition

The `Game` class contains objects of `Puzzle`, `Player`, and `ScoreBoard`, demonstrating has-a relationships.

Project 5: Chat Program: TCP Talk

Project Overview

The **TCP Talk** project is a console-based chat application developed in **C++** using **TCP sockets**. The program allows multiple clients to connect to a server and exchange messages in real time. The project emphasizes **Object-Oriented Programming (OOP) principles** to achieve modularity, maintainability, and extensibility, making it easy to add new features such as private messaging, group chats, or message history.

Objectives

- Implement a client-server chat application using TCP sockets.
- Apply OOP concepts to separate responsibilities and improve code organization.
- Demonstrate encapsulation, inheritance, and polymorphism through class design.
- Provide a scalable framework to support multiple clients and future enhancements.

Key Features

- **Server-Client Architecture:** Server handles multiple clients; clients can send and receive messages in real-time.
- **Modular Design:** Separate classes for server, client, and message handling.
- **Multi-Client Support:** Server supports multiple simultaneous client connections.
- **Clean Console Interface:** Users can send messages and view incoming messages asynchronously.
- **Extensible Framework:** Easy to add new chat functionalities, such as private messages or chat rooms.

OOP Design and Implementation

Classes and Objects

- `Server` – Manages TCP socket setup, listens for client connections, and broadcasts messages.
- `Client` – Connects to the server, sends messages, and receives messages from the server.
- `Message` – Represents a chat message with sender information and content.
- `User` – Optional class to store client-related information such as username, status, or connection info.

Encapsulation

Data members such as socket descriptors and buffers are private. Public methods provide controlled access, e.g., `connectToServer()`, `sendMessage()`, `receiveMessage()`.

Inheritance

Derived classes could implement different types of clients or messages (e.g., `AdminClient`, `SystemMessage`).

Polymorphism

Virtual functions allow different client types or message types to handle sending and receiving differently while using the same interface.

Composition

The `Server` class contains multiple `Client` objects representing connected users.

Sample TCP Socket Communication

Server Code

```
#include <iostream>
#include <string>
#include <thread>
#include <vector>
#include <arpa/inet.h>
#include <unistd.h>

class ClientHandler {
private:
    int clientSocket;
public:
    ClientHandler(int socket) : clientSocket(socket) {}
    void handle() {
        char buffer[1024];
        while (true) {
            int bytes = read(clientSocket, buffer, sizeof(buffer) - 1);
            if (bytes <= 0) break;
            buffer[bytes] = '\0';
            std::cout << "Client: " << buffer << std::endl;
        }
        close(clientSocket);
    }
};
```

```
class Server {
private:
    int serverSocket;
    std::vector<std::thread> threads;
public:
    void start(int port) {
        serverSocket = socket(AF_INET, SOCK_STREAM, 0);
        sockaddr_in serverAddr{};
        serverAddr.sin_family = AF_INET;
        serverAddr.sin_addr.s_addr = INADDR_ANY;
        serverAddr.sin_port = htons(port);

        bind(serverSocket, (sockaddr*)&serverAddr, sizeof(serverAddr));
        listen(serverSocket, 5);

        std::cout << "Server started on port " << port << std::endl;

        while (true) {
            sockaddr_in clientAddr{};
            socklen_t clientLen = sizeof(clientAddr);
            int clientSocket = accept(serverSocket, (sockaddr*)&clientAddr, &clientLe
            threads.emplace_back(&ClientHandler::handle, ClientHandler(clientSocket))
        }

        for (auto &t : threads) t.join();
    }
};

int main() {
    Server server;
    server.start(8080);
    return 0;
}
```

Client Code

```
#include <iostream>
#include <string>
#include <arpa/inet.h>
#include <unistd.h>

class Client {
private:
    int sock;
public:
    void connectToServer(const std::string &ip, int port) {
```

```
        sock = socket(AF_INET, SOCK_STREAM, 0);
        sockaddr_in serverAddr{};
        serverAddr.sin_family = AF_INET;
        serverAddr.sin_port = htons(port);
        inet_pton(AF_INET, ip.c_str(), &serverAddr.sin_addr);

        if (connect(sock, (sockaddr*)&serverAddr, sizeof(serverAddr)) < 0) {
            std::cerr << "Connection failed!\n";
            exit(1);
        }

        std::cout << "Connected to server.\n";
    }

    void chat() {
        std::string msg;
        while (true) {
            std::getline(std::cin, msg);
            send(sock, msg.c_str(), msg.size(), 0);
        }
    }
};

int main() {
    Client client;
    client.connectToServer("127.0.0.1", 8080);
    client.chat();
    return 0;
}
```